How to Authenticate a Non-Deterministic Computation

Shift-Hiding Functions, Compressed LWE Sampling, Broadcast Encryption, and Obfuscation

Damiano Abram* Giulio Malavolta† Lawrence Roy‡

Abstract

We propose a new method to construct homomorphic authentication codes supporting the evaluation of *non-deterministic* computations, extending the celebrated homomorphic lattice encodings [Boneh et al., Eurocrypt 2014]. Our approach relies on the hardness of the decomposed learning with errors problem (LWE), a recently introduced modification of Regev's LWE assumption.

We then use this new technical tool to make progress on several open problems in the literature. Specifically, we obtain:

- A constrained pseudorandom function (PRF), where the evaluation of the PRF on the master key does not depend on the complexity of the constraint, except for its circuit depth.
- A way to securely compress and re-expand LWE samples in the plain model.
- An adaptively secure broadcast encryption scheme, with ciphertext and secret keys growing poly-logarithmically with the size of the encrypted set.
- A pseudorandom obfuscation for all puncturable PRFs, additionally assuming the existence of sub-exponentially secure indistinguishability obfuscation (iO).

None of the above mentioned primitives was known to exist from lattice assumptions. As a bonus result, we also obtain a conceptually simple and direct heuristic construction of iO based on lattice techniques, which is not based on the function encryption-to-iO paradigm. We provide evidence that this approach can be used to build provably secure obfuscation for simple functionalities such as sampling lattice preimages using a hidden trapdoor.

*University of Edinburgh
†Bocconi University
‡Aarhus University, IBM Research Zürich

Contents

1 Introduction **2**
- 1.1 Our Results . . . 3
- 1.2 Technical Outline: Non-Deterministic Encodings . . . 6
- 1.3 Technical Outline: Applications . . . 10

2 Preliminaries **16**
- 2.1 Linear Algebra . . . 17
- 2.2 Discrete Gaussians and Lattices . . . 17
- 2.3 Homomorphic Encodings . . . 19

3 Non-Deterministic Encodings **22**
- 3.1 Definition of the Encodings . . . 22
- 3.2 Homomorphic Operations . . . 25
- 3.3 Flattening . . . 29
- 3.4 Putting Things Together . . . 31

4 Laconic Shift-Hiding Pseudorandom Function **33**
- 4.1 Laconic Function Evaluation . . . 33
- 4.2 Our Scheme . . . 34
- 4.3 Intractable Laconic Shift-Hiding PRF . . . 37
- 4.4 A Heuristic Approach to Obfuscation . . . 38

5 Compressing LWE Samples **38**
- 5.1 The Compression Algorithm . . . 41
- 5.2 Derandomized Indistinguishability . . . 43

6 Broadcast Encryption **48**
- 6.1 Definition . . . 48
- 6.2 Our Scheme . . . 49

7 Pseudorandom Obfuscation **52**

1 Introduction

Lattice-based cryptography [Ajt96, Reg05] has seen a remarkable success over the past decade, in part due to its conjectured *post-quantum* security, in part due to vast number of new cryptographic primitives enabled by lattices. A central technical toolkit for lattice-based cryptography is the ability of *computing on encrypted data*, which is the cornerstone of primitives such as fully-homomorphic encryption [Gen09, BV11], attribute-based encryption [GVW13, BGG+14], laconic function evaluation [QWW18], traitor tracing [GKW18], or succinct arguments [CJJ22], to mention a few. A central technical tool in this area is

the notion of *lattice encodings*, introduced in [BGG+14]. An encoding of a message $\mathbf{x}$ with respect to a matrix $\mathbf{M}$ is defined to be

$$\mathbf{s}^\intercal(\mathbf{M} - \mathbf{x}^\intercal \otimes \mathbf{G}) + \mathbf{e}^\intercal$$

where $\mathbf{G}$ is the gadget matrix [MP12] and $\mathbf{e}$ is a low-norm noise term. A useful perspective to take is to view such an encoding as a *message authentication code* on the vector $\mathbf{x}$. Indeed it is not hard to prove that producing an encoding of a different message under the same secret $\mathbf{s}$ leaks non-trivial information about $\mathbf{s}$. On the other hand, the learning with errors (LWE) [Reg05] assumption postulates that the above encoding is computationally indistinguishable from uniform, for a random $\mathbf{M}$.

However, the power of such encodings comes from their *homomorphic properties*. It is known that, for any function f, one can derive a low-norm matrix $\mathbf{H}_{f,\mathbf{x}}$ such that

$$(\mathbf{s}^\intercal(\mathbf{M} - \mathbf{x}^\intercal \otimes \mathbf{G}) + \mathbf{e}^\intercal)\,\mathbf{H}_{f,\mathbf{x}} \approx \mathbf{s}^\intercal(\mathbf{M}\mathbf{K}_f - f(\mathbf{x})^\intercal \otimes \mathbf{G}) + \mathbf{e}^\intercal$$

where $\mathbf{K}_f$ is a (low-norm) matrix that depends only of f. Given that the result is once again an encoding, by the same argument as before we can think of this process as an *authentication for the computation* of a given function f. Indeed, nearly all applications of these encodings, e.g., [GVW15b, GVW15a, BTVW17, QWW18, Tsa19, PS19, dCP23], implicitly use this property, one way or another.

A limitation of this authentication code is that the function f needs to be *deterministic*, in order for the matrix $\mathbf{K}_f$ to be fixed, which is important for applications. A natural question is whether one can authenticate *randomized* computation or, alternatively stated, authenticate a family of functions $\{f_r\}_r$ simultaneously in one encoding. The purpose of this work is to provide one possible answer to this question.

1.1 Our Results

Our main technical contribution is a new method to authenticate any polynomial-time computable function family $\{f_r\}_r$, where r can be understood as the randomness of the computation and comes from a (possibly) exponentially large domain. We abstract this notion into a new primitive, which we refer to as *non-deterministic encodings*, that allows one to (implicitly) *hash* lattice encodings of *all possible* bitstrings $r \in \{0,1\}^d$, homomorphically compute a function f over the hash, which can be later extended to obtain an encoding of f_r, for any r. Crucially, the computation of the matrix $\mathbf{M}_{f,r}$ used in this encoding can be split into two parts: first, we derive a matrix $\mathbf{M}_f$ that only depends on f, in time proportional to its circuit size; second, we derive $\mathbf{M}_{f,r}$ from $\mathbf{M}_f$ in time that depends only on the length of r and the depth δ of the computation. The circuit commitments of Wee [Wee25, Section 3.4] achieved similar properties, but the second step in the computation of $\mathbf{M}_{f,r}$ run in depth proportional to $\delta \cdot \mathsf{poly}(\log \delta)$. On the other hand, in our construction, the extension procedure requires depth $\log d \cdot \mathsf{poly}(\log \delta)$. This allows us to homomorphically run the extension over the non-deterministic encodings in a recursive fashion, enabling new cryptographic primitives, which prior to our work were unknown from lattices alone.

All of our results rely on the hardness of the *decomposed LWE* problem [AMR25a], a recent extension of Regev's LWE problem [Reg05]. In [AMR25a] it is shown that the hardness of the decomposed LWE problem is implied by the hardness of the succinct LWE problem [Wee24]. We discuss our applications in more details below and we refer the reader to Section 1.2 for a more detailed description of the non-deterministic encodings.

Application I: Laconic Shift-Hiding PRFs. A shift-hiding pseudorandom function (PRF) [PS18] is a regular PRF except that it has the additional ability to issue *shifted* keys for an arbitrary function f so that evaluating the shifted PRF on an input x results into the output "shifted" by $f(x)$. Furthermore, the shifted key must computationally hide f. Shift-hiding PRFs are known to imply constrained PRFs [BV15b, BLW17] and correlation-intractable hash functions [CGH98, LV22].

Using our non-deterministic encodings, we obtain the first *laconic* shift-hiding PRF, where the complexity of the (non-shifted) PRF evaluation is independent of the complexity of shifted function f, except for its circuit depth. This seemingly technical requirement turns out to be crucial for subsequent applications since it enables the shift function to recursively evaluate its own PRF, akin to bootstrapping in fully-homomorphic encryption [Gen09].

Theorem 1 (Informal)**.** *Assuming the hardness of the decomposed LWE problem, there exists a laconic shift-hiding PRF for all functions.*

Application II: Securely Compressing LWE Samples. We show a method to securely *compress-and-expand* LWE samples with respect to a matrix $\mathbf{X}$. To exemplify the usefulness of this tool, suppose that Alice holds an LWE secret $\mathbf{s}$ and wants to provide an LWE sample $\mathbf{s}^\intercal\mathbf{X} + \mathbf{e}^\intercal$ to Bob. Using our compression algorithm, Alice can send $\mathbf{s}^\intercal \cdot \mathsf{Hash}(\mathbf{X}) + \mathbf{e}^\intercal$ to Bob, where the size of $\mathsf{Hash}(\mathbf{X})$ is poly-logarithmic in the number of columns of $\mathbf{X}$. Finally, Bob can locally expand the sample sent by Alice to obtain an LWE sample with respect to $\mathbf{X}$.

For security, we show that if $\mathbf{X}$ is random, Alice's message is indistinguishable from random, under the hardness of decomposed LWE. This ensures that the secrecy of $\mathbf{s}$ is preserved. This is summarized by the following theorem.

Theorem 2 (Informal)**.** *There exist polynomial time algorithms* $(\mathsf{Hash}, \mathsf{Expand})$ *such that, if* $\mathbf{h}^\intercal \approx \mathbf{s}^\intercal \cdot \mathsf{Hash}(\mathbf{X})$*, then* $\mathsf{Expand}(\mathbf{h}, \mathbf{X}) \approx \mathbf{s}^\intercal \cdot \mathbf{X}$*. Moreover, if* $\mathbf{X}$ *is random, assuming the hardness of decomposed LWE,* $\mathbf{h}$ *is indistinguishable from random.*

Application III: Adaptively-Secure Broadcast Encryption. Broadcast encryption [FN94] allows one to broadcast the same message to a set of N participants. The notion requires the size of the ciphertext to be sublinear (ideally poly-logarithmic) in N. Traditionally, this notion entailed the presence of a trusted party that pre-distributes keys to the receivers, but recent work extended it to the *distributed* settings [BZ14] where each receiver samples its own key pair.

Arguably, the correct notion of security for broadcast encryption is *adaptive security*: The message should be hidden to the eyes of a distinguisher who can select the received set

adaptively, after seeing the keys of the receivers. Our main result is the first lattice-based (distributed) broadcast encryption that is adaptively secure in the plain model.

Theorem 3 (Informal)**.** *Assuming the hardness of the decomposed LWE problem, there exists an adaptively-secure (distributed) broadcast encryption scheme where the size of the public/private keys and ciphertexts is poly-logarithmic in N.*

Note that we achieve the stronger notion of distributed broadcast, but our scheme is also the first lattice-based (non-distributed) broadcast encryption satisfying the above security notion in the plain model. Our main technical contribution is an *unbounded* broadcast encryption scheme with essentially optimal parameters satisfying the weaker notion of semi-static security, then we invoke a recent compiler that generically transforms it into an adaptively-secure scheme in the standard model [HWW25]. We shall mention that the compilation process from [HWW25] introduces a large common reference string (linear in N), which is then inherited by our main construction. We leave open the problem of reducing the size of the common reference string to poly-logarithmic in N, while maintaining adaptive security.

Prior constructions achieving adaptive security (both in the distributed or traditional settings) required bilinear pairings [BW06, Wat09] or witness encryption [FWW23] or were only proven secure in the random oracle model [CHW25]. For context, witness encryption candidates based on lattices exist [Tsa22, VWW22] but the validity of the underlying assumptions has recently been called into question [BÜW24, DJM+25]. We also mention some recent lattice-based schemes satisfying the weaker notion of *selective security* [WW25], where the attacker has to commit to the challenge set ahead of time, prior to seeing the keys of the honest parties.

Application IV: Pseudorandom Obfuscation. An obfuscation scheme allows one to make a program unintelligible, without altering its functionality. The standard notion of security for an obfuscation scheme is indistinguishability obfuscation (iO) [BGI+01], which requires that the obfuscation of two functionally-equivalent programs must be computationally indistinguishable. A recent work [BDJ+25] proposes a variant of this notion, called *pseudorandom obfuscation* for keyed functions. The security of the primitive postulates that the obfuscation *itself* should be computationally indistinguishable from uniform bits, provided that the obfuscated function is also pseudorandom. However, in the same work [BDJ+25] the authors showed strong impossibility results for pseudorandom obfuscation for general functions.

This leaves open the question on what kind of functions admit a pseudorandom obfuscator. We make progress on this question, showing that all *puncturable* pseudorandom function admit a pseudorandom obfuscation.

Theorem 4 (Informal)**.** *Assuming the sub-exponential hardness of the decomposed LWE problem and the existence of sub-exponentially secure iO, there exists a pseudorandom obfuscation for all puncturable functions.*

A puncturable function [SW14] allows one to "puncture out" a point in its domain so that, even given the ability of evaluating the function on all other points, the output at the punctured point is computationally indistinguishable from uniform. Conditioned on the validity of the decomposed LWE assumption, this confirms a conjecture from [BDJ+25].

Application V: A Heuristic Obfuscation Scheme. As a bonus result, we show how our laconic shift-hiding PRF immediately yields a fully lattice-based construction of general purpose obfuscation for any circuit C as follows: Set the shift function to be $C(x) - \mathsf{Eval}(K, x)$, then the shifted evaluation algorithm returns

$$\mathsf{Eval}(K, x) + C(x) - \mathsf{Eval}(K, x) = C(x).$$

Note that the shift function is only well-defined because of the *laconic property*: The constrain depends on the evaluation algorithm, whose complexity potentially depends on the one of the constrain. The laconic property breaks this circular dependency.

Thus, we obtain a simple and direct obfuscation candidate that is not based on the functional encryption-to-obfuscation paradigm [AJ15, BV15a], and that is genuinely different from other lattice-based obfuscation candidates, e.g., [WW21, GP21, BDGM22].

We stress that, contrary to our laconic shift-hiding PRF, we do not know how to reduce the security of this construction to decomposed LWE (or any other standard assumption) and we leave a rigorous analysis of this scheme as a fascinating open problem. Nevertheless, we manage to show that for certain simple pseudorandom functionalities, the approach leads to provably secure obfuscation. This is the case, for instance, for circuits that compute lattice preimages using a hidden trapdoor.

1.2 Technical Outline: Non-Deterministic Encodings

We give a brief outline of the non-deterministic lattice encodings. We start by recalling the *compressed* lattice encoding of [AMR25a, AMR25b]. The compressed encoding of a vector $\mathbf{x}$ with respect to a low-norm matrix $\mathbf{M}$ and a secret $\mathbf{s}$ consists of a pair

$$\mathbf{c}^\intercal = \mathbf{s}^\intercal \mathbf{G} \cdot (\mathbf{M} - \mathsf{Hash}(\mathbf{x})^\intercal \otimes \mathbf{I}_n) + \mathbf{e}_0^\intercal, \qquad \mathbf{z}^\intercal = \mathbf{s}^\intercal \cdot \mathbf{G}\mathbf{Z} + \mathbf{e}_1^\intercal$$

where Hash is a deterministic and *linear* hash function, $\mathbf{Z}$ is a random decomposed LWE matrix provided as part of the public parameters [AMR25a], $\mathbf{G}$ is the gadget matrix with k rows and $n := k \log q$ columns [MP12], and $\mathbf{e}_0$ and $\mathbf{e}_1$ are noise terms. What makes such encodings useful is the fact that they can be expanded into a BGG^+ encoding of $\mathbf{x}$ [BGG+14]. That is, there exist efficiently computable low-norm matrices $\mathbf{P}$ and $\mathbf{Q}_\mathbf{x}$ (where $\mathbf{P}$ only depends on the public parameters and $\mathbf{Q}_\mathbf{x}$ also on $\mathbf{x}$) that satisfy the following *expansion equation*:

$$\mathbf{G}(\mathbf{M} - \mathsf{Hash}(\mathbf{x})^\intercal \otimes \mathbf{I}_n) \cdot \mathbf{P} + \mathbf{G}\mathbf{Z} \cdot \mathbf{Q}_\mathbf{x} = \mathbf{G}(\mathbf{M} \cdot \mathbf{P} - \mathbf{x}^\intercal \otimes \mathbf{I}_n).$$

Moreover, they admit homomorphic computation algorithms. That is, for a given function f, there exists a pair of efficient algorithms

- $\mathsf{EvalM}(f, \mathbf{M}) \mapsto \mathbf{K}_{f,\mathbf{M}}$
- $\mathsf{EvalC}(f, \mathbf{M}, \mathbf{x}) \mapsto \mathbf{H}_{f,\mathbf{M},\mathbf{x}}$

producing low-norm matrices that satisfy the following *key equation*:

$$\begin{bmatrix}\mathbf{G}(\mathbf{M} - \mathbf{h}^\intercal \otimes \mathbf{I}_n) & \mathbf{G}\mathbf{Z}\end{bmatrix} \cdot \mathbf{H}_{f,\mathbf{M},\mathbf{x}} = \mathbf{G}(\mathbf{M} \cdot \mathbf{K}_{f,\mathbf{M}} - \mathsf{Hash}(f(\mathbf{x}_1, \ldots \mathbf{x}_r))^\intercal \otimes \mathbf{I}_n).$$

where $\mathbf{h} = \begin{bmatrix}\mathsf{Hash}(\mathbf{x}_1) & \ldots & \mathsf{Hash}(\mathbf{x}_r)\end{bmatrix}$. In other words, we can homomorphically compute over lattice encodings. Our objective is to extend this concept to hashing *exponentially large* vectors, and computing homomorphically over these. Of course the challenge is to do so in polynomial time.

Non-Deterministic Encodings. Conceptually, our construction defines a binary hash tree of depth d where each internal node represents a deterministic hash of its children, and each leaf implicitly encodes the value at the corresponding position. More concretely, each internal node v corresponds to a triple $(\mathbf{x}_v, \mathbf{M}_v, \mathbf{c}_v)$ satisfying the invariant

$$\mathbf{c}_v^\intercal \approx \mathbf{s}^\intercal \mathbf{G}\left(\mathbf{M}_v - (\mathsf{Hash}(\mathbf{x}_v)^\intercal \otimes \mathbf{I}_n)\right) \tag{1}$$

with $\mathbf{x}_v = \mathsf{vec}(\begin{bmatrix}\mathbf{M}_{v_\mathsf{L}} & \mathbf{M}_{v_\mathsf{R}}\end{bmatrix})$, where v_L and v_R denote the left and right child, respectively. We typically use $\mathbf{x}_{v,0}$ and $\mathbf{x}_{v,1}$ to denote the left and right half of $\mathbf{x}_v$, respectively. For leaf nodes, $\mathbf{x}_v$ will simply be a vectorial representation of the entry of the 2^d-dimensional vector.

Of course, this tree is too big to be computed as a whole, as its size grows exponentially in d. However, we will show three important properties of this data structure. (i) The first one is that suitably structured 2^d-dimensional vectors admit a tree encoding for which, given any node v, we can efficiently derive the triple $(\mathbf{x}_w, \mathbf{M}_w, \mathbf{c}_w)$ for every node w on the path from the root to v. An example of such vectors are the rows of the parity-check matrix of the Hamming code. (ii) The second property is that given the triple $(\mathbf{x}_w, \mathbf{M}_w, \mathbf{c}_w)$ for every node w on the path from the root to a leaf α, we can efficiently derive a (compressed) lattice encoding of the corresponding value. (iii) The third property is that we will be able to perform computations over the entire tree in time polynomial in d. More concretely, imagine we could run in time 2^d, then we could simply transform a tree that encodes some 2^d-dimensional vector $\mathbf{y}$ into a tree that encodes the vector $f^{\otimes 2^d}(\mathbf{y})$, where the entry at every position α is $f(\mathbf{y}_\alpha)$, simply by computing f separately on each leaf. We will show how to achieve the same effect but keeping the runtime polynomial in d.

Property I: Tree Initialization. This is the procedure we use to set up a tree encodings for the rows in the parity-check matrix for the Hamming code. Essentially, to encode the i-th row, the task corresponds to generating a tree encoding where the value at the α-th leaf corresponds to the i-th bit of α. However, since each leaf α formally stores the value as a vectors $\mathbf{x}_\alpha$, we will map the bit 0 as zero vector $\mathbf{0}$ and 1 to the vectorization of a non-zero matrix $\mathbf{M}$ (for instance, it can have 1 in the top-left corner and 0 everywhere else).

Our procedure takes as input i and a string $\alpha \in \{0,1\}^d$ and produces the triples $(\mathbf{x}_w, \mathbf{M}_w, \mathbf{c}_w)$ for all the nodes from the root to the α-th leaf. We proceed bottom-up. For the bottom layer we compute

$$\mathbf{x}_d = \mathsf{vec}(\mathbf{M}) \qquad \mathbf{M}_d = \mathbf{G}^{-1}(\mathsf{Hash}(\mathbf{x}_d)^\intercal \otimes \mathbf{G}) \qquad \mathbf{c}_d = \mathbf{0}$$

whereas for $j < d$ we define

$$\mathbf{M}_j = \mathbf{G}^{-1}(\mathsf{Hash}(\mathbf{x}_j)^\intercal \otimes \mathbf{G}) \qquad \mathbf{c}_j = \mathbf{0}.$$

where

$$\mathbf{x}_j = \begin{bmatrix} \mathsf{vec}(\mathbf{M}_{j+1}) \\ \mathsf{vec}(\mathbf{M}_{j+1}) \end{bmatrix} \quad (\text{if } j \neq i) \qquad \mathbf{x}_j = \begin{bmatrix} \mathbf{0} \\ \mathsf{vec}(\mathbf{M}_{j+1}) \end{bmatrix} \quad (\text{otherwise})$$

If $\alpha_i = 1$, we return $\{\mathbf{x}_j, \mathbf{M}_j, \mathbf{c}_j\}_{j \in [d]}$. Otherwise, we return $\{\mathbf{x}_j, \mathbf{M}_j, \mathbf{c}_j\}_{j \leq i} \cup \{\mathbf{0}, \mathbf{0}, \mathbf{0}\}_{j > i}$.

A path $\rho(\alpha)^{(i)}$ thus defines a sequence $\{\mathbf{x}_j, \mathbf{M}_j, \mathbf{c}_j\}_j$ preserving the invariant that each $\mathbf{x}_j$ predicts the matrix of the next level and where the leaf is either $\mathbf{0}$ (if $\alpha_i = 0$) or $\mathsf{vec}(\mathbf{M})$ (if $\alpha_i = 1$). Furthermore, it is easy to verify by direct calculation that each triple satisfies the invariant defined by Equation (1), in fact for all possible $\mathbf{s}$.

Property II: Path Resolution. Imagine that, for every $j \in [d]$, we hold the triple $(\mathbf{x}_j, \mathbf{M}_j, \mathbf{c}_j)$ for the j-th node on the path from the root to the α-th leaf. We show how to efficiently derive a BGG^+ encoding of the value stored at the leaf. We refer to this procedure as *path resolution*.

The algorithm takes as input a sample $\mathbf{z}^\intercal \approx \mathbf{s}^\intercal \mathbf{Z}$. Notice that $(\mathbf{c}_0, \mathbf{z})$ corresponds to a compressed encoding of $\mathbf{x}_0$. Using the expansion equation, we can derive

$$\mathbf{c}_0^\intercal \mathbf{P} + \mathbf{z}^\intercal \mathbf{Q}_{\mathbf{x}_0} \approx \mathbf{s}^\intercal \mathbf{G}(\mathbf{M}_0 \mathbf{P} - \mathbf{x}_0 \otimes \mathbf{I}_n).$$

Now, there exist low-norm matrices $\mathbf{V}_0, \mathbf{V}_1$ such that for every matrix $\mathbf{M}$ of suitable dimensions we have $(\mathsf{vec}(\mathbf{M})^\intercal \otimes \mathbf{I}_n) \cdot \mathbf{V}_b = \mathbf{M}_b$, where $\mathbf{M}_0$ and $\mathbf{M}_1$ represent the left and right half of $\mathbf{M}$ respectively. Recalling that $\mathbf{x}_{0,\alpha_0} = \mathsf{vec}(\mathbf{M}_1)$, we obtain that

$$\begin{aligned} \mathbf{c}_0^\intercal \mathbf{P} \cdot \mathbf{V}_{\alpha_0} + \mathbf{z}^\intercal \mathbf{Q}_{\mathbf{x}_0} \cdot \mathbf{V}_{\alpha_0} + \mathbf{c}_1^\intercal &\approx \mathbf{s}^\intercal \mathbf{G}(\mathbf{M}_0 \mathbf{P} \mathbf{V}_{\alpha_0} - \mathbf{M}_1) + \mathbf{c}_1^\intercal \\ &\approx \mathbf{s}^\intercal \mathbf{G}(\mathbf{M}_0 \mathbf{P} \mathbf{V}_{\alpha_0} - \mathsf{Hash}(\mathbf{x}_1)^\intercal \otimes \mathbf{I}_n). \end{aligned}$$

Notice that the matrix $\mathbf{M}_0 \mathbf{P} \mathbf{V}_{\alpha_0}$ only depends on the public paramaters and the index of the leaf α. Together with $\mathbf{z}$ this gives a compressed encoding of $\mathbf{x}_1$. Effectively, this allowed us to translate the encoding down one level, and nothing stops us from proceeding all the way until the leaf. Notice that we can efficiently compute the matrix of this encoding given only $\alpha, \mathbf{M}_0$, and the public parameters. The computation requires depth $\log d \cdot \mathsf{poly}(\log \log q)$.

Property III: Homomorphic Computation. The last ingredient that we need in order to effectively use our encodings are algorithms to compute over paths. Specifically, given two paths $\rho(\alpha)^{(0)}$ and $\rho(\alpha)^{(1)}$ with leaf values $\mathbf{x}^{(0)}$ and $\mathbf{x}^{(1)}$, respectively, we show how to compute the following operations:

- (Addition) Returns a path $\tilde{\rho}(\alpha)$ with leaf $\mathbf{M}^{(0)} + \mathbf{M}^{(1)}$.
- (Bilinear function) Returns a path $\tilde{\rho}(\alpha)$ with leaf $\mathbf{F} \cdot (\mathbf{x}^{(0)} \otimes \mathbf{x}^{(1)})$ where $\mathbf{F}$ is a matrix.

It is easy to see that, if we add two trees together, we obtain a tree for the sum. Specifically, for every node v of the sum tree, the triple will be

$$(\mathbf{x}_v^{(0)} + \mathbf{x}_v^{(1)}, \mathbf{M}_v^{(0)} + \mathbf{M}_v^{(1)}, \mathbf{c}_v^{(0)} + \mathbf{c}_v^{(1)})$$

where $(\mathbf{x}_v^{(0)}, \mathbf{M}_v^{(0)}, \mathbf{c}_v^{(0)})$ and $(\mathbf{x}_v^{(1)}, \mathbf{M}_v^{(1)}, \mathbf{c}_v^{(1)})$ are the triples associated with v in the original trees we want to add. It is easy to see that the invariant Eq. (1) is satisfied thanks to the linearity of Hash. Moreover, due to the locality of this operation, if we restrict our attention to a single path, the homomorphic operation becomes efficient.

Next, we sketch how to apply bilinear functions. Suppose that we are given a bilinear function f and two trees, where each node v is associated with the triples $(\mathbf{x}_v^{(0)}, \mathbf{M}_v^{(0)}, \mathbf{c}_v^{(0)})$ and $(\mathbf{x}_v^{(1)}, \mathbf{M}_v^{(1)}, \mathbf{c}_v^{(1)})$, respectively. Our goal will be to derive a tree in which every leaf α encodes the value $f(\mathbf{x}_\alpha^{(0)}, \mathbf{x}_\alpha^{(1)})$. Moreover, we want the computation to be *local*: For every v, we should be able to efficiently derive the corresponding triple in the new tree from the corresponding triples in the input trees. This ensures that, if we restrict our attention to a single path, the homomorphic operation becomes efficient.

To understand the idea, consider the situation at any leaf α. Here, the problem is easy: By the homomorphic properties of the compressed encodings, from $(\mathbf{x}_\alpha^{(0)}, \mathbf{M}_\alpha^{(0)}, \mathbf{c}_\alpha^{(0)})$ and $(\mathbf{x}_\alpha^{(1)}, \mathbf{M}_\alpha^{(1)}, \mathbf{c}_\alpha^{(1)})$, we can efficiently derive a triple $(\mathbf{x}_\alpha^{(2)}, \mathbf{M}_\alpha^{(2)}, \mathbf{c}_\alpha^{(2)})$, where $\mathbf{x}_\alpha^{(2)} = f(\mathbf{x}_\alpha^{(0)}, \mathbf{x}_\alpha^{(1)})$ and $\mathbf{c}_\alpha^{(2)}$ is a compressed encoding of $\mathbf{x}_\alpha^{(2)}$ under the matrix $\mathbf{M}_\alpha^{(2)}$. At this point, one may be tempted to run this procedure also for the internal nodes of the tree. This, however, would ruin the invariant that, for every node v, $\mathbf{x}_v = \mathsf{vec}(\begin{bmatrix}\mathbf{M}_{v_\mathsf{L}} & \mathbf{M}_{v_\mathsf{R}}\end{bmatrix})$. We need to do something slightly different: if we look at how $\mathbf{M}_\alpha^{(2)}$ is calculated, we notice that this is essentially a bilinear function of $\mathsf{vec}(\mathbf{M}_\alpha^{(0)})$ and $\mathsf{vec}(\mathbf{M}_\alpha^{(0)})$.[1] This is not the same bilinear function as before, but it is nevertheless a bilinear function f' that, crucially, depends only on the public parameters and f. In other words, if v is a node in the second-last layer (just above the leaves), we can derive the corresponding triple $(\mathbf{x}_v^{(2)}, \mathbf{M}_v^{(2)}, \mathbf{c}_v^{(2)})$ by homomorphically applying

$$f_1(\mathbf{x}_v^{(0)}, \mathbf{x}_v^{(1)}) := (f'(\mathbf{x}_{v,0}^{(0)}, \mathbf{x}_{v,0}^{(1)}), f'(\mathbf{x}_{v,1}^{(0)}, \mathbf{x}_{v,1}^{(1)}))$$

[1] Essentially, the homomorphic computation expands the input encodings (just a linear computation on the matrices), then, homomorphically evaluates f on the resulting BGG^+ encodings (this is a bilinear operation on the matrices because, since all terms are low-norm, we do not need to perform bit-decompositions as in [BGG+14]) and finally, homomorphically hashes the result to obtain a compressed encoding (again, a linear operation on the matrices because Hash is linear).

on $(\mathbf{x}_v^{(0)}, \mathbf{M}_v^{(0)}, \mathbf{c}_v^{(0)})$ and $(\mathbf{x}_v^{(1)}, \mathbf{M}_v^{(1)}, \mathbf{c}_v^{(1)})$. Since f_1 is again bilinear, $\mathbf{M}_v^{(2)}$ will again be a bilinear function f_1' of $\mathbf{M}_v^{(0)}$ and $\mathbf{M}_v^{(1)}$, which depends only on the public parameters and f. That translates to applying some bilinear function f_2 one more layer up the tree. Continuing in this way, we reach the root. It is not too hard to see that the invariant Eq. (1) is satisfied.

Once we know how to initialise the tree and homomorphically compute additions and bilinear functions, we know how to homomorphically apply any polynomially computable function f. One minor issue we encounter is that, due to the fact that we never compute bit-decompositions throughout the homomorphic computations, the norm of of the matrices used in the triples continues to grow. This significantly affects the noise growth in the encodings to the point of making it increase as a double exponential in the depth of the computation. To prevent this blow up, we devise a procedure, called *flattening*, to sanitise a tree and reset the norm of the matrices without affecting the value at the leaves. Once again, the operation is local, so if we restrict it to a path, we can perform it efficiently. With this trick, the noise growth in the encodings will be a single exponential in the depth (same as in [BGG+14, AMR25a]). We refer to the technical sections for further details about this.

A New Key Equation. We conclude this discussion by rephrasing the construction in purely linear-algebra terms. Adopting the language of [BGG+14], we can summarize our deterministic encodings as a new *key equation*: there exist polynomial-time algorithms with the following syntax:

- $\mathsf{Eval}(f) \mapsto \mathbf{M}_f$
- $\mathsf{EvalH}(f, \mathbf{x}) \mapsto \mathbf{H}_{f,\mathbf{x}}$
- $\mathsf{Expand}(\mathbf{x}) \mapsto \mathbf{P}_{\mathbf{x}}$

such that for all functions f and all inputs $\mathbf{x} \in \{0,1\}^\ell$ it holds that

$$\mathbf{G}(\mathbf{M}_f\mathbf{P}_{\mathbf{x}} - f(\mathbf{x})^\intercal \otimes \mathbf{I}_n) = \mathbf{GZH}_{f,\mathbf{x}}.$$

Moreover $\|\mathbf{H}_{f,\mathbf{x}}\|_\infty \leq \mathsf{poly}(\lambda)^{\ell\delta}$ and $\|\mathbf{P}_{\mathbf{x}}\|_\infty \leq \mathsf{poly}(\lambda)^\ell$, where δ is the circuit depth of f. One can indeed verify that LHS of the above equation corresponds to the output of the path resolution algorithm, on input a root evaluated at f.

1.3 Technical Outline: Applications

We give an overview of the new applications enabled by our non-deterministic encodings.

Laconic Shift-Hiding PRF. In what follows, we assume familiarity with the notion of laconic function evaluation (LFE) [QWW18]. Our shift-hiding PRF [PS18] consists of the following algorithms:

- (Setup) To set up a key we first initialize encodings $\{\rho(0^d)^{(i)}\}_i$ for $\alpha = 0^d$ as discussed above.[2] Then we define the function g as

$$g(\alpha) = \mathsf{vec}(\mathbf{G}^{-1}(\mathsf{LFEHash}(U_\alpha)))$$

  where U_α is the universal circuit with α hardwired and $\mathbf{G}^{-1}$ is the bit-decomposition [MP12]. We run the homomorphic evaluation for g on the paths initialized above and we let $\tilde{\rho}(0^d)$ be the resulting path with $\tilde{\mathbf{M}}_0$ being the matrix at the root of the tree. The key consists of $\tilde{\mathbf{M}}_0$ along with a randomly sampled $\mathbf{s}$ and the public parameters.

- (Shift) To compute a key shifted by a function f, we simply compute $\mathsf{ec} \xleftarrow{\$} \mathsf{LFEEncode}(\mathbf{s}, f)$ and output ec along with $\mathbf{z}^\intercal \approx \mathbf{s}^\intercal \mathbf{G}\mathbf{Z}$.

- (Eval) On input α, the regular evaluation algorithm computes

$$\left\lceil \mathbf{s}^\intercal \mathbf{G} \tilde{\mathbf{M}}_\alpha \mathbf{V} \right\rfloor_p$$

  where $\tilde{\mathbf{M}}_\alpha$ is the publicly computable matrix resulting from resolving a given path with root $\tilde{\mathbf{M}}_0$ on α (as discussed above), and $\mathbf{V} := \begin{bmatrix}\mathbf{V}_0 & \mathbf{V}_1\end{bmatrix}$. We use $\lceil \cdot \rfloor_p$ to denote rounding onto $\mathbb{Z}_p$ for a smaller modulus p.

- (Shifted Eval) On input α, the shifted evaluation algorithm initializes paths $\{\rho(\alpha)^{(i)}\}_i$ and homomorphically evaluates g to obtain $\tilde{\rho}(\alpha)$. The output is set to be

$$\lceil \mathsf{LFEEval}(\mathsf{ec}, U_\alpha) + \mathsf{PathResolve}(\mathbf{z}, \alpha, \rho(\alpha)) \cdot \mathbf{V} \rfloor_p .$$

It is easy to show that the shifted key computationally hides the function f, with a direct reduction to the decomposed LWE assumption. The more interesting bit is to show correctness of the shifted eval algorithm. Indeed, observe that

$$\begin{aligned}
&\mathsf{LFEEval}(\mathsf{ec}, U_\alpha) + \mathsf{PathResolve}(\mathbf{z}, \alpha, \rho(\alpha)) \cdot \mathbf{V} \\
&\approx \mathbf{s}^\intercal \cdot \mathsf{LFEHash}(U_\alpha) + \frac{q}{p} U_\alpha(f) + \mathsf{PathResolve}(\mathbf{z}, \alpha, \tilde{\rho}(\alpha)) \cdot \mathbf{V} \\
&\approx \mathbf{s}^\intercal \cdot \mathsf{LFEHash}(U_\alpha) + \frac{q}{p} f(\alpha) + \mathbf{s}^\intercal \mathbf{G}(\tilde{\mathbf{M}}_\alpha - g(\alpha) \otimes \mathbf{I}_n) \cdot \mathbf{V} \\
&= \mathbf{s}^\intercal \mathbf{G} \tilde{\mathbf{M}}_\alpha \mathbf{V} + \frac{q}{p} f(\alpha)
\end{aligned}$$

where the first identity follows by the correctness of the LFE, the second by the correctness of the path resolve and the fact that we homomorphically evaluate g, and the third by the fact that $\mathbf{G} \cdot (g(\alpha) \otimes \mathbf{I}_n) \cdot \mathbf{V} = \mathsf{LFEHash}(U_\alpha)$. Ignoring noise terms (which are anyway killed off by the rounding) the shifted evaluation algorithm returns the correct result.

The crucial property of such construction is that it is *laconic*: The (non-shifted) evaluation algorithm does not depend on f nor on its complexity (except for an implicit depth dependence which we can ignore in this overview). As we shall see shortly, this is the property that will enable important applications.

[2] The choice of $\alpha = 0^d$ is arbitrary here, any other fixed bit-string would work.

A Heuristic Approach to Obfuscation. In a laconic shift-hiding function, the evaluation of an unshifted key has lower complexity than the evaluation of a shifted key. Thus, it is possible, in principle, to evaluate the unshifted PRF inside the shift. This leads to the following heuristic approach to obfuscation: To obfuscate a circuit C, we first run the setup algorithm to compute a PRF key K, then we define the function g_K to be

$$g_K(x) = C(x) - \mathsf{Eval}(K, x)$$

where Eval is the unshifted evaluation algorithm. The obfuscation will be $\tilde{K} \xleftarrow{\$} \mathsf{Shift}(K, g_K)$. One can see that evaluating the obfuscated circuit returns the correct result, since

$$\begin{aligned}\mathsf{ShiftedEval}(\tilde{K}, x) &= \mathsf{Eval}(K, x) + g_K(x) \\ &= \mathsf{Eval}(K, x) + C(x) - \mathsf{Eval}(K, x) \\ &= C(x).\end{aligned}$$

Of course, the circularity we introduce does not allow us to prove this construction secure, but we could not disprove it either. If this approach manages to withstand future cryptanalysis, this approach could provide a more direct way to constructing obfuscation, avoiding the large computational overheads introduced by recursion in the current iO constructions [BV15a, AJ15]. In the meantime, we advise caution.

Pseudorandom Obfuscation. Although the circularity prevents us from proving the security of our general-purpose iO construction, we shall see how we can circumvent this barrier, for specific applications. The first is a compiler that turns a regular iO obfuscator Obf into a pseudorandom one. We recall that a pseudorandom obfuscator guarantees that the obfuscation of a random PRF is indistinguishable from a random string [BDJ+25]. We show that our construction satisfies this property if the PRF is puncturable and subexponentially secure. To obfuscate a circuit C, we first derive $\tilde{g} = \mathsf{Obf}(1^\lambda, g_K)$, and then output $\mathsf{Shift}(K, \tilde{g})$. The correctness of the construction follows from the correctness of Obf and of the shift-hiding function, as shown above. For security, notice that if $C(x)$ computes the evaluation of a random puncturable PRF, one can apply a puncturing argument to gradually remove all information about K from $\tilde{g}$ [SW14], at which point pseudorandomness follows directly from the shift-hiding property.

Obfuscating Lattice Preimage Sampling. We show that the heuristic approach of evaluating the unshifted key inside the shift leads to a *provably secure obfuscation* for the lattice preimage sampling procedure [MP12], in a sense that we will clarify shortly. Specifically, we can construct an obfuscation for the functionality C that on input a matrix $\mathbf{M}$ and auxiliary information aux (e.g., an extractable NIZK) proceeds as follows:

- Use some hardwired secret τ to extract a lattice trapdoor $\mathbf{T}$ for $\mathbf{M}$ from aux.
- Uses $\mathbf{T}$ to generate a random low-norm vector $\tilde{\mathbf{w}}$ such that $\mathbf{M} \cdot \tilde{\mathbf{w}} = \mathbf{Q} \cdot \mathbf{y}_{\mathbf{M},\mathsf{aux}}$, where $\mathbf{y}_{\mathbf{M},\mathsf{aux}}$ is a low-norm vector and $\mathbf{Q}$ is a random matrix hardcoded in the circuit.

- Outputs $\tilde{\mathbf{w}}$.

As we will see later, this functionality is very useful: It ensures that if we provide an LWE sample $\mathbf{s}^\intercal \cdot \mathbf{M} \cdot \tilde{\mathbf{w}} + e$, where $\tilde{\mathbf{w}}$ is the output of the obfuscated program, the value of $\mathbf{s}$ remains secret. Indeed, we can apply LWE with respect to the random matrix $\mathbf{Q}$ as

$$\mathbf{s}^\intercal \mathbf{M} \cdot \tilde{\mathbf{w}} \approx \mathbf{s}^\intercal \mathbf{Q} \cdot \mathbf{y}_{\mathbf{M},\mathsf{aux}}.$$

To understand how we achieve this, we need to recall how the preimage sampling procedure of [MP12] works. A trapdoor $\mathbf{T}$ for a matrix $\mathbf{M}$ corresponds to a low-norm matrix such that $\mathbf{M} \cdot \mathbf{T} = \mathbf{G}$. The goal of preimage sampling is to derive, given as input a vector $\mathbf{u}$, a *low-norm* vector $\mathbf{w}$ such that $\mathbf{M} \cdot \mathbf{w} = \mathbf{u}$. The (simplified) idea of [MP12] is the following: First sample a random low norm vector $\mathbf{w}'$, then output $\mathbf{w} := \mathbf{w}' + \mathbf{T} \cdot \mathbf{G}^{-1}(\mathbf{u} - \mathbf{M} \cdot \mathbf{w}')$. It is easy to see that

$$\mathbf{M} \cdot \mathbf{w} = \mathbf{M} \cdot \mathbf{w}' + \mathbf{M} \cdot \mathbf{T} \cdot \mathbf{G}^{-1}(\mathbf{u} - \mathbf{M} \cdot \mathbf{w}') = \mathbf{u}.$$

Moreover, if $\mathbf{w}'$ is sampled from a suitable low-norm distribution, the output $\mathbf{w}$ looks like a random low-norm sample, conditioned on $\mathbf{M} \cdot \mathbf{w} = \mathbf{u}$. Contrary to [MP12], where $\mathbf{w}'$ is sampled from a discrete Gaussian, here we sample it from the uniform distribution over some bounded interval $[-p/2, p/2]$.

To obfuscate C we use two shift-hiding functions: The first SH_q outputs vectors over $\mathbb{Z}_q$ where $q \gg p$ is the modulus over which $\mathbf{M}$ is defined, the second one, SH_p, output vectors over $\mathbb{Z}_p$. Let K_0 be the unshifted key for SH_q and let K_1 be the unshifted key for SH_p. The obfuscation algorithm defines the shifted keys as:

- $\tilde{K}_0 \xleftarrow{\$} \mathsf{SH}_q.\mathsf{Shift}(K_0, g_0)$, where $g_0(\mathbf{M}, \mathsf{aux}) = -\mathbf{M} \cdot \mathbf{w} + g'(\mathbf{M}, \mathsf{aux})$, g' is a function we are going to define soon, and $\mathbf{w} := \mathsf{SH}_p.\mathsf{Eval}(K_1, (\mathbf{M}, \mathsf{aux}))$.

- $\tilde{K}_1 \xleftarrow{\$} \mathsf{SH}_p.\mathsf{Shift}(K_1, g_1)$, where $g_1(\mathbf{M}, \mathsf{aux}) = \mathbf{T} \cdot \mathbf{G}^{-1}(\tilde{\mathbf{u}})$ and $\tilde{\mathbf{u}} = \mathsf{SH}_q.\mathsf{ShiftEval}(\tilde{K}_0, (\mathbf{M}, \mathsf{aux}))$.

The obfuscation consists of the shifted key $\tilde{K}_1$.[3]

To evaluate the obfuscation on input $(\mathbf{M}, \mathsf{aux})$, we just evaluate the shifted key to obtain $\tilde{\mathbf{w}} \in \mathbb{Z}_p$. To see why this constitutes a valid output of C, notice that [4]

$$\begin{aligned}
\mathbf{M} \cdot \tilde{\mathbf{w}} &= \mathbf{M} \cdot (\mathbf{w} + \mathbf{T} \cdot \mathbf{G}^{-1}(\tilde{\mathbf{u}})) \\
&= \mathbf{M} \cdot \mathbf{w} + \tilde{\mathbf{u}} \\
&= \mathsf{SH}_q.\mathsf{Eval}(K_0, (\mathbf{M}, \mathsf{aux})) + g'(\mathbf{M}, \mathsf{aux}).
\end{aligned}$$

[3] Observe that the circularity in the definition of the shifts is only well-defined because of the laconic property of the shift-hiding PRFs.

[4] In the above computations, we are actually mixing modulo q and modulo p computations. There is the risk that wrap arounds compromise correctness, however, if we pick p to be significantly larger than the norm of the trapdoor $\mathbf{T}$, the probability of this event can be made negligible.

Now, let $f_{\mathbf{M},\mathsf{aux}}$ be the function $K_0 \mapsto \mathsf{SH}_q.\mathsf{Eval}(K_0, (\mathbf{M}, \mathsf{aux}))$ and define

$$\begin{aligned}
\mathbf{u} &:= \mathsf{SH}_q.\mathsf{Eval}(K_0, (\mathbf{M}, \mathsf{aux})) \\
\mathbf{Q}' &:= \mathbf{Q} - \mathsf{Bits}(K_0)^\intercal \otimes \mathbf{G} \\
\mathbf{K}_{\mathbf{M},\mathsf{aux}} &:= \mathsf{BGG}^+.\mathsf{EvalM}(f_{\mathbf{M},\mathsf{aux}}, \mathbf{Q}') \\
\mathbf{H}_{\mathbf{M},\mathsf{aux}} &:= \mathsf{BGG}^+.\mathsf{EvalC}(f_{\mathbf{M},\mathsf{aux}}, \mathbf{Q}', K_0).
\end{aligned}$$

Check what happens if we define

$$g'(\mathbf{M}, \mathsf{aux}) := \mathbf{Q}' \cdot \mathbf{K}_{\mathbf{M},\mathsf{aux}} \cdot \mathbf{v}$$

where $\mathbf{v}$ is the low-norm vector such that, for every $\mathbf{y} \in \mathbb{Z}_q^k$, we have $(\mathbf{G}^{-1}(\mathbf{y})^\intercal \otimes \mathbf{G}) \cdot \mathbf{v} = \mathbf{y}$. By the key equation of [BGG+14], we have that

$$\underbrace{(\mathbf{Q}' + \mathsf{Bits}(K_0)^\intercal \otimes \mathbf{G})}_{\mathbf{Q}} \cdot \mathbf{H}_{\mathbf{M},\mathsf{aux}} = \mathbf{Q}'\mathbf{K}_{\mathbf{M},\mathsf{aux}} + \mathbf{G}^{-1}(\mathbf{u})^\intercal \otimes \mathbf{G}$$

Therefore, if we multiply both sides by $\mathbf{v}$, we obtain that

$$\mathbf{Q} \cdot \mathbf{H}_{\mathbf{M},\mathsf{aux}} \cdot \mathbf{v} = \mathbf{u} + g'(\mathbf{M}, \mathsf{aux}).$$

We have just proven that $\mathbf{M} \cdot \tilde{\mathbf{w}} = \mathbf{Q} \cdot \mathbf{h}_{\mathbf{M},\mathsf{aux}}$, where $\mathbf{h}_{\mathbf{M},\mathsf{aux}}$ is a low-norm vector.

For security, we show that $\tilde{K}_1$ is indistinguishable from uniform. We start by observing that $\mathbf{Q}'$ does not leak any information about K_0 as $\mathbf{Q}$ masks all the information. Next, we apply the shift hiding property to substitute $\tilde{K}_0$ with a random string (we can do this because g_0, g_1 and g' do not use K_0). Finally, it is the turn of $\tilde{K}_1$. The operation can be performed because now $\tilde{K}_0$ no longer depend on K_1.

Compressing LWE Samples. Next, we show a method to compress LWE samples, which is going to be instrumental for our broadcast encryption scheme. Let us make our task more concrete: We are given as input a matrix $\mathbf{X}$ and we want to compute a hash $\mathsf{Hash}(\mathbf{X})$ such that an LWE sample with respect to the hash can be publicly expanded

$$\mathsf{Expand}(\mathbf{s}^\intercal \mathsf{Hash}(\mathbf{X}) + \mathbf{e}^\intercal) \mapsto \mathbf{s}^\intercal \mathbf{X} + \tilde{\mathbf{e}}^\intercal$$

into and LWE sample with respect to $\mathbf{X}$. We show that if $\mathbf{X}$ satisfies additional properties, e.g., $\mathbf{X}$ is sampled at random, the sample $\mathbf{s}^\intercal \mathsf{Hash}(\mathbf{X}) + \mathbf{e}^\intercal$ also looks indistinguishable from random. We defer the description of the security notion to a later point in this overview.

The basic intuition is that by multiplying the decomposed LWE matrices by low-norm matrices and performing some restructuring, we are able to derive matrices $\mathbf{U}, \mathbf{B} \in \mathbb{Z}_q^{t/n \times m}$ and $\mathbf{L} \in \mathbb{Z}_q^{mn \times t}$, where $\mathbf{B}$ and $\mathbf{L}$ are low-norm, $t > mn$, and

$$\begin{bmatrix} \mathbf{U} & \mathbf{B} \otimes \mathbf{G} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_t \\ \mathbf{L} \end{bmatrix} = \mathbf{I}_{t/n} \otimes \mathbf{G}. \tag{2}$$

In other words, $\mathbf{L}$ is a lattice trapdoor for $\begin{bmatrix}\mathbf{U} & \mathbf{B}\otimes\mathbf{G}\end{bmatrix}$. Under the hardness of decomposed LWE, we can show that any sample $(\mathbf{I}_{t/n}\otimes\mathbf{s}^\intercal)\cdot\mathbf{U}+\mathbf{E}$, where $\mathbf{E}$ is a noise term, looks random. This fact was already implicit in the work of [CHW25].

Now, take $\mathbf{X}\in\mathbb{Z}_q^{k\times t/n}$ and use $\mathbf{L}$ to compute low-norm vectors $\mathbf{w}_0,\mathbf{w}_1$ such that

$$\begin{bmatrix}\mathbf{U} & \mathbf{B}\otimes\mathbf{G}\end{bmatrix}\cdot\begin{bmatrix}\mathbf{w}_0\\ \mathbf{w}_1\end{bmatrix}=\mathsf{vec}(\mathbf{X}).$$

The hash of $\mathbf{X}$ will consist of $(\mathbf{I}_m\otimes\mathbf{G})\cdot\mathbf{w}_1$. Notice that this has size mn, while $\mathbf{X}$ has size t.

The compressed LWE sample consists of two components $\mathbf{h}\approx(\mathbf{I}_m\otimes\mathbf{s}^\intercal)\cdot(\mathbf{I}_m\otimes\mathbf{G})\cdot\mathbf{w}_1$ and $\mathbf{C}\approx(\mathbf{I}_{t/n}\otimes\mathbf{s}^\intercal)\cdot\mathbf{U}$. To expand it, one can simply compute:

$$\begin{aligned}\mathbf{C}\cdot\mathbf{w}_0+\mathbf{B}\cdot\mathbf{h}&\approx(\mathbf{I}_{t/n}\otimes\mathbf{s}^\intercal)\cdot\mathbf{U}\cdot\mathbf{w}_0+\mathbf{B}\cdot(\mathbf{I}_m\otimes\mathbf{s}^\intercal)\cdot(\mathbf{I}_m\otimes\mathbf{G})\cdot\mathbf{w}_1\\&=(\mathbf{I}_{t/n}\otimes\mathbf{s}^\intercal)\cdot\mathbf{U}\cdot\mathbf{w}_0+(\mathbf{I}_{t/n}\otimes\mathbf{s}^\intercal)\cdot(\mathbf{B}\otimes\mathbf{I}_k)\cdot(\mathbf{I}_m\otimes\mathbf{G})\cdot\mathbf{w}_1\\&=(\mathbf{I}_{t/n}\otimes\mathbf{s}^\intercal)\cdot(\mathbf{U}\cdot\mathbf{w}_0+(\mathbf{B}\otimes\mathbf{G})\cdot\mathbf{w}_1)\\&=(\mathbf{I}_{t/n}\otimes\mathbf{s}^\intercal)\cdot\mathsf{vec}(\mathbf{X})\\&=\mathbf{X}^\intercal\cdot\mathbf{s}.\end{aligned}$$

There are however two caveat. Here is the first one: As we described it, the above algorithm does not compress at all! In fact, $\mathbf{C}$ has size t^2/n, which is much larger than the size of $\mathbf{X}$. We deal with this aspect using recursion, similarly to [AMR25b, AMR25a, Wee25]. We can start from a matrix $\mathbf{X}$ with a much larger number of columns. Then, we hash it using a Merkle tree where the basic hash function is the one we described above. Given $\mathbf{C}$ (whose dimension remained unvaried) and an LWE sample with respect to the hash at the root, we can efficiently derive an LWE sample with respect to $\mathbf{X}$ by performing the expansion procedure layer after layer.

The second caveat is the fact that, in the current description, the hashing procedure is randomized. This allows us to argue security: if $\mathbf{X}$ is randomly sampled, we can prove that $(\mathbf{I}_m\otimes\mathbf{G})\cdot\mathbf{w}_1$ is a random vector. At this point, we could just appeal to the standard decomposed LWE assumption to argue that $\mathbf{s}$ remains hidden. The issue with a randomized construction is that to correctly re-expand the LWE sample, one needs to receive the randomness used by the hasher. This is as big as $\mathbf{X}$, defeating the whole purpose of the construction. To obviate this problem, we modify the above algorithm to sample $\mathbf{w}_0$ and $\mathbf{w}_1$ using a variant of our newly constructed *obfuscated preimage sampler*. Specifically, as part of the public parameters we will provide a shifted PRF key, which will be evaluated by the hasher (and the expander) on $\mathbf{X}$ to derive the randomness for the preimage sampling. We will show that we can simulate $\mathbf{X}$ and the shifted key so that $(\mathbf{I}_m\otimes\mathbf{G})\cdot\mathbf{w}_1$ becomes a matrix for which we can securely provide LWE samples.

Adaptive Broadcast Encryption. Recall that the goal of broadcast encryption is to allow a sender to encrypt a message under the public keys of a subset of participants, so that only the members of this subset will be able to decrypt. The challenge is to make the ciphertext of size, ideally, independent of the subset. We show how to use the techniques

introduced above to build broadcast encryption that remains secure even when the subset of recipients is chosen after seeing the public parameters and the public keys (this is known as the *semi-static* security settings).

In our construction, the public keys consist of matrices $\mathbf{Q}_i = \mathbf{D} \cdot \mathbf{R}_i$, where $\mathbf{D}$ is a random matrix included in the public parameters of the construction and $\mathbf{R}_i$ is a random low-norm matrix, known only to the i-th party. To compute an encryption of a message $\mu \in \{0, 1\}$, we first derive low-norm vectors $\mathbf{u}_0, \ldots, \mathbf{u}_{N-1}$ from an shifted PRF key, where N is the number of recipients in the set. Next, we derive the matrix

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_0 & \cdots & \mathbf{x}_{N-1} \end{bmatrix}$$

where $\mathbf{x}_i := \begin{bmatrix} \mathbf{Q}_i & \mathbf{D} \end{bmatrix} \cdot \mathbf{u}_i$ and $\mathbf{Q}_i$ are the public keys belonging to the i-th party in the set of recipients. The ciphertext consists of two components:

- A compressed LWE sample with respect to $\mathbf{X}$, which consists of $\mathbf{C} \approx (\mathbf{I}_{t/n} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U}$ and $\mathbf{h} \approx (\mathbf{I}_m \otimes \mathbf{s}^\intercal) \cdot \mathsf{Hash}(\mathbf{X})$.
- A vector $\mathbf{c}$, where $\mathbf{c}$ is a random if $\mu = 1$, otherwise $\mathbf{c}^\intercal \approx \mathbf{s}^\intercal \cdot \mathbf{D}$.

To decrypt, the i-th party can expand $\mathbf{C}$ and $\mathbf{h}$ to recover an LWE sample approximately equal to

$$\mathbf{s}^\intercal \mathbf{x}_i = \mathbf{s}^\intercal \cdot \begin{bmatrix} \mathbf{Q}_i & \mathbf{D} \end{bmatrix} \cdot \mathbf{u}_i = \mathbf{s}^\intercal \cdot \mathbf{D} \cdot \begin{bmatrix} \mathbf{R}_i & \mathbf{I}_m \end{bmatrix} \cdot \mathbf{u}_i.$$

The decryptor can therefore compare this against $\mathbf{c}^\intercal \cdot \begin{bmatrix} \mathbf{R}_i & \mathbf{I}_m \end{bmatrix} \cdot \mathbf{u}_i$. If the two quantities are close, it can deduce that $\mu = 0$, otherwise $\mu = 1$.

For security we need to argue that, if all the recipients are honest, the ciphertext is pseudorandom. We can do so by appealing to the security of the obfuscated lattice preimage sampler. In short, the obfuscator will ensure that, if all $\mathbf{Q}_i$ are random, $\mathsf{Hash}(\mathbf{X})$ is a matrix for which we can securely give LWE samples. Then, we appeal to the decomposed LWE assumption. We refer the reader to the technical sections for more details.

2 Preliminaries

We denote the security parameter by λ. We say that a function $\mu : \mathbb{N} \to \mathbb{R}$ is negligible if $|\mu(\lambda)| = 2^{-\omega(\log \lambda)}$. We denote by $x \xleftarrow{\$} X$ the sampling from a distribution X. We denote by $X \approx_c Y$ or $X \approx_s Y$ if the two distribution ensembles X and Y are computationally or statistically indistinguishable, respectively.

We use $[n]$ to denote the set $\{0, 1, \ldots, n-1\}$ and by convention all indices start counting from 0. Given any binary string $\alpha \in \{0, 1\}^*$, we denote its length by $|\alpha|$ and we denote the i-th bit of α by α_i; if this is not well-defined (either because $i < 0$ or because $i \geq |\alpha|$), we adopt the convention that $\alpha_i = 1$.

2.1 Linear Algebra

We denote vectors using lowercase bold font, e.g., $\mathbf{v}$. Matrices will be denoted using uppercase bold font, e.g., $\mathbf{A}$. We adopt the standard linear algebra convention in which vectors correspond to columns. We use the transposition symbol $^\intercal$ to denote row vectors. We define the Kroenecker product between two matrices $\mathbf{A}$ and $\mathbf{B}$ as

$$\mathbf{A} \otimes \mathbf{B} := \begin{bmatrix} a_{0,0} \cdot \mathbf{B} & \dots & a_{0,m-1} \cdot \mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{n-1,0} \cdot \mathbf{B} & \dots & a_{n-1,m-1} \cdot \mathbf{B} \end{bmatrix}$$

where n and m respectively denote the number of rows and columns of $\mathbf{A}$ and $a_{i,j}$ denotes the entry in the i-th row and j-th column of $\mathbf{A}$. We recall that for any matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}$ where the number of columns of $\mathbf{A}$ and $\mathbf{B}$ respectively match the number of rows of $\mathbf{C}$ and $\mathbf{D}$, it holds that

$$(\mathbf{A} \otimes \mathbf{B}) \cdot (\mathbf{C} \otimes \mathbf{D}) = (\mathbf{A} \cdot \mathbf{C}) \otimes (\mathbf{B} \cdot \mathbf{D}). \tag{3}$$

We use $\mathsf{vec}(\mathbf{A})$ to denote the vectorization of the matrix $\mathbf{A}$. Specifically, if $\mathbf{A} \in \mathbb{Z}^{n \times m}$, $\mathsf{vec}(\mathbf{A})$ consists of the $n \cdot m$-dimensional vector obtained by vertically concatenating $\mathbf{a}_0, \dots, \mathbf{a}_{n-1}$ where $\mathbf{a}_i^\intercal$ denotes the i-th row of $\mathbf{A}$. We denote the $n \times n$ identity matrix by $\mathbf{I}_n$. We recall that, for any matrix $\mathbf{A}$ with k rows and ℓ columns, and any matrix $\mathbf{B}$ with ℓ rows and m columns, it holds that

$$\mathsf{vec}(\mathbf{AB}) = (\mathbf{I}_m \otimes \mathbf{A})\mathsf{vec}(\mathbf{B}) = (\mathbf{B}^\intercal \otimes \mathbf{I}_k)\mathsf{vec}(\mathbf{A}). \tag{4}$$

We denote by $\|\mathbf{v}\|_\infty$ the ℓ_∞-norm of a vector $\mathbf{v}$ and for a matrix $\mathbf{A}$ we define $\|\mathbf{A}\|_\infty = \|\mathsf{vec}(\mathbf{A})\|_\infty$ and

$$\|\mathbf{A}\|_{\infty,\infty} = \max_{\mathbf{v}} \frac{\|\mathbf{vA}\|_\infty}{\|\mathbf{v}\|_\infty}.$$

We define the gadget matrix $\mathbf{G} = \mathbf{I} \otimes \mathbf{g}^\intercal$, where $\mathbf{g}^\intercal$ is the row vector $\begin{bmatrix} 1 & 2 & \dots & 2^{\log q} \end{bmatrix}$. As customary, we define the function $\mathbf{G}^{-1}$ as the binary decomposition operator, satisfying the identity $\mathbf{G}\mathbf{G}^{-1}(\mathbf{x}) = \mathbf{x}$ for all $\mathbf{x}$.

The following is a straightforward consequence of the leftover hash lemma [ILL89].

Lemma 1 (Leftover Hash Lemma)**.** *Let $m \geq 2n \log q$, then the following distributions are statistically close*

$$\left\{ \mathbf{A}, \mathbf{Ax} : \mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}, \mathbf{x} \xleftarrow{\$} \mathbb{Z}_2^m \right\} \approx_s \left\{ \mathbf{A}, \mathbf{y} : \mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}, \mathbf{y} \xleftarrow{\$} \mathbb{Z}_q^n \right\}.$$

2.2 Discrete Gaussians and Lattices

Let $\mathcal{D}_\sigma$ denote the discrete Gaussian distribution with parameter σ. We recall the following standard fact about Gaussians.

Lemma 2 (Gaussian Flooding)**.** *If X is sampled from D_σ then for any Y such that $|Y| \leq \tau$ and $\sigma/\tau = 2^{\omega(\log \lambda)}$, then the following distributions are statistically close:*

$$X \approx_s X + Y$$

where $X \leftarrow \mathcal{D}_\sigma$.

For $\sigma = \omega(\sqrt{\log(\lambda)})$, we have the following tail bound on the discrete Gaussian, which follows as an immediate consequence of [Ban93]:

$$\Pr\left[|x| \geq \sqrt{\lambda} \cdot \sigma : \ x \xleftarrow{\$} \mathcal{D}_\sigma\right] = O(2^{-\lambda}). \tag{5}$$

Preimage Sampling. We recall the structure and properties of the Micciancio–Peikert preimage sampling algorithm [MP12]. A preimage of a vector $\mathbf{x} \in \mathbb{Z}^n$ under a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ is a low-norm vector $\mathbf{y} \in \mathbb{Z}^m$ such that $\mathbf{Ay} = \mathbf{x}$. Given a trapdoor $\mathbf{T}$, i.e., a low-norm matrix such that $\mathbf{AT} = \mathbf{G}$, one can easily find a preimage by setting $\mathbf{y} = \mathbf{TG}^{-1}(\mathbf{x})$. However, such a preimage leaks information about $\mathbf{T}$, so the goal of a preimage sampler is generate $\mathbf{y}$ according to some distribution that is independent of $\mathbf{T}$.

In this work we are going to use a simplified version of the algorithm described in [MP12], which refer to as $\mathsf{ShiftedPreimage}$. Given an initial low-norm vector $\mathbf{y}'$, the sampler adds a low-norm offset to it to get a preimage of $\mathbf{x}$. More precisely, the output of the algorithm is defined as

$$\mathsf{ShiftedPreimage}(\mathbf{A}, \mathbf{T}, \mathbf{x}, \mathbf{y}') \mapsto \mathbf{y}' - \mathbf{TG}^{-1}(\mathbf{Ay}' - \mathbf{x}).$$

Using $\mathsf{ShiftedPreimage}$, one immediately obtains a very simple preimage sampler based on noise flooding: Pick some bound $p \ll q$ and sample

$$\mathbf{y}' \xleftarrow{\$} \left[-\frac{p-1}{2}, \frac{p-1}{2}\right]^m$$

viewed as a subset of $\mathbb{Z}_q^m$, then run $\mathsf{ShiftedPreimage}$. If $p \geq 2^{\omega(\log(\lambda))}\|\mathbf{T}\|_\infty$ then the shift $\mathbf{TG}^{-1}(\mathbf{t})$ applied to $\mathbf{y}'$ affects its distribution negligibly (other than changing it to be a valid preimage), which makes it a valid preimage sampler.

We shall clarify that we are not going to use explicitly the indistinguishability property of the sampler, but it rather serves as an intuition for our construction. Later on in Section 5.2 we will only use the definition of $\mathsf{ShiftedPreimage}$ and security will only appear as part of a more complex argument.

Learning with Errors. The fundamental problem underlying lattice-based cryptography is the *learning with errors* (LWE) problem [Reg05] which, roughly speaking, postulates that it is hard to distinguish between random vectors and random noisy linear equations. It is known [Reg05] that the hardness of the LWE problem follows from the worst-case quantum hardness SIVP and classical hardness of GapSVP. Below we recall the *decomposed LWE* problem, a recently introduce variant of the LWE problem [AMR25a].

Definition 1 (Decomposed LWE)**.** *Let $k = k(\lambda)$, $m = m(\lambda)$, $\sigma = \sigma(\lambda)$, $t = t(\lambda)$, and $q = q(\lambda)$ be integer parameters. Define $n = k \cdot \log q$. The decomposed LWE assumption postulates the following distributions are computationally indistinguishable*

$$\{\mathbf{A}, \mathbf{B}, (\mathbf{I}_t \otimes \mathbf{s}^\intercal) \cdot (\mathbf{AB} + \mathbf{G}) + \mathbf{E}\} \approx_c \{\mathbf{A}, \mathbf{B}, \mathbf{U}\}$$

where $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^k$, $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{t \cdot k \times m}$, $\mathbf{B} \xleftarrow{\$} \mathcal{D}_\sigma^{m \times t \cdot n}$, $\mathbf{E} \xleftarrow{\$} \mathcal{D}_\sigma^{t \times t \cdot n}$, *and* $\mathbf{U} \xleftarrow{\$} \mathbb{Z}_q^{t \times t \cdot n}$.

We note that the version defined in [AMR25a] uses two different Gaussian parameters for sampling the noise term and for the matrices, whereas here we sample them with the same parameter to keep the notation simple. All of our results generalize in an immediate way. In [AMR25a] it is also shown that the decomposed LWE assumption remains hard (with a reduction to Definition 1) even if the distinguisher is additionally given LWE samples with the same secret with respect to an auxiliary random matrix. We omit a formal definition since its hardness follows by Definition 1.

2.3 Homomorphic Encodings

We recall the syntax of homomorphic encodings from [AMR25b, AMR25a, Wee25], which were inspired by the work of Wee [Wee24]. In fact, we define here a simplified (non recursive) version that is going to be sufficient for this work.

Parameters. For convenience, we summarize here the parameters involved in the construction:

- The modulus $q = q(\lambda)$.
- The lattice dimension $k = k(\lambda)$ and $n = k \log q$.
- The matrix dimension $m = m(\lambda)$.
- The compression parameter $t = t(\lambda)$
- The gaussian parameter $\sigma = \sigma(\lambda)$.
- The the input size of the functions $r = r(\lambda)$.

Syntax. The encodings consists of the algorithms with the following syntax.

- $\mathsf{Setup}(1^\lambda, 1^t)$: On input the security parameter 1^λ and the compression parameter 1^t, the setup algorithm samples

$$\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{(t \cdot n) \times m} \qquad \text{and} \qquad \mathbf{B} \xleftarrow{\$} \mathcal{D}_\sigma^{m \times (t \cdot n)}.$$

  The algorithm returns the public parameters $\mathsf{pk} = (\mathbf{A}, \mathbf{B})$.

For convenience, we denote by $\mathbf{Z} = \begin{bmatrix}\mathbf{Z}_0 & \dots & \mathbf{Z}_{t-1}\end{bmatrix}$ where $\mathbf{Z}_i \in \mathbb{Z}_q^{n\times(n\cdot t)}$ is the i-th block of n rows of the matrix $\mathbf{A}\cdot\mathbf{B} - \mathbf{I}_t \otimes \mathbf{G}$. Note that $\mathbf{Z}$ is publicly computable from the public parameters, so we can assume without loss of generality that it is contained in pk.

- $\mathsf{Hash}(\mathsf{pk}, \mathbf{x})$: On input the public parameters pk, and input vector $\mathbf{x} \in \mathbb{Z}_q^t$, the hash function returns a digest $\mathbf{h} \in \mathbb{Z}_q^{m\cdot n}$. The hash function is *deterministic* and *linear* (over $\mathbb{Z}_q$) in the input $\mathbf{x}$.

  Denoting by $\mathbf{L}_{\mathsf{pk}}$ the linear operator associated with the hash, we also have that $\|\mathbf{L}_{\mathsf{pk}}\|_{\infty,\infty} \le t\sigma\sqrt{\lambda}$ with overwhelming probability, over the random choice of pk.

- $\mathsf{Encode}(\mathsf{pk}, \mathbf{M}, \mathbf{x})$: On input the public parameters pk, a matrix $\mathbf{M} \in \mathbb{Z}_q^{n\times m\cdot n^2}$, and an input $\mathbf{x} \in \mathbb{Z}_q^t$, the encoding algorithm computes

$$\mathbf{s}^\intercal \cdot \mathbf{G} \cdot (\mathbf{M} - \mathbf{h}^\intercal \otimes \mathbf{I}_n) + \mathbf{e}^\intercal \qquad \text{with} \qquad \mathbf{h} = \mathsf{Hash}(\mathsf{pk}, \mathbf{x})$$

  where $\mathbf{G} \in \mathbb{Z}_q^{k\times n}$, $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^k$, and $\mathbf{e} \xleftarrow{\$} \mathcal{D}_\sigma^{m\cdot n^2}$.

Encoding Expansion. It is shown in [AMR25a] that it is possible to publicly expand the encoding of the hash of a vector into the encoding of the vector itself. We summarize this fact via the following statement.

Lemma 3 (Encoding Expansion)**.** *There exists a pair of efficient deterministic algorithms:*

- $\mathsf{ExpandM}(\mathsf{pk}) \mapsto \mathbf{P} \in \mathbb{Z}_q^{(m\cdot n^2)\times(t\cdot n)}$
- $\mathsf{ExpandC}(\mathsf{pk}, \mathbf{x}) \mapsto \mathbf{Q}_{\mathbf{x}} \in \mathbb{Z}_q^{(n\cdot t^2)\times(t\cdot n)}$

such that for all pk *in the support of* $\mathsf{Setup}(1^\lambda, 1^t)$*, all matrices* $\mathbf{M} \in \mathbb{Z}_q^{n\times(m\cdot n^2)}$ *and all vectors* $\mathbf{x} \in \mathbb{Z}_q^t$ *it holds that*

$$\begin{bmatrix}\mathbf{G}\cdot(\mathbf{M} - \mathsf{Hash}(\mathsf{pk}, \mathbf{x})^\intercal \otimes \mathbf{I}_n) & \mathbf{G}\cdot\mathbf{Z}\end{bmatrix} \cdot \begin{bmatrix}\mathbf{P} \\ \mathbf{Q}_{\mathbf{x}}\end{bmatrix} = \mathbf{G}\cdot(\mathbf{M}\cdot\mathbf{P} - \mathbf{x}^\intercal \otimes \mathbf{I}_n)$$

and furthermore $\|\mathbf{P}\|_{\infty,\infty} \le nm$ *and* $\|\mathbf{Q}_{\mathbf{x}}\|_{\infty,\infty} \le \|\mathbf{x}\|_\infty \cdot t$.

Homomorphic Operations. In the same work [AMR25a], the authors showed how to perform homomorphic operations on lattice encodings, even if given an encoding of the hash of the input vector. We recall the procedure in the following.

Lemma 4 (Homomorphic Computation)**.** *There exists a pair of efficient deterministic algorithms:*

- $\mathsf{EvalM}(\mathsf{pk}, f, \mathbf{M}) \mapsto \mathbf{K}_{f,\mathbf{M}} \in \mathbb{Z}_q^{(r\cdot m\cdot n^2)\times(m\cdot n^2)}$

- $\mathsf{EvalC}(\mathsf{pk}, f, \mathbf{M}, \mathbf{x}) \mapsto \mathbf{H}_{f,\mathbf{M},\mathbf{x}} \in \mathbb{Z}_q^{(r\cdot m\cdot n^2+t^2\cdot n)\times(m\cdot n^2)}$

such that for all pk *in the support of* $\mathsf{Setup}(1^\lambda, 1^t)$*, all matrices* $\mathbf{M} \in \mathbb{Z}_q^{n\times(r\cdot m\cdot n^2)}$*, all sets of* r *vectors* $\mathbf{x}_i \in \mathbb{Z}_q^t$*, and all functions* $f : (\mathbb{Z}_q^t)^r \to \mathbb{Z}_q^t$ *it holds that*

$$\begin{bmatrix}\mathbf{G}\cdot(\mathbf{M}-\mathbf{h}^\intercal\otimes\mathbf{I}_n) & \mathbf{G}\cdot\mathbf{Z}\end{bmatrix}\cdot\mathbf{H}_{f,\mathbf{M},\mathbf{x}} = \mathbf{G}\cdot(\mathbf{M}\cdot\mathbf{K}_{f,\mathbf{M}} - \mathsf{Hash}(\mathsf{pk}, f(\mathbf{x}_1,\ldots\mathbf{x}_r))^\intercal\otimes\mathbf{I}_n).$$

where $\mathbf{h} = \begin{bmatrix}\mathsf{Hash}(\mathsf{pk},\mathbf{x}_1) & \ldots & \mathsf{Hash}(\mathsf{pk},\mathbf{x}_r)\end{bmatrix}$.

Finally, we show that for the case of linear operations, the evaluation matrix has a particularly simple form, which is going to be convenient for our analysis later.

Lemma 5 (Linear Operators)**.** *Let* $f : \mathbb{Z}_q^t \to \mathbb{Z}_q^t$ *be a linear map* $f(\mathbf{x}) := \mathbf{F}\cdot\mathbf{x}$*. Then there exists a deterministic efficient algorithm that, on input* pk *and the matrix* $\mathbf{F}$*, computes a matrix* $\tilde{\mathbf{F}} \in \mathbb{Z}_q^{(m\cdot n^3)\times(m\cdot n^3)}$ *such that*

$$\mathsf{vec}(\mathbf{M}\cdot\mathbf{K}_{f,\mathbf{M}}) = \tilde{\mathbf{F}}\cdot\mathsf{vec}(\mathbf{M}),$$

for every $\mathbf{M} \in \mathbb{Z}_q^{n\times(m\cdot n^2)}$ *and where* $\mathbf{K}_{f,\mathbf{M}} = \mathsf{EvalM}(\mathsf{pk}, f, \mathbf{M})$*. Moreover, we have that*

- $\|\tilde{\mathbf{F}}^\intercal\|_{\infty,\infty} \le \|\mathbf{P}\|_{\infty,\infty}\cdot\|\mathbf{F}^\intercal\|_{\infty,\infty}\cdot\|\mathbf{L}_{\mathsf{pk}}\|_{\infty,\infty}$*,*
- $\|\mathbf{K}_{f,\mathbf{M}}\|_{\infty,\infty} \le \|\mathbf{H}_0\|_{\infty,\infty} \le \|\mathbf{P}\|_{\infty,\infty}\cdot\|\mathbf{F}^\intercal\|_{\infty,\infty}\cdot\|\mathbf{L}_{\mathsf{pk}}\|_{\infty,\infty}$*, and*
- $\|\mathbf{H}_1\|_{\infty,\infty} \le \|\mathbf{Q}_{\mathbf{x}}\|_{\infty,\infty}\cdot\|\mathbf{F}^\intercal\|_{\infty,\infty}\cdot\|\mathbf{L}_{\mathsf{pk}}\|_{\infty,\infty}$

where

$$\begin{bmatrix}\mathbf{H}_0\\ \mathbf{H}_1\end{bmatrix} = \mathbf{H}_{f,\mathbf{M},\mathbf{x}} = \mathsf{EvalC}(\mathsf{pk}, f, \mathbf{M}, \mathbf{x}).$$

A similar property also holds for bilinear operations.

Lemma 6 (Bilinear Operators)**.** *Let* $f : \mathbb{Z}_q^t\times\mathbb{Z}_q^t \to \mathbb{Z}_q^t$ *be a bilinear map* $f(\mathbf{x},\mathbf{y}) := \mathbf{F}\cdot(\mathbf{x}\otimes\mathbf{y})$*. Then there exists a deterministic efficient algorithm that, on input* pk *and the matrix* $\mathbf{F}$*, computes a matrix* $\tilde{\mathbf{F}} \in \mathbb{Z}_q^{(m\cdot n^3)\times(m^2\cdot n^6)}$ *such that*

$$\mathsf{vec}(\mathbf{M}\cdot\mathbf{K}_{f,\mathbf{M}}) = \tilde{\mathbf{F}}\cdot(\mathsf{vec}(\mathbf{M}_0)\otimes\mathsf{vec}(\mathbf{M}_1)),$$

for every $\mathbf{M} := \begin{bmatrix}\mathbf{M}_0 & \mathbf{M}_1\end{bmatrix} \in \mathbb{Z}_q^{n\times 2(m\cdot n^2)}$ *and where* $\mathbf{K}_{f,\mathbf{M}} \leftarrow \mathsf{EvalM}(\mathsf{pk}, f, \mathbf{M})$*. Moreover, we have that*

- $\|\tilde{\mathbf{F}}^\intercal\|_{\infty,\infty} \le \|\mathbf{P}\|_{\infty,\infty}^2\cdot n\cdot\|\mathbf{F}^\intercal\|_{\infty,\infty}\cdot\|\mathbf{L}_{\mathsf{pk}}\|_{\infty,\infty}$*,*
- $\|\mathbf{K}_{f,\mathbf{M}}\|_{\infty,\infty} \le \|\mathbf{H}_0\|_{\infty,\infty} \le \|\mathbf{P}\|_{\infty,\infty}^2\cdot n\cdot\|\mathbf{M}_1\|_\infty\cdot\|\mathbf{F}^\intercal\|_{\infty,\infty}\cdot\|\mathbf{L}_{\mathsf{pk}}\|_{\infty,\infty}$*,*
- $\|\mathbf{H}_1\|_{\infty,\infty} \le \|\mathbf{P}\|_{\infty,\infty}\cdot\|\mathbf{x}_0\|_\infty\cdot\|\mathbf{F}^\intercal\|_{\infty,\infty}\cdot\|\mathbf{L}_{\mathsf{pk}}\|_{\infty,\infty}$*, and*
- $\|\mathbf{H}_2\|_{\infty,\infty} \le (\|\mathbf{Q}_{\mathbf{x}}\|_{\infty,\infty}\cdot\|\mathbf{P}\|_{\infty,\infty}\cdot\|\mathbf{M}_1\|_\infty + \|\mathbf{Q}_{\mathbf{x}}\|_{\infty,\infty}\cdot\|\mathbf{x}_0\|_\infty)\|\mathbf{F}^\intercal\|_{\infty,\infty}\cdot\|\mathbf{L}_{\mathsf{pk}}\|_{\infty,\infty}$

where

$$\begin{bmatrix}\mathbf{H}_0\\ \mathbf{H}_1\\ \mathbf{H}_2\end{bmatrix} = \mathbf{H}_{f,\mathbf{M},\mathbf{x}} = \mathsf{EvalC}(\mathsf{pk}, f, \mathbf{M}, \mathbf{x}).$$

3 Non-Deterministic Encodings

We present a construction of recursive encodings, where we implicitly define an exponentially large hash tree of encodings. Then we show how to perform homomorphic computation over the hashes.

3.1 Definition of the Encodings

Before we present a formal definition of the encodings, we establish some terminology that will be useful to describe our scheme. For a tree of depth d, any binary string $\alpha \in \{0,1\}^\ell$, for $\ell \leq d$ defines a *path* $\rho(\alpha)$ of the binary tree in the canonical enumeration, and each path consists of $\ell + 1$ *nodes* of the form

$$\rho(\alpha) = \{\mathbf{c}_i, \mathbf{M}_i, \mathbf{x}_i\}_{i \in [\ell+1]} .$$

We refer to $\mathbf{x}_\ell$ as the *leaf* of the path. Throughout the scheme, we will maintain the invariant that

$$\mathbf{c}_i^\intercal \approx \mathbf{s}^\intercal \cdot \mathbf{G} \cdot (\mathbf{M}_i - \mathbf{h}_i^\intercal \otimes \mathbf{I}_n) \quad \text{where} \quad \mathbf{h}_i = \mathsf{Hash}(\mathsf{pk}, \mathbf{x}_i) \quad \text{and} \quad \mathbf{x}_{i,\alpha_i} = \mathsf{vec}(\mathbf{M}_{i+1}).$$

for all $i \in [\ell + 1]$.

Tree Initialization. Our construction will use the homomorphic encodings presented in Section 2.3, setting $t = 2m \cdot n^3$. The (implicit) tree initialization procedure consists of two algorithms ($\mathsf{TreeSetup}, \mathsf{TreeInit}$), where the former is sampled once and for all at the beginning of the execution. The $\mathsf{TreeInit}$ algorithm allows one to (implicitly) initialize many copies of an entire tree, starting by assigning a matrix value to a desired leaf, then recursively deriving the nodes upwards in the tree. We are now ready to formally describe the algorithms.

- $\mathsf{TreeSetup}(1^\lambda)$: Compute $\mathsf{pk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^t)$.
- $\mathsf{TreeInit}(\mathsf{pk}, d, \mathbf{M}, i, \alpha)$: On input the public key pk, an integer d, a matrix $\mathbf{M}$, an index $i \in [d+1]$, and a binary string $\alpha \in \{0,1\}^\ell$, for some $\ell \leq d$, the algorithm proceeds as follows. Set:

  $$\mathbf{x}_d = \mathsf{vec}(\mathbf{M}) \qquad \mathbf{M}_d = \mathbf{G}^{-1}(\mathsf{Hash}(\mathsf{pk}, \mathbf{x}_d)^\intercal \otimes \mathbf{G}) \qquad \mathbf{c}_d = \mathbf{0}^{m \cdot n^2}.$$

  For $j = d-1, \ldots, 0$ define:

  $$\mathbf{M}_j = \mathbf{G}^{-1}(\mathsf{Hash}(\mathsf{pk}, \mathbf{x}_j)^\intercal \otimes \mathbf{G}) \qquad \mathbf{c}_j = \mathbf{0}^{m \cdot n^2}.$$

  where

  $$\mathbf{x}_j = \begin{bmatrix} \mathsf{vec}(\mathbf{M}_{j+1}) \\ \mathsf{vec}(\mathbf{M}_{j+1}) \end{bmatrix} \quad (\text{if } j \neq i-1) \qquad \mathbf{x}_j = \begin{bmatrix} \mathbf{0}^{m \cdot n^3} \\ \mathsf{vec}(\mathbf{M}_{j+1}) \end{bmatrix} \quad (\text{otherwise})$$

  If $\alpha_{i-1} = 1$, return $\{\mathbf{x}_j, \mathbf{M}_j, \mathbf{c}_j\}_{j \in [\ell+1]}$. Otherwise, return $\{\mathbf{x}_j, \mathbf{M}_j, \mathbf{c}_j\}_{j \in [i]} \cup \{\mathbf{0}, \mathbf{0}, \mathbf{0}\}_{j \geq i}$.

First, observe that if $\tilde{\alpha}$ is a prefix of α, then it is the case that the path produced by $\mathsf{TreeInit}(\mathsf{pk}, d, \mathbf{M}, i, \tilde{\alpha})$ is also a prefix of the path produced by $\mathsf{TreeInit}(\mathsf{pk}, d, \mathbf{M}, i, \alpha)$. The following lemma proves that the path produced by the above initialization procedure is consistent, i.e., it maintains the a valid chain of hashes.

Lemma 7 (Path Consistency)**.** *Let d be an integer, $\mathbf{M}_{\ell+1}$ a matrix, $i \in [d+1]$ and index, and $\alpha \in \{0,1\}^\ell$ a binary string, for some $\ell \leq d$. Let $\{\mathbf{x}_j, \mathbf{M}_j, \mathbf{c}_j\}_{j\in[\ell+1]} = \mathsf{TreeInit}(\mathsf{pk}, d, \mathbf{M}_{\ell+1}, i, \alpha)$, then for all $j \in [\ell+1]$ it holds that*

$$\mathbf{c}_j^\intercal = \mathbf{s}^\intercal \mathbf{G}(\mathbf{M}_j - (\mathsf{Hash}(\mathsf{pk}, \mathbf{x}_j)^\intercal \otimes \mathbf{I}_n)) \qquad \textit{and} \qquad \|\mathbf{M}_j\|_\infty \leq 1$$

for all $\mathbf{s} \in \mathbb{Z}_q^k$. Moreover, for all $j \in [\ell]$, it holds that $\mathbf{x}_{j,\alpha_j} = \mathsf{vec}(\mathbf{M}_{j+1})$.

Proof. If the triple $\{\mathbf{x}_j, \mathbf{M}_j, \mathbf{c}_j\} = \{\mathbf{0}, \mathbf{0}, \mathbf{0}\}$, then the equalities follow immediately by the fact that $\mathsf{Hash}(\mathsf{pk}, \mathbf{0}) = \mathbf{0}$, since it is a linear function. Otherwise, by definition of $\mathbf{M}_j$ we have that

$$\begin{aligned}
\mathbf{s}^\intercal \mathbf{G}(\mathbf{M}_j - (\mathsf{Hash}(\mathsf{pk}, \mathbf{x}_j)^\intercal \otimes \mathbf{I}_n)) &= \mathbf{s}^\intercal \mathbf{G}(\mathbf{G}^{-1}(\mathsf{Hash}(\mathsf{pk}, \mathbf{x}_j)^\intercal \otimes \mathbf{G}) - (\mathsf{Hash}(\mathsf{pk}, \mathbf{x}_j)^\intercal \otimes \mathbf{I}_n)) \\
&= \mathbf{s}^\intercal((\mathsf{Hash}(\mathsf{pk}, \mathbf{x}_j)^\intercal \otimes \mathbf{G}) - (\mathsf{Hash}(\mathsf{pk}, \mathbf{x}_j)^\intercal \otimes \mathbf{G})) \\
&= \mathbf{0} \\
&= \mathbf{c}_j.
\end{aligned}$$

The last condition is easily checked from the definition of the algorithm. □

Path Resolution. Next, we describe an algorithm that turns an encoding with respect to the public key, into an encoding with respect to the matrix encoded in a leaf. In more details, let

$$\mathbf{z}^\intercal = \mathbf{s}^\intercal \mathbf{G}\mathbf{Z} + \mathbf{e}^\intercal$$

where $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^k$ and $\mathbf{e} \xleftarrow{\$} \mathcal{D}_\sigma^{n\cdot t^2}$. The $\mathsf{TreeSolve}$ algorithm is described in the following.

- $\mathsf{TreeSolve}(\mathsf{pk}, \mathbf{z}, \alpha, \rho(\alpha))$: On input the public key pk, an encoding $\mathbf{z} \in \mathbb{Z}_q^{n\cdot t^2}$, a string $\alpha \in \{0,1\}^\ell$, and the corresponding path $\rho(\alpha) = \{\mathbf{x}_j, \mathbf{M}_j, \mathbf{c}_j\}_{j\in[\ell+1]}$, the algorithm proceeds as follows.

  Compute $\mathbf{P} = \mathsf{ExpandM}(\mathsf{pk})$, then set $\tilde{\mathbf{c}}_0 = \mathbf{c}_0$. For $j \in [\ell]$, compute the following:

  - Compute $\mathbf{Q}_{\mathbf{x}_j} = \mathsf{ExpandC}(\mathsf{pk}, \mathbf{x}_j)$.
  - Compute $\tilde{\mathbf{c}}_j^\intercal \cdot \mathbf{P} + \mathbf{z}^\intercal \cdot \mathbf{Q}_{\mathbf{x}_j} = \begin{bmatrix}\mathbf{d}_{j,0}^\intercal & \mathbf{d}_{j,1}^\intercal\end{bmatrix}$.
  - Set $\tilde{\mathbf{c}}_{j+1}^\intercal = \mathbf{d}_{j,\alpha_j}^\intercal \cdot \mathbf{V} + \mathbf{c}_{j+1}^\intercal$.

  Here $\mathbf{V} \in \{0,1\}^{(m\cdot n^4)\times(m\cdot n^2)}$ is defined to be the binary matrix such that for every $\mathbf{A} \in \mathbb{Z}_q^{n\times m\cdot n^2}$, we have

  $$(\mathsf{vec}(\mathbf{A})^\intercal \otimes \mathbf{I}_n) \cdot \mathbf{V} = \mathbf{A}. \tag{6}$$

  The algorithm returns $\tilde{\mathbf{c}}_\ell^\intercal$.

The following lemma shows that the output of the algorithm is a valid encoding with respect to the selected tree node. Moreover, we provide a bound on the magnitude of the noise, assuming that the path given as input to the algorithm is consistent.

Lemma 8 (Path Resolution)**.** *Let* $\mathbf{z}^\intercal = \mathbf{s}^\intercal \mathbf{G}\mathbf{Z} + \mathbf{e}^\intercal$ *be sampled as above,* $\alpha \in \{0,1\}^\ell$ *be a string,* $\rho(\alpha) = \{\mathbf{x}_j, \mathbf{M}_j, \mathbf{c}_j\}_{j\in[\ell+1]}$ *be the corresponding path such that, for all* $j \in [\ell+1]$*, we have*

$$\mathbf{c}_j^\intercal = \mathbf{s}^\intercal \mathbf{G}(\mathbf{M}_j - \mathsf{Hash}(\mathsf{pk}, \mathbf{x}_j)^\intercal \otimes \mathbf{I}_n) + \mathbf{e}_j^\intercal \qquad \textit{and} \qquad \mathbf{x}_{j,\alpha_j} = \mathsf{vec}(\mathbf{M}_{j+1}).$$

Then we have that

$$\mathsf{TreeSolve}(\mathsf{pk}, \mathbf{z}, \alpha, \rho(\alpha)) = \mathbf{s}^\intercal \mathbf{G}\left(\mathbf{M}_0 \prod_{j\in[\ell]} \left(\mathbf{P}_{\alpha_j}\mathbf{V}\right) - \mathsf{Hash}(\mathsf{pk}, \mathbf{x}_\ell)^\intercal \otimes \mathbf{I}_n\right) + \tilde{\mathbf{e}}^\intercal$$

with $\|\tilde{\mathbf{e}}\|_\infty \leq \sum_{k=0}^{\ell} \|\mathbf{e}_k\|_\infty \cdot (mn)^{\ell-k} + \sum_{k=1}^{\ell} \|\mathbf{M}_k\|_\infty \cdot t\sigma\sqrt{\lambda}(mn)^{\ell-k}$.

Proof. We claim that, for all $j \in [\ell+1]$, we have that

$$\tilde{\mathbf{c}}_j^\intercal = \mathbf{s}^\intercal \mathbf{G}\left(\mathbf{M}_0 \prod_{k\in[j]} (\mathbf{P}_{\alpha_k}\mathbf{V}) - (\mathsf{Hash}(\mathsf{pk}, \mathbf{x}_j)^\intercal \otimes \mathbf{I}_n)\right) + \tilde{\mathbf{e}}_j^\intercal \tag{7}$$

with $\|\tilde{\mathbf{e}}_j\|_\infty \leq \sum_{k=0}^{j} \|\mathbf{e}_k\|_\infty \cdot (mn)^{j-k} + \sum_{k=1}^{j} \|\mathbf{M}_k\|_\infty \cdot t\sigma\sqrt{\lambda}(mn)^{j-k}$. Proving Equation (7) concludes the proof, so all is left to show is that this holds by induction. For $j = 0$ this trivially holds because $\tilde{\mathbf{c}}_0 = \mathbf{c}_0$. For the inductive case, let us first substitute

$$\begin{aligned}
&\tilde{\mathbf{c}}_j^\intercal \cdot \mathbf{P} + \mathbf{z}^\intercal \cdot \mathbf{Q}_{\mathbf{x}_j} \\
&= \left(\mathbf{s}^\intercal \mathbf{G}\left(\mathbf{M}_0 \prod_{k\in[j]} (\mathbf{P}_{\alpha_k}\mathbf{V}) - (\mathsf{Hash}(\mathsf{pk}, \mathbf{x}_j)^\intercal \otimes \mathbf{I}_n)\right) + \tilde{\mathbf{e}}_j^\intercal\right)\mathbf{P} + (\mathbf{s}^\intercal \mathbf{G}\mathbf{Z} + \mathbf{e}^\intercal)\,\mathbf{Q}_{\mathbf{x}_j} \\
&= \mathbf{s}^\intercal \mathbf{G}\left(\mathbf{M}_0 \prod_{k\in[j]} (\mathbf{P}_{\alpha_k}\mathbf{V}) \cdot \mathbf{P} - \mathbf{x}_j^\intercal \otimes \mathbf{I}_n\right) + \tilde{\mathbf{e}}_j^\intercal \mathbf{P} + \mathbf{e}^\intercal \mathbf{Q}_{\mathbf{x}_j}
\end{aligned}$$

where the first equality follows by induction hypothesis and the second equality follows by Lemma 3. Splitting the vector into two components, selecting according to α_j, and using the fact that $\mathbf{x}_{j,\alpha_j} = \mathsf{vec}(\mathbf{M}_{j+1})$, we obtain

$$\mathbf{d}_{j,\alpha_j}^\intercal = \mathbf{s}^\intercal \mathbf{G}\left(\mathbf{M}_0 \prod_{k\in[j]} (\mathbf{P}_{\alpha_k}\mathbf{V}) \cdot \mathbf{P}_{\alpha_j} - \mathsf{vec}(\mathbf{M}_{j+1})^\intercal \otimes \mathbf{I}_n\right) + \tilde{\mathbf{e}}_j^\intercal \mathbf{P}_{\alpha_j} + \mathbf{e}^\intercal \mathbf{Q}_{\alpha_j,\mathbf{x}_j} \tag{8}$$

where $\mathbf{P} = \begin{bmatrix}\mathbf{P}_0 & \mathbf{P}_1\end{bmatrix}$ and $\mathbf{Q}_{\mathbf{x}_j} = \begin{bmatrix}\mathbf{Q}_{0,\mathbf{x}_j} & \mathbf{Q}_{1,\mathbf{x}_j}\end{bmatrix}$. Substituting Equation (8) to the last step of the algorithm, we obtain

$$\begin{aligned}
\tilde{\mathbf{c}}_{j+1}^{\intercal} &= \mathbf{d}_{j,\alpha_j}^{\intercal} \cdot \mathbf{V} + \mathbf{c}_{j+1}^{\intercal} \\
&= \mathbf{s}^{\intercal}\mathbf{G}\left(\mathbf{M}_0 \prod_{k\in[j+1]} (\mathbf{P}_{\alpha_k}\mathbf{V}) - \mathbf{M}_{j+1}\right) + \tilde{\mathbf{e}}_j^{\intercal}\mathbf{P}_{\alpha_j}\mathbf{V} + \mathbf{e}^{\intercal}\mathbf{Q}_{\alpha_j,\mathbf{x}_j}\mathbf{V} + \mathbf{c}_{j+1}^{\intercal} \\
&= \mathbf{s}^{\intercal}\mathbf{G}\left(\mathbf{M}_0 \prod_{k\in[j+1]} (\mathbf{P}_{\alpha_k}\mathbf{V}) - \mathsf{Hash}(\mathsf{pk},\mathbf{x}_j)^{\intercal} \otimes \mathbf{I}_n\right) + \tilde{\mathbf{e}}_j^{\intercal}\mathbf{P}_{\alpha_j}\mathbf{V} + \mathbf{e}^{\intercal}\mathbf{Q}_{\alpha_j,\mathbf{x}_j}\mathbf{V} + \mathbf{e}_{j+1}^{\intercal}.
\end{aligned}$$

Finally, we can bound

$$\begin{aligned}
\left\|\tilde{\mathbf{e}}_j^{\intercal}\mathbf{P}_{\alpha_j}\mathbf{V} + \mathbf{e}^{\intercal}\mathbf{Q}_{\alpha_j,\mathbf{x}_j}\mathbf{V} + \mathbf{e}_{j+1}^{\intercal}\right\|_\infty &\leq \left\|\tilde{\mathbf{e}}_j^{\intercal}\mathbf{P}_{\alpha_j}\mathbf{V}\right\|_\infty + \left\|\mathbf{e}^{\intercal}\mathbf{Q}_{\alpha_j,\mathbf{x}_j}\mathbf{V}\right\|_\infty + \left\|\mathbf{e}_{j+1}^{\intercal}\right\|_\infty \\
&\leq \left\|\tilde{\mathbf{e}}_j^{\intercal}\right\|_\infty \cdot mn + \|\mathbf{M}_{j+1}\|_\infty \cdot t\sigma\sqrt{\lambda} + \left\|\mathbf{e}_{j+1}^{\intercal}\right\|_\infty
\end{aligned}$$

by Lemma 3 and Equation (5), concluding our proof. □

3.2 Homomorphic Operations

In the following we describe how to homomorphically evaluate functions over paths. In short, we want to define operations that allow one to turn a collection of paths with arbitrary length and arbitrary leaf values, into a single path where the leaf is the output of a function f on input the leaves. Given that leaves encode vectors, all we need to show is how to perform component-wise addition and multiplication between two vectors, plus bounded-coefficient linear operations over a single vector. Given that permutations of the vector coordinates can be implemented with the latter, the above set of operations is sufficient to implement all functions. We are now ready to formally specify the homomorphic operations.

- (Linear Operations) On input a path $\rho(\alpha) = \{\mathbf{x}_j, \mathbf{M}_j, \mathbf{c}_j\}_{j\in[\ell+1]}$ and a linear function $\mathbf{F} : \mathbb{Z}_q^t \mapsto \mathbb{Z}_q^t$, the algorithm proceeds as follows.

  Let $\mathbf{F}_\ell = \mathbf{F}$, then for all $i \in [\ell]$ define $\mathbf{F}_i = \mathbf{I}_2 \otimes \tilde{\mathbf{F}}_i$, where $\tilde{\mathbf{F}}_i$ is the matrix generated by Lemma 5 on input $\mathbf{F}_{i+1}$. For $j \in [\ell+1]$:

  - Compute $\tilde{\mathbf{x}}_j = \mathbf{F}_j \cdot \mathbf{x}_j$.
  - Compute $\mathbf{K}_{\mathbf{F}_j,\mathbf{M}_j} = \mathsf{EvalM}(\mathsf{pk},\mathbf{F}_j,\mathbf{M}_j)$ and $\tilde{\mathbf{M}}_j = \mathbf{M}_j \cdot \mathbf{K}_{\mathbf{F}_j,\mathbf{M}_j}$.
  - Compute $\mathbf{H}_{\mathbf{F}_j,\mathbf{M}_j,\mathbf{x}_j} = \mathsf{EvalC}(\mathsf{pk},\mathbf{F}_j,\mathbf{M}_j,\mathbf{x}_j)$ and $\tilde{\mathbf{c}}_j^{\intercal} = \begin{bmatrix}\mathbf{c}_j^{\intercal} & \mathbf{z}^{\intercal}\end{bmatrix} \cdot \mathbf{H}_{\mathbf{F}_j,\mathbf{M}_j,\mathbf{x}_j}$.

  Return $\left\{\tilde{\mathbf{x}}_j, \tilde{\mathbf{M}}_j, \tilde{\mathbf{c}}_j\right\}_{j\in[\ell+1]}$.

- (Addition) On input two paths

$$\rho(\alpha)^{(0)} = \left\{\mathbf{x}_j^{(0)}, \mathbf{M}_j^{(0)}, \mathbf{c}_j^{(0)}\right\}_{j\in[\ell+1]} \qquad \text{and} \qquad \rho(\alpha)^{(1)} = \left\{\mathbf{x}_j^{(1)}, \mathbf{M}_j^{(1)}, \mathbf{c}_j^{(1)}\right\}_{j\in[\ell+1]}$$

  the addition algorithm returns

$$\left\{\mathbf{x}_j^{(0)} + \mathbf{x}_j^{(1)}, \mathbf{M}_j^{(0)} + \mathbf{M}_j^{(1)}, \mathbf{c}_j^{(0)} + \mathbf{c}_j^{(1)}\right\}_{j\in[\ell+1]}.$$

- (Multiplication) On input two paths

$$\rho(\alpha)^{(0)} = \left\{\mathbf{x}_j^{(0)}, \mathbf{M}_j^{(0)}, \mathbf{c}_j^{(0)}\right\}_{j\in[\ell+1]} \qquad \text{and} \qquad \rho(\alpha)^{(1)} = \left\{\mathbf{x}_j^{(1)}, \mathbf{M}_j^{(1)}, \mathbf{c}_j^{(1)}\right\}_{j\in[\ell+1]}$$

  and a bilinear function $\mathbf{F} : \mathbb{Z}_q^{t^2} \mapsto \mathbb{Z}_q^t$, let us denote

$$\mathbf{M}_j = \begin{bmatrix}\mathbf{M}_j^{(0)} & \mathbf{M}_j^{(1)}\end{bmatrix} \qquad \mathbf{x}_j = \begin{bmatrix}\mathbf{x}_j^{(0)} \\ \mathbf{x}_j^{(1)}\end{bmatrix} \qquad \mathbf{c}_j = \begin{bmatrix}\mathbf{c}_j^{(0)} \\ \mathbf{c}_j^{(1)}\end{bmatrix}.$$

  The algorithm proceeds as follows.

  Let $\mathbf{F}_\ell = \mathbf{F}$ and let $\mathbf{S}$ to be the permutation matrix mapping

$$\mathbf{S}\begin{bmatrix}\mathbf{x}_0 \\ \mathbf{x}_1\end{bmatrix} \otimes \begin{bmatrix}\mathbf{y}_0 \\ \mathbf{y}_1\end{bmatrix} = \begin{bmatrix}\mathbf{x}_0 \otimes \mathbf{y}_0 \\ \mathbf{x}_0 \otimes \mathbf{y}_1 \\ \mathbf{x}_1 \otimes \mathbf{y}_0 \\ \mathbf{x}_1 \otimes \mathbf{y}_1\end{bmatrix}.$$

  Then for all $i \in [\ell]$ let $\tilde{\mathbf{F}}_i$ be the matrix generated by Lemma 6 on input $\mathbf{F}_{i+1}$. Define

$$\mathbf{F}_i = \begin{bmatrix}\tilde{\mathbf{F}}_i & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \tilde{\mathbf{F}}_i\end{bmatrix}\mathbf{S}. \tag{9}$$

  For $j \in [\ell+1]$:

  - Compute $\tilde{\mathbf{x}}_j = \mathbf{F}_j \cdot (\mathbf{x}_j^{(0)} \otimes \mathbf{x}_j^{(1)})$.
  - Compute $\mathbf{K}_{\mathbf{F}_j,\mathbf{M}_j} = \mathsf{EvalM}\,(\mathsf{pk}, \mathbf{F}_j, \mathbf{M}_j)$ and $\tilde{\mathbf{M}}_j = \mathbf{M}_j \cdot \mathbf{K}_{\mathbf{F}_j,\mathbf{M}_j}$.
  - Compute $\mathbf{H}_{\mathbf{F}_j,\mathbf{M}_j,\mathbf{x}_j} = \mathsf{EvalC}\,(\mathsf{pk}, \mathbf{F}_j, \mathbf{M}_j, \mathbf{x}_j)$ and $\tilde{\mathbf{c}}_j^\intercal = \begin{bmatrix}\mathbf{c}_j^\intercal & \mathbf{z}^\intercal\end{bmatrix} \cdot \mathbf{H}_{\mathbf{F}_j,\mathbf{M}_j,\mathbf{x}_j}$.

  Return $\left\{\tilde{\mathbf{x}}_j, \tilde{\mathbf{M}}_j, \tilde{\mathbf{c}}_j\right\}_{j\in[\ell+1]}$.

Next, we prove correctness of the homomorphic operations. Assuming that $\mathbf{z}^\intercal = \mathbf{s}^\intercal\mathbf{G}\mathbf{Z} + \mathbf{e}^\intercal$, and that the inputs paths $\rho(\alpha) = \{\mathbf{x}_j, \mathbf{M}_j, \mathbf{c}_j\}_{j\in[\ell+1]}$ are such that for all $j \in [\ell+1]$, we have

$$\mathbf{c}_j^\intercal = \mathbf{s}^\intercal\mathbf{G}(\mathbf{M}_j - \mathsf{Hash}(\mathsf{pk}, \mathbf{x}_j)^\intercal \otimes \mathbf{I}_n) + \mathbf{e}_j^\intercal \qquad \text{and} \qquad \mathbf{x}_{j,\alpha_j} = \mathsf{vec}(\mathbf{M}_{j+1}).$$

Then the following bounds hold.

Claim 1 (Correctness of Linear Operations)**.** *Let $\tilde{\rho}(\alpha)$ be the output of the linear algorithm, then $\tilde{\mathbf{x}}_\ell = \mathbf{F} \cdot \mathbf{x}_\ell$ and for all $j \in [\ell+1]$ we have:*

$$\|\tilde{\mathbf{M}}_j\|_\infty \leq \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot \|\mathbf{M}_j\|_\infty \cdot (mnt\sigma\sqrt{\lambda})^{\ell-j+1}$$

and

$$\|\tilde{\mathbf{e}}_j\|_\infty \leq \|\mathbf{e}_j\|_\infty \cdot \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot (mnt\sigma\sqrt{\lambda})^{\ell-j+1} + \|\mathbf{M}_{j+1}\|_\infty \cdot \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot t^2\sigma\sqrt{\lambda} \cdot (mnt\sigma\sqrt{\lambda})^{\ell-j}.$$

Proof. For all $j \in [\ell+1]$ we have

$$\begin{aligned}\tilde{\mathbf{c}}_j^\intercal &= \mathbf{s}^\intercal \mathbf{G}(\mathbf{M}_j \mathbf{K}_{\mathbf{F}_j,\mathbf{M}_j} - \mathsf{Hash}(\mathsf{pk}, \mathbf{F}_j \cdot \mathbf{x}_j)^\intercal \otimes \mathbf{I}_n) + \mathbf{e}_j^\intercal \mathbf{H}_0 + \mathbf{e}^\intercal \mathbf{H}_1 \\ &= \mathbf{s}^\intercal \mathbf{G}(\tilde{\mathbf{M}}_j - \mathsf{Hash}(\mathsf{pk}, \tilde{\mathbf{x}}_j)^\intercal \otimes \mathbf{I}_n) + \tilde{\mathbf{e}}_j^\intercal\end{aligned}$$

by Lemma 4. By Lemma 5, we can bound the norm $\|\mathbf{F}_j^\intercal\|_{\infty,\infty} = \|\tilde{\mathbf{F}}_j^\intercal\|_{\infty,\infty} \leq \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot (mnt\sigma\sqrt{\lambda})^{\ell-j}$ which allows us to bound the noise terms

$$\begin{aligned}\|\tilde{\mathbf{e}}_j\|_\infty &\leq \|\mathbf{e}_j^\intercal \mathbf{H}_0\|_\infty + \|\mathbf{e}^\intercal \mathbf{H}_1\|_\infty \\ &\leq \|\mathbf{e}_j\|_\infty \cdot \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot (mnt\sigma\sqrt{\lambda})^{\ell-j+1} + \|\mathbf{M}_{j+1}\|_\infty \cdot \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot t^2\sigma\sqrt{\lambda} \cdot (mnt\sigma\sqrt{\lambda})^{\ell-j}\end{aligned}$$

using again Lemma 5. By definition, $\tilde{\mathbf{x}}_\ell = \mathbf{F}_\ell \cdot \mathbf{x}_\ell = \mathbf{F} \cdot \mathbf{x}_\ell$, whereas for $j \in [\ell]$ we have

$$\tilde{\mathbf{x}}_j = \mathbf{F}_j \cdot \mathbf{x}_j = \mathbf{I}_2 \otimes \tilde{\mathbf{F}}_j \cdot \mathbf{x}_j = \begin{bmatrix} \tilde{\mathbf{F}}_j \cdot \mathbf{x}_{j,0} \\ \tilde{\mathbf{F}}_j \cdot \mathbf{x}_{j,1} \end{bmatrix}.$$

Therefore, by Lemma 5, we can rewrite

$$\tilde{\mathbf{x}}_{j,\alpha_j} = \tilde{\mathbf{F}}_j \cdot \mathbf{x}_{j,\alpha_j} = \tilde{\mathbf{F}}_j \cdot \mathsf{vec}(\mathbf{M}_{j+1}) = \mathsf{vec}(\mathbf{M}_{j+1}\mathbf{K}_{\mathbf{F}_{j+1},\mathbf{M}_{j+1}}) = \mathsf{vec}(\tilde{\mathbf{M}}_{j+1})$$

as expected. Another application of Lemma 5 yields

$$\|\tilde{\mathbf{M}}_{j+1}\|_\infty \leq \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot (mnt\sigma\sqrt{\lambda})^{\ell-j} \cdot \|\mathbf{M}_{j+1}\|_\infty$$

concluding our proof. □

Claim 2 (Correctness of Addition)**.** *Let $\tilde{\rho}(\alpha)$ be the output of the addition algorithm, then $\tilde{\mathbf{x}}_\ell = \mathbf{x}_\ell^{(0)} + \mathbf{x}_\ell^{(1)}$ and for all $j \in [\ell+1]$ we have:*

$$\|\tilde{\mathbf{M}}_j\|_\infty \leq \|\mathbf{M}_j^{(0)}\|_\infty + \|\mathbf{M}_j^{(1)}\|_\infty \qquad \textit{and} \qquad \|\tilde{\mathbf{e}}_j\|_\infty \leq \|\mathbf{e}_j^{(0)}\|_\infty + \|\mathbf{e}_j^{(1)}\|_\infty.$$

Proof. The claim follows as a straightforward consequence of the linearity of addition and a triangle inequality. □

Claim 3 (Correctness of Multiplication)**.** *Let $\tilde{\rho}(\alpha)$ be the output of the multiplication algorithm, then $\tilde{\mathbf{x}}_\ell = \mathbf{F} \cdot (\mathbf{x}_\ell^{(0)} \otimes \mathbf{x}_\ell^{(1)})$ and for all $j \in [\ell+1]$ we have*

$$\|\tilde{\mathbf{M}}_j\|_\infty \leq \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot \|\mathbf{M}_j\|_\infty \cdot (m^2 n^3 t\sigma\sqrt{\lambda})^{\ell-j+1}$$

where $\mathbf{M}_j = \begin{bmatrix} \mathbf{M}_j^{(0)} & \mathbf{M}_j^{(1)} \end{bmatrix}$. Moreover, we have

$$\|\tilde{\mathbf{e}}_j\|_\infty \leq \left(\|\mathbf{e}_j^{(0)}\|_\infty \cdot \|\mathbf{M}_j^{(1)}\|_\infty + \|\mathbf{e}_j^{(1)}\|_\infty \cdot \|\mathbf{x}_j^{(0)}\|_\infty + (\|\mathbf{M}_j^{(1)}\|_\infty \cdot mn + 2) \cdot \|\mathbf{x}_j\|_\infty \cdot \frac{t\sigma\sqrt{\lambda}}{m^2 n^3} \right)$$
$$\cdot \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot (m^2 n^3 t\sigma\sqrt{\lambda})^{\ell-j+1}.$$

where $\mathbf{x}_j = \begin{bmatrix} \mathbf{x}_j^{(0)} & \mathbf{x}_j^{(1)} \end{bmatrix}$.

Proof. Let us define

$$\mathbf{M}_j = \begin{bmatrix} \mathbf{M}_j^{(0)} & \mathbf{M}_j^{(1)} \end{bmatrix} \qquad \mathbf{x}_j = \begin{bmatrix} \mathbf{x}_j^{(0)} \\ \mathbf{x}_j^{(1)} \end{bmatrix} \qquad \mathbf{c}_j = \begin{bmatrix} \mathbf{c}_j^{(0)} \\ \mathbf{c}_j^{(1)} \end{bmatrix}$$

as in the multiplication algorithm. By Lemma 4 we can rewrite

$$\begin{aligned} \tilde{\mathbf{c}}_j^\intercal &= \mathbf{s}^\intercal \mathbf{G}(\mathbf{M}_j \mathbf{K}_{\mathbf{F}_j,\mathbf{M}_j} - \mathsf{Hash}(\mathsf{pk}, \mathbf{F}_j \cdot (\mathbf{x}_j^{(0)} \otimes \mathbf{x}_j^{(1)}))^\intercal \otimes \mathbf{I}_n) + \sum_{b\in\{0,1\}} (\mathbf{e}_j^{(b)})^\intercal \mathbf{H}_b + \mathbf{e}^\intercal \mathbf{H}_2 \\ &= \mathbf{s}^\intercal \mathbf{G}(\tilde{\mathbf{M}}_j - \mathsf{Hash}(\mathsf{pk}, \tilde{\mathbf{x}}_j)^\intercal \otimes \mathbf{I}_n) + \tilde{\mathbf{e}}_j^\intercal. \end{aligned}$$

By Lemma 6, we have $\|\mathbf{F}_j^\intercal\|_{\infty,\infty} = \|\tilde{\mathbf{F}}_j^\intercal\|_{\infty,\infty} \leq \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot (m^2 n^3 t\sigma\sqrt{\lambda})^{\ell-j}$ which allows us to bound

$$\begin{aligned} \|\tilde{\mathbf{e}}_j\|_\infty &\leq \|(\mathbf{e}_j^{(0)})^\intercal \mathbf{H}_0\|_\infty + \|(\mathbf{e}_j^{(1)})^\intercal \mathbf{H}_1\|_\infty + \|\mathbf{e}^\intercal \mathbf{H}_2\|_\infty \\ &\leq \|\mathbf{e}_j^{(0)}\|_\infty \cdot \|\mathbf{M}_j^{(1)}\|_\infty \cdot \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot (m^2 n^3 t\sigma\sqrt{\lambda})^{\ell-j+1} \\ &\quad + \|\mathbf{e}_j^{(1)}\|_\infty \cdot \|\mathbf{x}_j^{(0)}\|_\infty \cdot \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot (m^2 n^3 t\sigma\sqrt{\lambda})^{\ell-j+1} \cdot \frac{1}{n^2 m} \\ &\quad + (\|\mathbf{M}_j^{(1)}\|_\infty \cdot mn + 2) \cdot \|\mathbf{x}_j\|_\infty \cdot \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot (m^2 n^3 t\sigma\sqrt{\lambda})^{\ell-j+1} \cdot \frac{t\sigma\sqrt{\lambda}}{m^2 n^3}. \end{aligned}$$

Moreover, $\tilde{\mathbf{x}}_\ell = \mathbf{F} \cdot (\mathbf{x}_\ell^{(0)} \otimes \mathbf{x}_\ell^{(1)})$ and for $j \in [\ell]$, note that

$$\tilde{\mathbf{x}}_j = \mathbf{F}_j \cdot (\mathbf{x}_j^{(0)} \otimes \mathbf{x}_j^{(1)}) = \begin{bmatrix} \tilde{\mathbf{F}}_j \cdot (\mathbf{x}_{j,0}^{(0)} \otimes \mathbf{x}_{j,0}^{(1)}) \\ \tilde{\mathbf{F}}_j \cdot (\mathbf{x}_{j,1}^{(0)} \otimes \mathbf{x}_{j,1}^{(1)}) \end{bmatrix}$$

by definition of $\mathbf{F}_j$ in Equation (9). Consequently, by Lemma 6, we can rewrite

$$\tilde{\mathbf{x}}_{j,\alpha_j} = \tilde{\mathbf{F}}_j \cdot (\mathbf{x}_{j,\alpha_j}^{(0)} \otimes \mathbf{x}_{j,\alpha_j}^{(1)}) = \tilde{\mathbf{F}}_j \cdot (\mathsf{vec}(\mathbf{M}_{j+1}^{(0)}) \otimes \mathsf{vec}(\mathbf{M}_{j+1}^{(1)})) = \mathsf{vec}(\mathbf{M}_{j+1} \mathbf{K}_{\mathbf{F}_{j+1},\mathbf{M}_{j+1}}) = \mathsf{vec}(\tilde{\mathbf{M}}_{j+1}).$$

Another application of Lemma 6 allows us to bound

$$\|\tilde{\mathbf{M}}_{j+1}\|_\infty \leq \|\mathbf{F}^\intercal\|_{\infty,\infty} \cdot \|\mathbf{M}_{j+1}\|_\infty \cdot (m^2 n^3 t\sigma\sqrt{\lambda})^{\ell-j}$$

as desired. □

3.3 Flattening

Next, we describe a generic procedure to reset the norm on the matrices belonging to a given path of the hash. In line with the cryptographic literature on lattices, we refer to such procedure as *flattening*. We shall think of this flattening procedure as being executed after every homomorphic operation and for every node in the path, starting from the root.

Before formally describing our procedure, let us define some notation. For a given vector $\mathbf{x} \in \mathbb{Z}_q^{m \cdot n^3}$, let $\mathbf{X} \in \mathbb{Z}_q^{n \times (m \cdot n^2)}$ such that $\mathsf{vec}(\mathbf{X}) = \mathbf{x}$. Then we define the linear operator $\mathbf{R} \in \mathbb{Z}_q^{t \times (m \cdot n^3)}$ as

$$\mathbf{R} \cdot \mathbf{x} = \begin{bmatrix} \mathsf{vec}(\mathbf{X} \cdot \mathbf{P}_0 \cdot \mathbf{V}) \\ \mathsf{vec}(\mathbf{X} \cdot \mathbf{P}_1 \cdot \mathbf{V}) \end{bmatrix} \tag{10}$$

where $\begin{bmatrix} \mathbf{P}_0 & \mathbf{P}_1 \end{bmatrix} = \mathbf{P} = \mathsf{ExpandM}(\mathsf{pk})$ and $\mathbf{V} \in \{0,1\}^{(m \cdot n^4) \times (m \cdot n^2)}$ is defined as in Equation (6). Similarly, we define the operator $\mathbf{L} \in \mathbb{Z}_q^{(mn^3) \times t}$ as

$$\mathbf{L} \cdot \mathbf{x} = \mathsf{vec}(\mathsf{Hash}(\mathsf{pk}, \mathbf{x})^\intercal \otimes \mathbf{I}_n). \tag{11}$$

Finally, for all $j \in [\ell]$ we let

$$\mathbf{W}_j = \mathbf{L} \cdot (\mathbf{I}_2 \otimes \mathbf{W}_{j+1}) \cdot \mathbf{R} \tag{12}$$

with $\mathbf{W}_\ell = \mathbf{I}_{mn^3}$.

- $\mathsf{Flatten}(\mathsf{pk}, \mathbf{z}, \alpha, \rho(\alpha))$: On input the public key pk, an encoding $\mathbf{z} \in \mathbb{Z}_q^{n \cdot t^2}$, a string $\alpha \in \{0,1\}^\ell$, and the corresponding path $\rho(\alpha) = \{\mathbf{x}_j, \mathbf{M}_j, \mathbf{c}_j\}_{j \in [\ell+1]}$, the algorithm proceeds as follows.

  Let $\mathbf{x}_0$ be the value of the root, redefine

  $$\mathbf{M}_0 = \mathbf{G}^{-1}(\mathsf{Hash}(\mathsf{pk}, 1, \mathbf{x}_0) \otimes \mathbf{G}) \qquad \mathbf{c}_0 = \mathbf{0}^{m \cdot n^2}$$

  and set $\mathbf{m}_0 = \mathsf{vec}(\mathbf{M}_0)$. Then, for $j \in [\ell + 1]$ proceed as follows:

  - Define $\tilde{\mathbf{M}}_j \in \mathbb{Z}_q^{n \times (m \cdot n^2)}$ to be the matrix such that $\mathbf{W}_j \cdot \mathbf{m}_j = \mathsf{vec}(\tilde{\mathbf{M}}_j)$.
  - Define $\begin{bmatrix} \mathbf{m}_{j+1,0} \\ \mathbf{m}_{j+1,1} \end{bmatrix} = \mathbf{R} \cdot \mathbf{m}_j$ and let $\mathbf{m}_{j+1} = \mathbf{m}_{j+1,\alpha_j}$.
  - If $j \neq \ell$: Define $\tilde{\mathbf{c}}_j = \mathbf{0}^{m \cdot n^2}$ and $\tilde{\mathbf{x}}_j = (\mathbf{I}_2 \otimes \mathbf{W}_{j+1}) \cdot \mathbf{R} \cdot \mathbf{m}_j$.
  - If $j = \ell$: Define $\tilde{\mathbf{c}}_\ell = \mathsf{TreeSolve}(\mathsf{pk}, \mathbf{z}, \alpha, \rho(\alpha))$ and $\tilde{\mathbf{x}}_\ell = \mathbf{x}_\ell$.

  Return the updated path $\{\tilde{\mathbf{x}}_j, \tilde{\mathbf{M}}_j, \tilde{\mathbf{c}}_j\}_{j \in [\ell+1]}$.

Note that the flattening operation does not change the value of the leaf since $\tilde{\mathbf{x}}_\ell = \mathbf{x}_\ell$. Next, we want to argue that the output of the flattening algorithm is still a consistent path. Towards showing this, observe that, for all $j \in [\ell]$, we have that

$$\mathsf{vec}(\tilde{\mathbf{M}}_j) = \mathbf{W}_j \cdot \mathbf{m}_j = \mathbf{L} \cdot (\mathbf{I}_2 \otimes \mathbf{W}_{j+1}) \cdot \mathbf{R} \cdot \mathbf{m}_j = \mathbf{L} \cdot \tilde{\mathbf{x}}_j = \mathsf{vec}(\mathsf{Hash}(\mathsf{pk}, \tilde{\mathbf{x}}_j)^\intercal \otimes \mathbf{I}_n)$$

by Equation (12) and Equation (11). Consequently, $\tilde{\mathbf{M}}_j = \mathsf{Hash}(\mathsf{pk}, \tilde{\mathbf{x}}_j)^\intercal \otimes \mathbf{I}_n$ and therefore

$$\tilde{\mathbf{c}}_j^\intercal = \mathbf{s}^\intercal \mathbf{G}(\tilde{\mathbf{M}}_j - \mathsf{Hash}(\mathsf{pk}, \tilde{\mathbf{x}}_j)^\intercal \otimes \mathbf{I}_n) = \mathbf{0}. \tag{13}$$

On the other hand, we have that

$$\tilde{\mathbf{x}}_j = (\mathbf{I}_2 \otimes \mathbf{W}_{j+1}) \cdot \mathbf{R} \cdot \mathbf{m}_j = \begin{bmatrix} \mathbf{W}_{j+1} \cdot \mathbf{m}_{j+1,0} \\ \mathbf{W}_{j+1} \cdot \mathbf{m}_{j+1,1} \end{bmatrix}$$

and by definition

$$\tilde{\mathbf{x}}_{j,\alpha_j} = \mathbf{W}_{j+1} \cdot \mathbf{m}_{j+1,\alpha_j} = \mathbf{W}_{j+1} \cdot \mathbf{m}_{j+1} = \mathsf{vec}(\tilde{\mathbf{M}}_{j+1}). \tag{14}$$

Equation (13) and Equation (14) show that the path is consistent, up to the last node. By Lemma 8 we know that

$$\tilde{\mathbf{c}}_\ell^\intercal = \mathbf{s}^\intercal \mathbf{G} \left(\mathbf{M}_0 \prod_{j \in [\ell]} \left(\mathbf{P}_{\alpha_j} \mathbf{V} \right) - \mathsf{Hash}(\mathsf{pk}, \mathbf{x}_\ell)^\intercal \otimes \mathbf{I}_n \right) + \tilde{\mathbf{e}}_\ell^\intercal$$

and furthermore $\mathbf{x}_\ell = \tilde{\mathbf{x}}_\ell$. So all is left to be shown is that $\tilde{\mathbf{M}}_\ell = \mathbf{M}_0 \prod_{j \in [\ell]} \left(\mathbf{P}_{\alpha_j} \mathbf{V} \right)$. We show this by an inductive argument, where our hypothesis is that

$$\mathbf{m}_j = \mathsf{vec} \left(\mathbf{M}_0 \prod_{k \in [j]} \left(\mathbf{P}_{\alpha_k} \mathbf{V} \right) \right) \tag{15}$$

for all $j \in [\ell + 1]$, adopting the convention that $[0] = \{\}$. The base case is immediate since $\mathbf{m}_0 = \mathsf{vec}(\mathbf{M}_0)$. For the inductive step, observe that Equation (10) allows us to rewrite

$$\mathbf{R} \cdot \mathbf{m}_j = \begin{bmatrix} \mathbf{m}_{j+1,0} \\ \mathbf{m}_{j+1,1} \end{bmatrix} = \begin{bmatrix} \mathsf{vec}(\mathbf{M}'_j \cdot \mathbf{P}_0 \cdot \mathbf{V}) \\ \mathsf{vec}(\mathbf{M}'_j \cdot \mathbf{P}_1 \cdot \mathbf{V}) \end{bmatrix}$$

where $\mathbf{M}'_j$ is such that $\mathbf{m}_j = \mathsf{vec}(\mathbf{M}'_j)$. By induction hypothesis, we have that $\mathbf{M}'_j = \mathbf{M}_0 \prod_{k \in [j]} \left(\mathbf{P}_{\alpha_k} \mathbf{V} \right)$ and thus we can conclude that

$$\mathbf{m}_{j+1} = \mathbf{m}_{j+1,\alpha_j} = \mathsf{vec} \left(\mathbf{M}_0 \cdot \prod_{k \in [j+1]} \left(\mathbf{P}_{\alpha_k} \mathbf{V} \right) \right)$$

as desired. Finally, appealing to Equation (15) with j set to ℓ, we obtain

$$\mathsf{vec}(\tilde{\mathbf{M}}_\ell) = \mathbf{W}_\ell \cdot \mathbf{m}_\ell = \mathsf{vec} \left(\mathbf{M}_0 \prod_{j \in [\ell]} \left(\mathbf{P}_{\alpha_j} \mathbf{V} \right) \right)$$

concluding the argument. The following establishes a bound on the norm of the updated node of the tree.

Claim 4 (Correctness of Flattening)**.** *Let $\tilde{\rho}(\alpha)$ be the output of the flattening algorithm, then for all $j \in [\ell+1]$ we have*

$$\|\tilde{\mathbf{M}}_j\|_\infty \leq (mnt\sigma\sqrt{\lambda})^{\ell-j} \cdot (mn)^j.$$

Moreover, for $j \in [\ell]$ we have $\|\tilde{\mathbf{e}}_j\|_\infty = 0$ and

$$\|\tilde{\mathbf{e}}_\ell\|_\infty \leq \sum_{k=0}^{\ell} \|\mathbf{e}_k\|_\infty \cdot (mn)^{\ell-k} + \sum_{k=1}^{\ell} \|\mathbf{M}_k\|_\infty \cdot t\sigma\sqrt{\lambda}(mn)^{\ell-k}.$$

Proof. The bound on the noise terms follows immediately by Equation (13) and by Lemma 8. On the other hand, for the matrix terms, we have

$$\|\tilde{\mathbf{M}}_j\|_\infty = \|\mathbf{W}_j \cdot \mathbf{m}_j\|_\infty \leq \|\mathbf{W}_j^\intercal\|_{\infty,\infty} \cdot \|\mathbf{m}_j\|_\infty.$$

To bound the first factor, we recall that $\mathbf{W}_j = \mathbf{L} \cdot (\mathbf{I}_2 \otimes \mathbf{W}_{j+1}) \cdot \mathbf{R}$. Expanding and solving the recursive relation with base $\mathbf{W}_\ell = \mathbf{I}_{mn^3}$, we find that

$$\|\mathbf{W}_j^\intercal\|_{\infty,\infty} \leq (mnt\sigma\sqrt{\lambda})^{\ell-j}$$

To bound the second factor we rewrite

$$\|\mathbf{m}_j\|_\infty = \|\mathbf{M}_0 \prod_{k\in[j]} (\mathbf{P}_{\alpha_k}\mathbf{V})\|_\infty \leq \|\mathbf{M}_0\|_\infty \cdot (mn)^j = (mn)^j$$

concluding the proof. □

3.4 Putting Things Together

We discuss how to use the algorithms defined above to homomorphically compute over encodings. We run the setup algorithm $\mathsf{TreeSetup}$ to generate the public parameters pk. Then, on input a given index $\alpha \in \{0,1\}^d$, we generate the encodings by calling

$$\mathsf{TreeInit}(\mathsf{pk}, d, \mathbf{M}, 0, 0^d), \ldots, \mathsf{TreeInit}(\mathsf{pk}, d, \mathbf{M}, d-1, 0^d) \qquad \text{with} \qquad \mathbf{M} = \mathbf{u}_1\mathbf{u}_1^\intercal$$

where $\mathbf{u}_1$ is the first standard basis vector. This results into a collection of paths $\{\rho(\alpha)^{(i)}\}_{i\in[d]}$. By construction, running the same algorithms with a prefix of α will result in prefixes of such paths. If we instead input a different index α' then it is also easy to see that the paths will match up until the first index k such that $\alpha_k \neq \alpha'_k$. Moreover, denoting by $\mathbf{x}_d^{(i)}$ the leaf of the i-th path, we have

$$\mathbf{x}_d^{(i)} = \begin{cases} \mathsf{vec}(\mathbf{M}) & \text{if } \alpha_i = 1 \\ \mathbf{0} & \text{otherwise.} \end{cases}$$

It is therefore useful to think of the collection of paths as encoding α in the leaves, using the binary alphabet $\{\mathsf{vec}(\mathbf{M}), \mathbf{0}\}$. Given a Boolean function $f : \{0,1\}^d \mapsto \{0,1\}$, we can

compute a path whose leaf is $f(\alpha) \cdot \mathsf{vec}(\mathbf{M})$ as follows: Consider a circuit representation of f, each gate can be mapped as a vector operation consisting of (i) component-wise addition, (ii) component-wise multiplication, and (iii) permutations. Thus, it suffices to apply the operations presented in Section 3.2 to the input paths, and all is left to be shown is a bound on the growth of the norm of the matrices and on the noise, induced by each operation. We discuss the case of the multiplication algorithm, since it results into the worst bound, and linear operations and addition follow along the same lines.

Prior to any homomorphic operation, for any given path it holds that the maximal norm of the noise and of the matrices is bounded by

$$\max_k \|\mathbf{M}_k\|_\infty \leq 1 \qquad \text{and} \qquad \max_k \|\mathbf{e}_k\|_\infty = 0.$$

After applying the multiplication algorithm, by Claim 3 the norms of the resulting path can be bounded as

$$\max_k \|\tilde{\mathbf{M}}_k\|_\infty \leq \max_k \|\mathbf{M}_k\|_\infty \cdot (mnt\sigma\sqrt{\lambda})^{10d}$$

and

$$\max_k \|\tilde{\mathbf{e}}_k\|_\infty \leq \max_k \|\mathbf{e}_k\|_\infty \cdot \max_k \|\mathbf{M}_k\|_\infty^2 \cdot 10(mnt\sigma\sqrt{\lambda})^{10d}$$

where the quantities on the RHS denote the maximum over both input paths and we used the fact that $\max_k \|\mathbf{x}_k\|_\infty \leq \max_k \|\mathbf{M}_k\|_\infty$ and $\|\mathbf{F}^\intercal\|_{\infty,\infty} = 1$, since it suffices to consider component-wise products. To avoid an exponential growth on the matrix norms, we have to run the flattening algorithm on the path. In fact, we can think of the flattening algorithm being run after *every* homomorphic operation. To flatten, we will invoke the routine described in Section 3.3 on input α. By Claim 4, we can bound the updated norms (after all the d-many calls) as

$$\max_k \|\tilde{\mathbf{M}}_k\|_\infty \leq (mnt\sigma\sqrt{\lambda})^d$$

and

$$\max_k \|\tilde{\mathbf{e}}_k\|_\infty \leq \left(\max_k \|\mathbf{e}_k\|_\infty + \max_k \|\mathbf{M}_k\|_\infty\right) \cdot (d+1)(mnt\sigma\sqrt{\lambda})^d$$

where the quantities on the RHS refer to the norms of the path before any call of the flattening algorithm. Importantly, observe that the bound on the matrix norm is fixed and in particular independent on the input path.

Alternating homomorphic operations with flattening and solving the recursive relation on the noise bound we obtain the following statement.

Lemma 9 (Circuit Evaluation)**.** *Let $\alpha \in \{0,1\}^d$, let $\{\rho(\alpha)^{(i)} = \mathsf{TreeInit}(\mathsf{pk}, d, \mathbf{M}, i, \alpha)\}_{i\in[d]}$ for pk sampled as above, let $\mathbf{z} = \mathbf{s}^\intercal \mathbf{G}\mathbf{Z} + \mathbf{e}^\intercal$, and let $f : \{0,1\}^d \mapsto \{0,1\}$ be a function computable by a circuit of depth δ. Then there exists a polynomial-time algorithm $\mathsf{TreeEval}(\mathsf{pk}, f, \mathbf{z}, \{\rho(\alpha)^{(i)}\}_{i\in[d]})$ that returns a path $\tilde{\rho}(\alpha)$ with leaf $f(\alpha) \cdot \mathsf{vec}(\mathbf{M})$ and*

$$\max_k \|\tilde{\mathbf{M}}_k\|_\infty \leq (mnt\sigma\sqrt{\lambda})^d \qquad \textit{and} \qquad \max_k \|\tilde{\mathbf{e}}_k\|_\infty \leq (mnt\sigma\sqrt{\lambda})^{O(d\delta)} \cdot 2^{O(\delta)} \cdot d^{O(\delta)}.$$

We can also extend Lemma 9 to outputs in $\mathbb{Z}_q$, i.e., the final path encodes a vector $\mathbf{y} \in \mathbb{Z}_q^t$, which is defined to be the output of $f(\alpha)$. This can be done by computing the output bit-by-bit, then increasing each bit by the appropriate power of two (for instance by adding the path to itself and squaring it appropriately many times), and finally summing up all output paths, permuting if appropriate. We omit details since we do not need such an extension in this work.

4 Laconic Shift-Hiding Pseudorandom Function

In the following we present a shift-hiding pseudorandom function (PRF) [PS18] that satisfies the *laconic property*, i.e., its master secret key does not depend on the complexity of the shift function (except for its circuit depth).

4.1 Laconic Function Evaluation

We recall the notion of laconic function evaluation LFE [QWW18]. For notational convenience, we are going to adopt a slightly modified syntax and security notion, which follow from routine modifications of existing work. To keep things simple, we are going to explicitly use the scheme from [QWW18] along with the GSW [GSW13] fully homomorphic encryption (FHE), although more recent constructions [AMR25b, AMR25a] would also work. We leave the exploration of different tradeoff offered by different LFE scheme as ground for future work.

- $\mathsf{LFE.Setup}(1^\lambda, 1^\eta)$: The setup algorithm samples uniform matrices
$$\hat{\mathbf{A}}_1, \ldots, \hat{\mathbf{A}}_{\tilde{\eta}} \in \mathbb{Z}_q^{k \times m}$$
where $\tilde{\eta}$ denotes η times the size of an FHE ciphertext. It outputs $\mathsf{crs} = (\hat{\mathbf{A}}_0, \ldots, \hat{\mathbf{A}}_{\tilde{\eta}-1})$.

- $\mathsf{LFE.Hash}(\mathsf{crs}, f)$: On input a function f with η input bits, the hashing algorithm computes
$$\hat{\mathbf{A}}_f = \mathbf{G}^{-1}\left(\mathsf{EvalPk}(\mathsf{crs}, f)\right)$$
where the EvalPk is the standard lattice homomorphic evaluation described in [QWW18].

- $\mathsf{LFE.Encode}(\mathsf{crs}, \mathbf{s}, x)$: Parse $\mathbf{s}$ as a GSW secret key. For all $i \in [\eta]$, let
$$\mathbf{C}_i = \begin{bmatrix} \hat{\mathbf{B}}_i \\ \mathbf{s}^\intercal \hat{\mathbf{B}}_i + \mathbf{e}_i^\intercal \end{bmatrix} + x_i \mathbf{G}$$
be a GSW encryption of x_i, for uniformly sampled $\hat{\mathbf{B}}_i$. Let $\mathbf{c} = \mathsf{vec}(\mathbf{C}_0, \ldots, \mathbf{C}_{\eta-1})$, then, for all $i \in [\tilde{\eta}]$, set
$$\mathbf{b}_i = \mathbf{s}^\intercal(\hat{\mathbf{A}}_i - \mathbf{c}_i \cdot \bar{\mathbf{G}}) + \hat{\mathbf{e}}_i^\intercal$$
where $\hat{\mathbf{e}}_i \xleftarrow{\$} \mathcal{D}_\sigma^m$ and $\bar{\mathbf{G}}$ is the gadget matrix without the last row. Output the encoding $\mathsf{ec} = (\mathbf{c}, \mathbf{b}_0, \ldots, \mathbf{b}_{\tilde{\eta}-1})$.

- $\mathsf{LFE.Eval}(\mathsf{crs}, \mathsf{ec}, f)$: Run the lattice homomorphic evaluation algorithm EvalCt of the GSW evaluation of f, as described in [QWW18].

For correctness, we have that

$$\left\| \mathbf{s}^\intercal \mathbf{G} \hat{\mathbf{A}}_f + \frac{q}{p} f(x)^\intercal - \mathsf{LFE.Eval}(\mathsf{crs}, \mathsf{ec}, f) \right\|_\infty \leq \varphi(\lambda)^\delta \tag{16}$$

where φ is some fixed polynomial and δ is the depth of the circuit computing f. For security, we prove that the output of the encoding algorithm is pseudorandom, also in the presence of a decomposed LWE sample (see Section 2.3)

Lemma 10 (Pseudorandomness)**.** *If the decomposed LWE assumption holds, then for all $x \in \{0,1\}^\eta$ the following distributions are computationally indistinguishable*

$$\{\mathsf{crs}, \mathbf{Z}, \mathbf{s}^\intercal \mathbf{G} \mathbf{Z} + \mathbf{e}^\intercal, \mathsf{ec}\} \approx_c \{\mathsf{crs}, \mathbf{Z}, \mathbf{u}\}.$$

where $\mathsf{crs} \xleftarrow{\$} \mathsf{LFE.Setup}(1^\lambda, 1^\eta)$, $\mathbf{Z} \xleftarrow{\$} \mathsf{TreeSetup}(1^\lambda)$, $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^k$, $\mathbf{e} \xleftarrow{\$} \mathcal{D}_\sigma^{nt^2}$, $\mathsf{ec} \xleftarrow{\$} \mathsf{LFE.Encode}(\mathsf{crs}, \mathbf{s}, x)$ *and* $\mathbf{u} \xleftarrow{\$} \mathbb{Z}_q^{m\tilde{\eta}+m^2\eta+nt^2}$.

Proof. For uniformly sampled $\hat{\mathbf{A}}_i^*$ we define $\hat{\mathbf{A}}_i = \hat{\mathbf{A}}_i^* + \mathbf{c}_i \cdot \bar{\mathbf{G}}$, where $\mathbf{c}$ is defined as in the encoding algorithm, which does not change the view of the distinguisher. Then the claim reduces to distinguishing

$$\mathbf{s}^\intercal \begin{bmatrix} \mathbf{Z} & \hat{\mathbf{A}}_0^* & \dots & \hat{\mathbf{A}}_{\tilde{\eta}-1}^* & \hat{\mathbf{B}}_0 & \dots & \hat{\mathbf{B}}_{\eta-1} \end{bmatrix} + \mathbf{E}$$

from uniform. This is precisely the decomposed LWE problem (Definition 1) with auxiliary random matrix, which is as hard as the standard decomposed LWE [AMR25a]. □

4.2 Our Scheme

Let $f : \{0,1\}^d \times \{0,1\}^\eta \mapsto \mathbb{Z}_p^{mn^2}$ be a, without loss of generality, fixed shift function. We define the following additional parameters:

- The input length $\eta = \eta(\lambda)$.
- The rounding modulus $p = p(\lambda)$, which we set so that

$$\frac{q}{p} \geq (mnt\sigma\sqrt{\lambda})^{O(d\delta)} \cdot 2^{O(\delta)} \cdot d^{O(\delta)} \cdot 2^{\omega(\log \lambda)}$$

  where δ is the depth of the circuit computing the LFE hash of f, which is at most a polynomial (in λ) factor larger than the depth of the circuit computing f.

We describe our scheme in the following.

- $\mathsf{Setup}(1^\lambda, 1^\eta)$: Sample

$$\mathsf{pk} \xleftarrow{\$} \mathsf{TreeSetup}(1^\lambda) \quad \mathsf{crs} \xleftarrow{\$} \mathsf{LFE.Setup}(1^\lambda, 1^\eta) \quad \mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^k \quad \mathbf{e} \xleftarrow{\$} \mathcal{D}_\sigma^{nt^2} \quad \mathbf{t} \xleftarrow{\$} \mathbb{Z}_p^{mn^2}$$

  and define $\mathbf{z}^\intercal = \mathbf{s}^\intercal \mathbf{G}\mathbf{Z} + \mathbf{e}^\intercal$. For all $i \in [d]$, compute

$$\rho(0^d)^{(i)} = \mathsf{TreeInit}(\mathsf{pk}, d, \mathbf{M}, i, 0^d).$$

  Let g_{crs} be the function that takes as input some $\alpha \in \{0,1\}^d$ and outputs $\mathsf{vec}(\mathsf{LFE.Hash}(\mathsf{crs}, f_\alpha))$, where f_α denotes the function f with α hardwired as its first input. Compute

$$\left\{\tilde{\mathbf{x}}_j, \tilde{\mathbf{M}}_j, \tilde{\mathbf{c}}_j\right\}_{j\in[d+1]} = \tilde{\rho}(0^d) = \mathsf{TreeEval}(\mathsf{pk}, g_{\mathsf{crs}}, \mathbf{z}, \{\rho(0^d)^{(i)}\}_{i\in[d]}).$$

  Output the key $K = (\mathsf{pk}, \mathsf{crs}, \mathbf{z}, \tilde{\mathbf{M}}_0, \mathbf{s}, \mathbf{t})$.

- $\mathsf{Shift}(K, \beta)$: Compute

$$\mathsf{ec} \xleftarrow{\$} \mathsf{LFE.Encode}(\mathsf{crs}, \mathbf{s}, \beta)$$

  and output the shifted key $K_\beta = (\mathsf{pk}, \mathsf{crs}, \mathbf{z}, \mathsf{ec}, \mathbf{t})$.

- $\mathsf{Eval}(K, \alpha)$: Output

$$\left\lceil \mathbf{s}^\intercal \mathbf{G}\tilde{\mathbf{M}}_0 \cdot \left(\prod_{j\in[d]} (\mathbf{P}_{\alpha_j} \cdot \mathbf{V})\right) \cdot \mathbf{P} \cdot \mathbf{V}' + \mathbf{t}^\intercal \right\rceil_p$$

  where $\mathbf{V}'$ is defined as in Equation (6) but with twice the number of rows.

- $\mathsf{ShiftEval}(K_\beta, \alpha)$: Compute

$$\mathbf{c}^\intercal = \mathsf{LFE.Eval}(\mathsf{crs}, \mathsf{ec}, f_\alpha)$$

  and, for all $i \in [d]$, compute

$$\rho(\alpha)^{(i)} = \mathsf{TreeInit}(\mathsf{pk}, d, \mathbf{M}, i, \alpha).$$

  Derive $\tilde{\rho}(\alpha) = \mathsf{TreeEval}(\mathsf{pk}, g_{\mathsf{crs}}, \mathbf{z}, \{\rho(\alpha)^{(i)}\}_{i\in[d]})$, denoting by $\tilde{\mathbf{x}}_d$ its leaf, and $\tilde{\mathbf{c}}^\intercal = \mathsf{TreeSolve}(\mathsf{pk}, \mathbf{z}, \alpha, \tilde{\rho}(\alpha))$. Output

$$\left\lceil \begin{bmatrix} \tilde{\mathbf{c}}^\intercal & \mathbf{z}^\intercal \end{bmatrix} \cdot \begin{bmatrix} \mathbf{P} \\ \mathbf{Q}_{\tilde{\mathbf{x}}_d} \end{bmatrix} \cdot \mathbf{V}' + \mathbf{c}^\intercal + \mathbf{t}^\intercal \right\rceil_p$$

  where the rounding modulo p operation is extended to vectors component-wise.

We show that the scheme satisfies correctness, that is, the output of the shifted key equals the output of the evaluation algorithm, shifted by the value dictated by f.

Lemma 11 (Correctness)**.** *There exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and all $\beta \in \{0,1\}^\eta$ we have*

$$\Pr_K \left(\exists \alpha : \mathsf{ShiftEval}(K_\beta, \alpha) \neq \mathsf{Eval}(K, \alpha) + f(\alpha, \beta)\right) \leq \mu(\lambda).$$

Proof. By Equation (16) we have that

$$\mathbf{c}^\intercal = \mathbf{s}^\intercal \mathbf{G} \hat{\mathbf{A}}_{f_\alpha} + \frac{q}{p} f(\alpha, \beta)^\intercal + \hat{\mathbf{e}}^\intercal \tag{17}$$

where $\hat{\mathbf{A}}_{f_\alpha} = \mathsf{LFE.Hash}(\mathsf{crs}, f_\alpha)$. By Lemma 9, we have that $\tilde{\rho}(\alpha)$ is a valid encoding with leaf $\tilde{\mathbf{x}}_d = g_{\mathsf{crs}}(\alpha)$ and therefore

$$\begin{aligned}
\tilde{\mathbf{c}}^\intercal &= \mathsf{TreeSolve}(\mathsf{pk}, \mathbf{z}, \alpha, \tilde{\rho}(\alpha)) \\
&= \mathbf{s}^\intercal \mathbf{G} \left(\tilde{\mathbf{M}}_0 \prod_{j \in [d]} \left(\mathbf{P}_{\alpha_j} \mathbf{V}\right) - \mathsf{Hash}(\mathsf{pk}, g_{\mathsf{crs}}(\alpha))^\intercal \otimes \mathbf{I}_n \right) + \tilde{\mathbf{e}}^\intercal \\
&= \mathbf{s}^\intercal \mathbf{G} \left(\tilde{\mathbf{M}}_0 \prod_{j \in [d]} \left(\mathbf{P}_{\alpha_j} \mathbf{V}\right) - \mathsf{Hash}(\mathsf{pk}, \mathsf{vec}(\hat{\mathbf{A}}_{f_\alpha}))^\intercal \otimes \mathbf{I}_n \right) + \tilde{\mathbf{e}}^\intercal
\end{aligned}$$

by Lemma 8 and by definition of g_{crs}. Note that the matrix $\tilde{\mathbf{M}}_0$ is indeed the same as the one computed in the setup, since the function evaluated homomorphically is identical and the root is common to all paths. Applying the encoding expansion and appealing to Lemma 3, we obtain

$$\begin{bmatrix} \tilde{\mathbf{c}}^\intercal & \mathbf{z}^\intercal \end{bmatrix} \cdot \begin{bmatrix} \mathbf{P} \\ \mathbf{Q}_{\tilde{\mathbf{x}}_d} \end{bmatrix} = \mathbf{s}^\intercal \mathbf{G} \left(\tilde{\mathbf{M}}_0 \prod_{j \in [d]} \left(\mathbf{P}_{\alpha_j} \mathbf{V}\right) \cdot \mathbf{P} - \mathsf{vec}(\hat{\mathbf{A}}_{f_\alpha})^\intercal \otimes \mathbf{I}_n \right) + \tilde{\mathbf{e}}^\intercal \mathbf{P} + \mathbf{e}^\intercal \mathbf{Q}_{\tilde{\mathbf{x}}_d}. \tag{18}$$

Combining Equation (17) and Equation (18) we obtain

$$\begin{bmatrix} \tilde{\mathbf{c}}^\intercal & \mathbf{z}^\intercal \end{bmatrix} \cdot \begin{bmatrix} \mathbf{P} \\ \mathbf{Q}_{\tilde{\mathbf{x}}_d} \end{bmatrix} \cdot \mathbf{V}' + \mathbf{c}^\intercal = \mathbf{s}^\intercal \mathbf{G} \tilde{\mathbf{M}}_0 \prod_{j \in [d]} \left(\mathbf{P}_{\alpha_j} \mathbf{V}\right) \cdot \mathbf{P}\mathbf{V}' + \frac{q}{p} f(\alpha, \beta)^\intercal + \tilde{\mathbf{e}}^\intercal \mathbf{P}\mathbf{V}' + \mathbf{e}^\intercal \mathbf{Q}_{\tilde{\mathbf{x}}_d} \mathbf{V}' + \hat{\mathbf{e}}.$$

We can then bound

$$\begin{aligned}
\|\tilde{\mathbf{e}}^\intercal \mathbf{P}\mathbf{V}' + \mathbf{e}^\intercal \mathbf{Q}_{\tilde{\mathbf{x}}_d} \mathbf{V}' + \hat{\mathbf{e}}\|_\infty &\leq \|\tilde{\mathbf{e}}^\intercal \mathbf{P}\mathbf{V}'\|_\infty + \|\mathbf{e}^\intercal \mathbf{Q}_{\tilde{\mathbf{x}}_d} \mathbf{V}'\|_\infty + \|\hat{\mathbf{e}}\|_\infty \\
&\leq \|\tilde{\mathbf{e}}\|_\infty \cdot mn + \|\mathbf{e}\|_\infty \cdot t + \|\hat{\mathbf{e}}\|_\infty \\
&\leq (mnt\sigma\sqrt{\lambda})^{O(d\delta)} \cdot 2^{O(\delta)} \cdot d^{O(\delta)}
\end{aligned}$$

where the second inequality follows by Lemma 3 and the fact that $\|\tilde{\mathbf{x}}_d\|_\infty = \|\hat{\mathbf{A}}_{f_\alpha}\|_\infty \leq 1$ and the third inequality follows by the fact that the sum is dominated by the term

$$\|\tilde{\mathbf{e}}\|_\infty \leq (d+1) \cdot (mn)^d \cdot (mnt\sigma\sqrt{\lambda})^{O(d\delta)} \cdot 2^{O(\delta)} \cdot d^{O(\delta)} + d \cdot t\sigma\sqrt{\lambda}(mn)^d \cdot (mnt\sigma\sqrt{\lambda})^d$$

by Lemma 8 and Lemma 9.

Using a standard argument one can show that, for any fixed α, the probability that the noise changes the rounding is bounded by $\mu(\lambda) \cdot 2^{-d}$, for a negligible μ, over the random choice of $\mathbf{t}$. The desired statement follows by a union bound. □

We note that the construction is *laconic*, in the sense that the complexity of the Shift and Eval algorithms depends (polynomially) on the depth of the circuit computing f and on the size of its input, but it is otherwise independent on the circuit complexity of f. Furthermore the circuit depth of the Eval algorithm can be bounded by a fixed polynomial in the security parameter.

Finally, we show that our construction satisfies shift hiding. In fact we prove a slightly stronger property, which immediately implies the desired statement.

Lemma 12 (Shift Hiding)**.** *If the decomposed LWE assumption holds, then for all $\beta \in \{0,1\}^\eta$ the following distributions are computationally indistinguishable*

$$\left\{\mathsf{Shift}(K,\beta) : K \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^\eta)\right\} \approx_c \left\{U : U \xleftarrow{\$} \mathbb{Z}_q^{tn\times m} \times \mathcal{D}_\sigma^{m\times tn} \times \mathbb{Z}_q^{k\times m} \times \mathbb{Z}_q^{m\tilde{\eta}+m^2\eta+nt^2} \times \mathbb{Z}_p^{mn^2}\right\}.$$

Proof. Follows as an immediate consequence of Lemma 10 and by definition of each component of the shifted key. □

4.3 Intractable Laconic Shift-Hiding PRF

Some of our results will later need a laconic shift-hiding PRF where it is hard for the adversary to find an input α such that, for every entry i, we have

$$\left|\left(\mathsf{Eval}(K,\alpha)_i - \frac{p}{2}\right) \bmod p\right| < B$$

where $\mathsf{Eval}(K,\alpha)_i$ denotes the i-th entry of $\mathsf{Eval}(K,\alpha)$. If a construction satisfies this property, we call it B-intractable. We show how to construct it using any laconic shift-hiding PRF and a correlation intractable hash function (CI) for searchable relations over $\mathbb{Z}_{p/(2B)}$. Recall that a hash function is *correlation intractable* [CGH98] for a function f if, given the hash key hk it is computationally hard to find an x such that

$$\mathsf{Hash}(\mathsf{hk}, x) = f(x). \tag{19}$$

It is known [PS19] that CI-hash for all efficiently computable (sparse) functions can be constructed from LWE.

We can achieve our goal by composing the PRF with a CI hash. In more details, we augment the shifted key with keys for a CI hash $\{\mathsf{hk}_i\}_i$, one for each entry of the output. Then, when we evaluate the (shifted or non-shifted) PRF on a value α, we add $2B \cdot \mathsf{Hash}(\mathsf{hk}_i, \alpha)$ to the i-th entry of the result (for every i), and we reduce modulo p. It is immediate to see that the construction is still correct and shift-hiding. All we need to show is the intractability property.

To see it, suppose for simplicity that $2B$ divides p, and generate hk_i to be intractable for the relation defined by the function

$$f(\alpha) = \left\lceil \left(\left(\frac{p}{2} - \mathsf{Eval}(K, \alpha)_i \right) \bmod p \right) \cdot (2B)^{-1} \right\rfloor \bmod \frac{p}{2B},$$

where $\lceil \cdot \rfloor$ denotes rounding. If $p \geq 2B \cdot 2^{\omega(\log \lambda)}$ then this function defines a sparse relation. Now, if the adversary manages to find α such that

$$\left| \left(\mathsf{Eval}(K, \alpha)_i + 2B \cdot \mathsf{Hash}(\mathsf{hk}_i, \alpha) - \frac{p}{2} \right) \bmod p \right| < B,$$

it must be that $p/2 - \mathsf{Eval}(K, \alpha)_i$ and $2B \cdot \mathsf{Hash}(\mathsf{hk}_i, \alpha)$ are within distance strictly smaller than B. Therefore, if we divide both sides by $2B$, we round and reduce modulo $\frac{p}{2B}$, we obtain the same value. However,

$$\left\lceil \frac{p/2 - \mathsf{Eval}(K, \alpha)_i}{2B} \right\rfloor \bmod \frac{p}{2B} = \left\lceil \frac{2B \cdot \mathsf{Hash}(\mathsf{hk}_i, \alpha)}{2B} \right\rfloor \bmod \frac{p}{2B} = \mathsf{Hash}(\mathsf{hk}_i, \alpha).$$

This satisfies Equation (19) and contradicts the correlation-intractability of hk_i.

4.4 A Heuristic Approach to Obfuscation

We discuss a simple approach to construct an obfuscator for all circuit, as a simple application of our construction in Section 4.2. To obfuscate a circuit C, we define the constraint β_K and the function f to be such that

$$f(\alpha, \beta_K) = C(\alpha) - \mathsf{Eval}(K, \alpha)$$

and we let the obfuscation be the key shifted at β_K. One can evaluate the obfuscated circuit by running

$$\mathsf{ShiftEval}(K_{\beta_K}, \alpha) = \mathsf{Eval}(K, \alpha) + C(\alpha) - \mathsf{Eval}(K, \alpha) = C(\alpha)$$

by Lemma 11. We note that this construction is only well-defined because of the laconic property of the scheme, since the complexity of the Eval algorithm only depends on the depth of the constrained function and moreover the depth of Eval is bounded by a polynomial in λ (regardless of the constraint).

Unfortunately, the shift hiding property of our laconic shift-hiding PRF (Lemma 12) holds for any fixed constraint β, but the proof strategy fails if β is allowed to depend on K. Nevertheless, we view this circularity as a reasonable heuristic for security, and we conjecture that the obfuscator described above is secure. We leave the analysis of such construction as a fascinating open problem.

5 Compressing LWE Samples

In the following we show how to give *compressed* LWE samples with respect to a particular class of matrices, which later on can be expanded to fully-fledged LWE samples. Before

presenting our construction, we define the *block-transpose* operator Π_k. Let $\mathbf{M} \in \mathbb{Z}_q^{k \cdot n_0 \times n_1}$ be a matrix, and write $\mathbf{M}$ as a block matrix of k-dimensions vectors $\mathbf{m}_{i,j} \in \mathbb{Z}_q^k$:

$$\mathbf{M} = \{\mathbf{m}_{i,j}\}_{i,j} = \begin{bmatrix} \mathbf{m}_{0,0} & \cdots & \mathbf{m}_{0,n_1-1} \\ \vdots & \ddots & \vdots \\ \mathbf{m}_{n_0-1,0} & \cdots & \mathbf{m}_{n_0-1,n_1-1} \end{bmatrix}.$$

We define the action of the operator Π_k as

$$\Pi_k(\mathbf{M}) = \{\mathbf{m}_{j,i}\}_{i,j} = \begin{bmatrix} \mathbf{m}_{0,0} & \cdots & \mathbf{m}_{n_0,0} \\ \vdots & \ddots & \vdots \\ \mathbf{m}_{0,n_1-1} & \cdots & \mathbf{m}_{n_1-1,n_0-1} \end{bmatrix} \in \mathbb{Z}_q^{k \cdot n_1 \times n_0}.$$

In other words, the output of the operator is a matrix where the blocks are transposed, where each block is a k-dimensional vector. It can be verified by direct calculation that Π_k is linear and that, for all vectors $\mathbf{s} \in \mathbb{Z}_q^k$ and matrices $\mathbf{M} \in \mathbb{Z}_q^{k \cdot n_0 \times n_1}$, it holds that

$$\Pi_k(\Pi_k(\mathbf{M})) = \mathbf{M} \qquad \text{and} \qquad (\mathbf{I}_{n_1} \otimes \mathbf{s}^\intercal) \cdot \Pi_k(\mathbf{M}) = ((\mathbf{I}_{n_0} \otimes \mathbf{s}^\intercal) \cdot \mathbf{M})^\intercal. \tag{20}$$

We also introduce the following two lemmas, which allow us to construct matrix trapdoors by appropriately combining matrices.

Lemma 13 (Matrix Trapdoors)**.** *Consider any matrices* $\mathbf{A} \in \mathbb{Z}_q^{t \cdot k \times m}$ *and* $\mathbf{B} \in \mathbb{Z}_q^{m \times t/n}$*. Let*

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_0 \\ \vdots \\ \mathbf{A}_{t-1} \end{bmatrix}$$

be the blocks of $\mathbf{A}$ *and define*

$$\mathbf{U} = \Pi_k(\mathbf{AB}) + \mathbf{I}_{t/n} \otimes \mathbf{G}, \qquad \mathbf{L} = -\mathbf{G}^{-1} \begin{bmatrix} \mathsf{vec}(\mathbf{A}_0) & \cdots & \mathsf{vec}(\mathbf{A}_{t-1}) \end{bmatrix}.$$

Then, it holds that $\|\mathbf{L}\|_\infty \leq 1$ *and*

$$\begin{bmatrix} \mathbf{U} & \mathbf{B}^\intercal \otimes \mathbf{G} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_t \\ \mathbf{L} \end{bmatrix} = \begin{bmatrix} \mathbf{U} & \mathbf{B}^\intercal \otimes \mathbf{I}_k \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_t \\ (\mathbf{I}_m \otimes \mathbf{G})\mathbf{L} \end{bmatrix} = \mathbf{I}_{t/n} \otimes \mathbf{G}, \tag{21}$$

Proof. The first identity in Equation (21) follows by Equation (3) since

$$(\mathbf{B}^\intercal \otimes \mathbf{G})\mathbf{L} = (\mathbf{B}^\intercal \otimes \mathbf{I}_k) \cdot (\mathbf{I}_m \otimes \mathbf{G})\mathbf{L}.$$

For the second identity, observe that

$$(\mathbf{I}_m \otimes \mathbf{G})\mathbf{L} = -\begin{bmatrix} \mathsf{vec}(\mathbf{A}_0) & \cdots & \mathsf{vec}(\mathbf{A}_{t-1}) \end{bmatrix}.$$

Moreover, by Equation (4)

$$(\mathbf{B}^\intercal \otimes \mathbf{I}_k) \cdot \mathsf{vec}(\mathbf{A}_i) = \mathsf{vec}(\mathbf{A}_i\mathbf{B}).$$

Finally, by definition of the block-transpose operator

$$\Pi_k(\mathbf{AB}) = \Pi_k \begin{bmatrix} \mathbf{A}_0\mathbf{B} \\ \vdots \\ \mathbf{A}_{t-1}\mathbf{B} \end{bmatrix} = \begin{bmatrix} \mathsf{vec}(\mathbf{A}_0\mathbf{B}) & \cdots & \mathsf{vec}(\mathbf{A}_{t-1}\mathbf{B}) \end{bmatrix}.$$

Putting all these observations together, we derive Equation (21). □

The following is a recursive version of the lemma above.

Lemma 14 (Recursive Matrix Trapdoor)**.** *Consider any matrices* $\mathbf{A} \in \mathbb{Z}_q^{t\cdot k\times m}$ *and* $\mathbf{B} \in \mathbb{Z}_q^{m\times t/n}$ *and define* $\mathbf{U}$ *and* $\mathbf{L}$ *as in Lemma 13. Set* $c = t/nm$ *and*

$$\mathbf{U}_0 = \mathbf{U}, \qquad \mathbf{B}_0 = \mathbf{B}, \qquad \mathbf{L}_0 = \mathbf{L}.$$

Then, inductively define $\mathbf{U}_i \in \mathbb{Z}_q^{m\cdot c^{i+1}\cdot k\,\times\, t\cdot c^i}, \mathbf{B}_i \in \mathbb{Z}_q^{m\,\times\, m\cdot c^{i+1}}, \mathbf{L}_i \in \mathbb{Z}_q^{n\cdot m\,\times\, n\cdot m\cdot c^{i+1}}$ *as*

$$\begin{aligned}
\mathbf{U}_{i+1} &= \mathbf{I}_c \otimes \mathbf{U}_i + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal \otimes \mathbf{I}_k) \cdot \mathbf{U} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \\
\mathbf{B}_{i+1} &= \mathbf{B} \cdot (\mathbf{I}_c \otimes \mathbf{B}_i) \\
\mathbf{L}_{i+1} &= \mathbf{L} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i).
\end{aligned}$$

For every $i \in \mathbb{N}$*, it holds that* $\|\mathbf{L}_i\|_\infty \le (nm)^i$*,* $\|\mathbf{B}_i\|_\infty \le \|\mathbf{B}\|_\infty (m\|\mathbf{B}\|_\infty)^i$*, and*

$$\begin{bmatrix} \mathbf{U}_i & \mathbf{B}_i^\intercal \otimes \mathbf{G} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_{t\cdot c^i} \\ \mathbf{L}_i \end{bmatrix} = \begin{bmatrix} \mathbf{U}_i & \mathbf{B}_i^\intercal \otimes \mathbf{I}_k \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_{t\cdot c^i} \\ (\mathbf{I}_m \otimes \mathbf{G})\mathbf{L}_i \end{bmatrix} = \mathbf{I}_{m\cdot c^{i+1}} \otimes \mathbf{G} \tag{22}$$

Proof. The norm bounds follow by a straightforward induction and the first identity in Equation (22) holds by the same reasoning as in Lemma 13. To prove the second identity of Equation (22) we proceed with an induction argument over i. The base case ($i = 0$) is precisely proven in Lemma 13. As for the inductive step, observe that:

$$\begin{aligned}
&\begin{bmatrix} \mathbf{U}_{i+1} & \mathbf{B}_{i+1}^\intercal \otimes \mathbf{G} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_{t\cdot c^{i+1}} \\ \mathbf{L}_{i+1} \end{bmatrix} \\
&= \mathbf{U}_{i+1} + (\mathbf{B}_{i+1}^\intercal \otimes \mathbf{G}) \cdot \mathbf{L}_{i+1} \\
&= \mathbf{U}_{i+1} + \Big(((\mathbf{I}_c \otimes \mathbf{B}_i^\intercal) \cdot \mathbf{B}^\intercal) \otimes \mathbf{G} \Big) \cdot \mathbf{L} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \\
&= \mathbf{U}_{i+1} + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal \otimes \mathbf{I}_k) \cdot (\mathbf{B}^\intercal \otimes \mathbf{G}) \cdot \mathbf{L} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \\
&= \mathbf{I}_c \otimes \mathbf{U}_i + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal \otimes \mathbf{I}_k) \cdot \mathbf{U} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal \otimes \mathbf{I}_k) \cdot (\mathbf{B}^\intercal \otimes \mathbf{G}) \cdot \mathbf{L} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \\
&= \mathbf{I}_c \otimes \mathbf{U}_i + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal \otimes \mathbf{I}_k) \cdot \Big(\mathbf{U} + (\mathbf{B}^\intercal \otimes \mathbf{G}) \cdot \mathbf{L} \Big) \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \\
&= \mathbf{I}_c \otimes \mathbf{U}_i + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal \otimes \mathbf{I}_k) \cdot (\mathbf{I}_{m\cdot c} \otimes \mathbf{G}) \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \\
&= \mathbf{I}_c \otimes \Big(\mathbf{U}_i + (\mathbf{B}_i^\intercal \otimes \mathbf{I}_k) \cdot (\mathbf{I}_m \otimes \mathbf{G}) \cdot \mathbf{L}_i \Big) \\
&= \mathbf{I}_c \otimes \mathbf{I}_{m\cdot c^{i+1}} \otimes \mathbf{G}
\end{aligned}$$

where the last identity follows by induction hypothesis. This concludes our proof. □

5.1 The Compression Algorithm

We describe the scheme along with its correctness proof. Later on, we will show how to sample the inputs to the algorithms so that we can prove its security. We introduce new parameters:

- $s = s(\lambda)$ be the depth of the recursion.
- $\sigma' = \sigma'(\lambda)$ be another Gaussian parameter.
- $c > 1$ a constant.

We describe the algorithms of our scheme in the following.

- $\mathsf{Setup}(1^\lambda)$: Sample $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{tk \times m}$ and $\tilde{\mathbf{B}} \xleftarrow{\$} \mathcal{D}_\sigma^{m \times tn}$. Define $\mathbf{B} = \tilde{\mathbf{B}}\mathbf{G}^{-1}(\Pi_k(\mathbf{I}_{t/n} \otimes \mathbf{G})))$. Output $\mathsf{hk} = (\mathbf{A}, \mathbf{B})$.
- $\mathsf{Hash}(\mathsf{hk}, \mathbf{X}, \mathbf{w})$: Suppose that $\mathbf{X}$ has k rows and $m \cdot c^s$ columns. Set $\mathbf{x} = \mathsf{vec}(\mathbf{X})$ and compute

$$\begin{bmatrix} \mathbf{w}_0 \\ \mathbf{w}_1 \end{bmatrix} = \mathsf{ShiftedPreimage}\left(\begin{bmatrix} \mathbf{U}_{s-1} & \mathbf{B}_{s-1}^\intercal \otimes \mathbf{G} \end{bmatrix}, \begin{bmatrix} \mathbf{I}_{t \cdot c^{s-1}} \\ \mathbf{L}_{s-1} \end{bmatrix}, \mathbf{x}, \mathbf{w} \right)$$

  with variables defined as in Lemma 14. Output the hash $\mathbf{d} = (\mathbf{I}_m \otimes \mathbf{G}) \cdot \mathbf{w}_1$ and the expander key $\mathsf{ek} = \mathbf{w}_0$.
- $\mathsf{Gen}(\mathsf{hk}, \mathbf{d}, \mathbf{s})$: Output

$$\mathbf{C} = (\mathbf{I}_{t/n} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U} + \mathbf{E}, \qquad \mathbf{h} = (\mathbf{I}_m \otimes \mathbf{s}^\intercal) \cdot \mathbf{d} + \mathbf{e},$$

  where $\mathbf{E} \xleftarrow{\$} \mathcal{D}_\sigma^{t/n \times t}$ and $\mathbf{e} \xleftarrow{\$} \mathcal{D}_{\sigma'}^m$.
- $\mathsf{Expand}(\mathsf{hk}, \mathsf{ek}, \mathbf{C}, \mathbf{h})$: Set $\mathbf{C}_0 = \mathbf{C}$. Then, inductively compute

$$\mathbf{C}_{i+1} = \mathbf{I}_c \otimes \mathbf{C}_i + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal) \cdot \mathbf{C} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i).$$

  Output $\mathbf{v} = \mathbf{C}_{s-1} \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot \mathbf{h}$.

Before proving the correctness of expansion, we show the following useful technical lemma.

Claim 5 (Noise Bound)**.** *For every $i \in [s]$, with overwhelming probability, it holds that*

$$\|\mathbf{C}_i - (\mathbf{I}_{m \cdot c^{i+1}} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U}_i\|_\infty \leq 2(tmn\sigma\sqrt{\lambda})^{2(i+1)}$$

Proof. We proceed by induction over i. It is easy to see that the base case $i = 0$ holds. For the inductive step, define $\mathbf{E}_i = \mathbf{C}_i - (\mathbf{I}_{m \cdot c^{i+1}} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U}_i$. We observe that

$$\begin{aligned}
&\mathbf{C}_{i+1} - \mathbf{I}_c \otimes \mathbf{E}_i - (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal) \cdot \mathbf{E} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \\
&= \mathbf{I}_c \otimes \mathbf{C}_i + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal) \cdot \mathbf{C} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) - \mathbf{I}_c \otimes \mathbf{E}_i - (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal) \cdot \mathbf{E} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \\
&= \mathbf{I}_c \otimes (\mathbf{C}_i - \mathbf{E}_i) + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal) \cdot (\mathbf{C} - \mathbf{E}) \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \\
&= \mathbf{I}_c \otimes \Big((\mathbf{I}_{m \cdot c^{i+1}} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U}_i \Big) + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal) \cdot \Big((\mathbf{I}_{m \cdot c} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U} \Big) \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \\
&= (\mathbf{I}_{m \cdot c^{i+2}} \otimes \mathbf{s}^\intercal) \cdot (\mathbf{I}_c \otimes \mathbf{U}_i) + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal) \cdot (\mathbf{I}_{m \cdot c} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \\
&= (\mathbf{I}_{m \cdot c^{i+2}} \otimes \mathbf{s}^\intercal) \cdot (\mathbf{I}_c \otimes \mathbf{U}_i) + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal \otimes \mathbf{s}^\intercal) \cdot \mathbf{U} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \\
&= (\mathbf{I}_{m \cdot c^{i+2}} \otimes \mathbf{s}^\intercal) \cdot (\mathbf{I}_c \otimes \mathbf{U}_i) + (\mathbf{I}_{m \cdot c^{i+2}} \otimes \mathbf{s}^\intercal) \cdot (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal \otimes \mathbf{I}_k) \cdot \mathbf{U} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \\
&= (\mathbf{I}_{m \cdot c^{i+2}} \otimes \mathbf{s}^\intercal) \cdot \Big(\mathbf{I}_c \otimes \mathbf{U}_i + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal \otimes \mathbf{I}_k) \cdot \mathbf{U} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i) \Big) \\
&= (\mathbf{I}_{m \cdot c^{i+2}} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U}_{i+1},
\end{aligned}$$

so

$$\mathbf{E}_{i+1} = \mathbf{I}_c \otimes \mathbf{E}_i + (\mathbf{I}_c \otimes \mathbf{B}_i^\intercal) \cdot \mathbf{E} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i).$$

With overwhelming probability, we have

$$\|(\mathbf{I}_c \otimes \mathbf{B}_i^\intercal) \cdot \mathbf{E} \cdot (\mathbf{I}_c \otimes \mathbf{L}_i)\|_\infty \leq m^2 n \sigma \sqrt{\lambda} \cdot \|\mathbf{B}_i\|_\infty \cdot \|\mathbf{L}_i\|_\infty \leq (tmn\sigma\sqrt{\lambda})^{2(i+2)}$$
$$\|\mathbf{I}_c \otimes \mathbf{E}_i\|_\infty \leq 2(tmn\sigma\sqrt{\lambda})^{2(i+1)}.$$

Putting together these two, we obtain the claim. □

Next, we show that the output of the expansion algorithm is indeed a valid LWE sample with respect to the matrix $\mathbf{X}$.

Lemma 15 (Correctness of Expansion)**.** *For all matrices* $\mathbf{X} \in \mathbb{Z}_q^{k \times mc^s}$ *it holds that*

$$\|\mathbf{s}^\intercal \mathbf{X} - \mathsf{Expand}(\mathsf{hk}, \mathsf{ek}, \mathbf{C}, \mathbf{h})^\intercal\|_\infty \leq 2tc^{s-1} \cdot (tmn\sigma\sqrt{\lambda})^{2s} \cdot \|\mathsf{ek}\|_\infty + m(\sigma\sqrt{\lambda}tnm)^s \cdot \|\mathbf{e}\|_\infty$$

with overwhelming probability, where $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$*,* $(\mathbf{d}, \mathsf{ek}) = \mathsf{Hash}(\mathsf{hk}, \mathbf{X}, \mathbf{w}')$*,* $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^k$*,* $\mathbf{E} \xleftarrow{\$} \mathcal{D}_\sigma^{t/n \times t}$*,* $\mathbf{C} = (\mathbf{I}_{t/n} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U} + \mathbf{E}$*, and* $\mathbf{h} = (\mathbf{I}_m \otimes \mathbf{s}^\intercal) \cdot \mathbf{d} + \mathbf{e}$*. The probability is taken over the randomness of* Setup *and* $\mathbf{E}$ *only.*

Proof. Let $\mathbf{E}_i = \mathbf{C}_i - (\mathbf{I}_{m\cdot c^{i+1}} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U}_i$. We expand

$$\begin{aligned}
\mathbf{v} &= \mathbf{C}_{s-1} \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot \mathbf{h} \\
&= \Big((\mathbf{I}_{m\cdot c^s} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U}_{s-1} + \mathbf{E}_{s-1} \Big) \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot \Big((\mathbf{I}_m \otimes \mathbf{s}^\intercal) \cdot \mathbf{d} + \mathbf{e} \Big) \\
&= (\mathbf{I}_{m\cdot c^s} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U}_{s-1} \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot (\mathbf{I}_m \otimes \mathbf{s}^\intercal) \cdot \mathbf{d} + \mathbf{E}_{s-1} \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot \mathbf{e} \\
&= (\mathbf{I}_{m\cdot c^s} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U}_{s-1} \cdot \mathbf{w}_0 + (\mathbf{B}_{s-1}^\intercal \otimes \mathbf{s}^\intercal) \cdot \mathbf{d} + \mathbf{E}_{s-1} \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot \mathbf{e} \\
&= (\mathbf{I}_{m\cdot c^s} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U}_{s-1} \cdot \mathbf{w}_0 + (\mathbf{B}_{s-1}^\intercal \otimes \mathbf{s}^\intercal) \cdot (\mathbf{I}_m \otimes \mathbf{G}) \cdot \mathbf{w}_1 + \mathbf{E}_{s-1} \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot \mathbf{e} \\
&= (\mathbf{I}_{m\cdot c^s} \otimes \mathbf{s}^\intercal) \cdot \mathbf{U}_{s-1} \cdot \mathbf{w}_0 + (\mathbf{I}_{m\cdot c^s} \otimes \mathbf{s}^\intercal) \cdot (\mathbf{B}_{s-1}^\intercal \otimes \mathbf{G}) \cdot \mathbf{w}_1 + \mathbf{E}_{s-1} \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot \mathbf{e} \\
&= (\mathbf{I}_{m\cdot c^s} \otimes \mathbf{s}^\intercal) \cdot \Big(\mathbf{U}_{s-1} \cdot \mathbf{w}_0 + (\mathbf{B}_{s-1}^\intercal \otimes \mathbf{G}) \cdot \mathbf{w}_1 \Big) + \mathbf{E}_{s-1} \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot \mathbf{e} \\
&= (\mathbf{I}_{m\cdot c^s} \otimes \mathbf{s}^\intercal) \cdot \mathbf{x} + \mathbf{E}_{s-1} \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot \mathbf{e} \\
&= (\mathbf{I}_{m\cdot c^s} \otimes \mathbf{s}^\intercal) \cdot \mathsf{vec}(\mathbf{X}) + \mathbf{E}_{s-1} \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot \mathbf{e} \\
&= \mathsf{vec}(\mathbf{s}^\intercal \mathbf{X}) + \mathbf{E}_{s-1} \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot \mathbf{e} \\
&= \mathbf{X}^\intercal \mathbf{s} + \mathbf{E}_{s-1} \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot \mathbf{e}.
\end{aligned}$$

By Claim 5, we have

$$\|\mathbf{E}_{s-1} \cdot \mathbf{w}_0 + \mathbf{B}_{s-1}^\intercal \cdot \mathbf{e}\|_\infty \leq 2tc^{s-1} \cdot (tmn\sigma\sqrt{\lambda})^{2s} \cdot \|\mathbf{w}_0\|_\infty + m(\sigma\sqrt{\lambda}tnm)^s \cdot \|\mathbf{e}\|_\infty$$

concluding our proof. □

5.2 Derandomized Indistinguishability

We state and prove the security notion that is satisfied by our construction. In order to do this, we first have to introduce some additional notation. We rely on three separate instances of the laconic shift-hiding PRF (Section 4): $\mathsf{SH}, \mathsf{SH}_1, \mathsf{SH}_2$. We define such functions in the following:

- (SH) Defined with respect to a public function

$$\begin{aligned}
f(\alpha, \beta) &= f((\mathbf{Y}, \mathsf{aux}), (\tilde{K}_1, K_2),) \\
&= -(\mathbf{B}_{s-1}^\intercal \otimes \mathbf{G}) \cdot \mathsf{SH}_1.\mathsf{ShiftEval}(\tilde{K}_1, (\mathbf{Y}, \mathsf{aux})) + \mathbf{Y} \cdot \mathsf{SH}_2.\mathsf{Eval}(K_2, (\mathbf{Y}, \mathsf{aux})) \in \mathbb{Z}_q^{kmc^s}
\end{aligned}$$

  where $c > 1$.

- (SH_1) Defined with respect to a public function

$$\begin{aligned}
f_1(\alpha, \beta) &= f_1((\mathbf{Y}, \mathsf{aux}), (K, \mathbf{M}_0, \mathbf{M}_1)) \\
&= \mathbf{L}_{s-1} \cdot \mathbf{G}^{-1}(\mathsf{SH}.\mathsf{Eval}(K, (\mathbf{Y}, \mathsf{aux}))) - \mathbf{G}^{-1}(\mathsf{vec}(\mathbf{M}_0 \mathbf{K}_{\mathbf{M}_0, \mathbf{Y}} \mathbf{V} + \mathbf{M}_1)) \in \mathbb{Z}_p^{mn}
\end{aligned}$$

  where $\mathbf{K}_{\mathbf{M}_0, \mathbf{Y}} = \mathsf{BGG}.\mathsf{EvalM}(\mathbf{M}_0, \mathsf{SH}_1.\mathsf{Eval}(\cdot, (\mathbf{Y}, \mathsf{aux})))$ as defined in [BGG+14]. Moreover, we assume that SH_1 is $(mnt^s + 1)$-intractable.

- (SH_2) Defined with respect to a public function

$$\begin{aligned} f_2(\alpha, \beta) &= f_2((\mathbf{Y}, \mathsf{aux}), (\tilde{K}, \tau)) \\ &= \mathsf{Extract}(\tau, \mathbf{Y}, \mathsf{aux}) \cdot \mathbf{G}^{-1}(\mathsf{SH.ShiftEval}(\tilde{K}, (\mathbf{Y}, \mathsf{aux}))) \in \mathbb{Z}_p^L. \end{aligned}$$

  Above L denote the number of columns of the matrix $\mathbf{Y}$. We further assume that SH_2 is mnc^s-intractable.

Observe that despite the circular dependency of the three schemes SH, SH_1 and SH_2 are still well defined by their laconic properties. For the parameters, we require that

$$p = t^s \cdot 2^{\omega(\log \lambda)} \qquad t = cnm \qquad \sigma = 2^{\omega(\log \lambda)} \qquad \sigma' = \sigma \cdot 2^{\delta_1 \cdot \omega(\log \lambda)}$$

where $c > 1$ and δ_1 is the circuit depth of $\mathsf{SH}_1.\mathsf{Eval}$. We are now ready to formulate the security definition for our compressed sampler. We mention that here we prove a slightly more general definition than what is actually needed for our application, specifically we include an auxiliary input to the PRF, which we imagine could be a useful generalization for future applications of this theorem.

Definition 2 (Derandomized Indistinguishability)**.** *Consider the following experiment between a PPT attacker and a challenger.*

- *(Preliminary Phase) The attacker sends τ to the challenger.*
- *(Setup Phase) The challenger samples*

$$\mathbf{D} \xleftarrow{\$} \mathbb{Z}_q^{k \times m} \quad \mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda) \quad K_1 \xleftarrow{\$} \mathsf{SH}_1.\mathsf{Setup}(1^\lambda, 1^\eta) \quad K_2 \xleftarrow{\$} \mathsf{SH}_2.\mathsf{Setup}(1^\lambda, 1^\eta)$$

  and set $\tilde{K}_1 \xleftarrow{\$} \mathsf{SH}_1.\mathsf{Shift}(K_1, 0)$ $\tilde{K}_2 \xleftarrow{\$} \mathsf{SH}_2.\mathsf{Shift}(K_2, 0)$. Send $(\mathbf{D}, \mathsf{hk}, \tilde{K}_1, \tilde{K}_2)$ to the attacker.

- *(Admissibility) The attacker returns a pair $(\mathbf{Y}, \mathsf{aux})$ and the challenger executes $\mathbf{T} = \mathsf{Extract}(\tau, \mathbf{Y}, \mathsf{aux})$ and checks if*

$$\|\mathbf{T}\|_\infty \leq 1 \qquad \textit{and} \qquad \mathbf{Y} \cdot \mathbf{T} = \mathbf{I}_{mc^s} \otimes \mathbf{G}.$$

  The challenger aborts if any of these checks fail.

- *(Challenge Phase) The challenger samples a random bit $b \xleftarrow{\$} \{0, 1\}$ and proceeds as follows.*
  - *If $b = 1$: Sample $\mathbf{h} \xleftarrow{\$} \mathbb{Z}_q^m$, $\mathbf{C} \xleftarrow{\$} \mathbb{Z}_q^{t/n \times t}$, and $\mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^m$.*

– *If $b = 0$: Compute*

$$\mathbf{r}_1 = \mathsf{SH}_1.\mathsf{ShiftEval}(\tilde{K}_1, (\mathbf{Y}, \mathsf{aux}))$$
$$\mathbf{u} = \mathsf{SH}_2.\mathsf{ShiftEval}(\tilde{K}_2, (\mathbf{Y}, \mathsf{aux}))$$
$$\mathbf{d} = \mathsf{Hash}\left(\mathsf{hk}, \mathbf{X}, \begin{bmatrix} \mathbf{0}^{t \cdot c^{s-1}} \\ \mathbf{r}_1 \end{bmatrix}\right)$$

where $\mathbf{X} \in \mathbb{Z}_q^{k \times mc^s}$ is such that $\mathsf{vec}(\mathbf{X}) = \mathbf{Yu}$. Sample $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^k$ and $\mathbf{e} \xleftarrow{\$} \mathcal{D}_\sigma^m$ then set $(\mathbf{C}, \mathbf{h}) \xleftarrow{\$} \mathsf{Gen}(\mathsf{hk}, \mathbf{d}, \mathbf{s})$ and $\mathbf{a} = \mathbf{s}^\intercal \mathbf{D} + \mathbf{e}^\intercal$.

- *(Answer Phase) The attacker is given $\mathbf{C}$ and $\mathbf{h}$ and returns a bit b'. The attacker wins if $b' = b$.*

We say that a compressed sampler satisfies derandomized indistinguishability if there exists a PPT algorithm Extract *such that for all PPT attackers the probability that they win in the above experiment is negligibly close to $\frac{1}{2}$.*

Observe that with our choice of derandomization, we have that $\|\mathsf{ek}\|_\infty \leq 1$. We are now ready to state the security of our construction.

Theorem 5 (Derandomized Indistinguishability)**.** *Assuming the hardness of the decomposed LWE problem, the construction as defined above satisfies derandomized indistinguishability.*

Proof. To prove the result, we proceed by means of a sequence of computationally indistinguishable hybrids.

Hybrid 0: This correspond to the game in which $b = 0$.

Hybrid 1: In this hybrid, we change the distribution of $\tilde{K}_2$. We sample $K \xleftarrow{\$} \mathsf{SH.Setup}(1^\lambda, 1^\eta)$ and we set $\tilde{K} \xleftarrow{\$} \mathsf{SH.Shift}(K, 0)$. Then, we set $\tilde{K}_2 \xleftarrow{\$} \mathsf{SH}_2.\mathsf{Shift}(K_2, (\tilde{K}, \tau))$. Observe that this hybrid is indistinguishable from Hybrid 0 by the shift hiding property of SH_2.

Hybrid 2: In this hybrid, we change the distribution of $\tilde{K}$. We set $\tilde{K} \xleftarrow{\$} \mathsf{SH.Shift}(K, (\tilde{K}_1, K_2))$. Observe that this hybrid is indistinguishable from Hybrid 1 by the shift-hiding property of SH.

Hybrid 3: In this hybrid, we change the distribution of $\tilde{K}_1$. We sample random matrices $\mathbf{M}_0 \xleftarrow{\$} \mathbb{Z}_q^{k \times n\ell_1}$ and $\mathbf{M}_1 \xleftarrow{\$} \mathbb{Z}_q^{k \times m}$, where ℓ_1 is the bit-length of K_1. Then, we set $\tilde{K}_1 \xleftarrow{\$} \mathsf{SH}_1.\mathsf{Shift}(K_1, (K, \mathbf{M}_0, \mathbf{M}_1))$. Observe that this hybrid is indistinguishable from Hybrid 2 by the shift-hiding property of SH_1.

Hybrid 4: In this hybrid, we change the way we compute $\mathbf{d}$. Specifically, we set it to

$$\mathbf{d} = (\mathbf{I}_m \otimes \mathbf{G}) \cdot \mathsf{SH}_1.\mathsf{Eval}(K_1, (\mathbf{Y}, \mathsf{aux})) - \mathsf{vec}(\mathbf{M}_0 \cdot \mathbf{K}_{\mathbf{M}_0, \mathbf{Y}} \cdot \mathbf{V} + \mathbf{M}_1).$$

Recall that, in Hybrid 3, $\mathbf{d}$ is defined as

$$\mathbf{d} = (\mathbf{I}_m \otimes \mathbf{G}) \cdot \mathbf{w}_1 = (\mathbf{I}_m \otimes \mathbf{G}) \cdot (\mathbf{r}_1 - \mathbf{L}_{s-1} \cdot \mathbf{G}^{-1}(\mathbf{t}))$$

where $\mathbf{t} = (\mathbf{B}_{s-1}^{\intercal} \otimes \mathbf{G}) \cdot \mathbf{r}_1 - \mathbf{Y} \cdot \mathbf{u}$. By the correctness of the shift-hiding function, we have that

$$\mathbf{r}_1 = \mathsf{SH}_1.\mathsf{Eval}(K_1, (\mathbf{Y}, \mathsf{aux})) + \mathbf{L}_{s-1} \cdot \mathbf{G}^{-1}(\overline{\mathbf{t}}) - \mathbf{G}^{-1}(\mathsf{vec}(\mathbf{M}_0 \cdot \mathbf{K}_{\mathbf{M}_0,\mathbf{Y}} \cdot \mathbf{V} + \mathbf{M}_1)) \quad (23)$$

with $\overline{\mathbf{t}} = \mathsf{SH}.\mathsf{Eval}(K, (\mathbf{Y}, \mathsf{aux}))$. In fact Equation (23) holds over the integers (without modular reduction) since $\mathsf{SH}_1.\mathsf{Eval}(K_1, \cdot)$ is $(mnt^s + 1)$-intractable and

$$\|\mathbf{L}_{s-1} \cdot \mathbf{G}^{-1}(\overline{\mathbf{t}}) - \mathbf{G}^{-1}(\mathsf{vec}(\mathbf{M}_0 \cdot \mathbf{K}_{\mathbf{M}_0,\mathbf{Y}} \cdot \mathbf{V} + \mathbf{M}_1))\|_\infty \leq mnt^s + 1.$$

Moreover, by the correctness of SH_2, we have that

$$\mathbf{u} = \overline{\mathbf{u}} - \mathbf{T} \cdot \mathbf{G}^{-1}(\tilde{\mathbf{t}}) \quad (24)$$

where $\tilde{\mathbf{t}} = \mathsf{SH}.\mathsf{ShiftEval}(K', (\mathbf{Y}, \mathsf{aux}))$ and $\overline{\mathbf{u}} = \mathsf{SH}_2.\mathsf{Eval}(K_2, (\mathbf{Y}, \mathsf{aux}))$. Again, Equation (24) holds over the integers (no modular reduction) since $\mathsf{SH}_2.\mathsf{Eval}(K_2, \cdot)$ is mnc^s-intractable and

$$\|\mathbf{T} \cdot \mathbf{G}^{-1}(\tilde{\mathbf{t}})\|_\infty \leq mnc^s.$$

Finally, by the correctness of SH, we have that

$$\tilde{\mathbf{t}} = \overline{\mathbf{t}} - (\mathbf{B}_{s-1}^{\intercal} \otimes \mathbf{G}) \cdot \mathbf{r}_1 + \mathbf{Y} \cdot \overline{\mathbf{u}}.$$

Then we can use Equation (24) to conclude that

$$\begin{aligned}
\overline{\mathbf{t}} &= \tilde{\mathbf{t}} + (\mathbf{B}_{s-1}^{\intercal} \otimes \mathbf{G}) \cdot \mathbf{r}_1 - \mathbf{Y} \cdot \overline{\mathbf{u}} \\
&= \tilde{\mathbf{t}} + (\mathbf{B}_{s-1}^{\intercal} \otimes \mathbf{G}) \cdot \mathbf{r}_1 - \mathbf{Y} \cdot \mathbf{u} - \mathbf{Y} \cdot \mathbf{T} \cdot \mathbf{G}^{-1}(\tilde{\mathbf{t}}) \\
&= \tilde{\mathbf{t}} + (\mathbf{B}_{s-1}^{\intercal} \otimes \mathbf{G}) \cdot \mathbf{r}_1 - \mathbf{Y} \cdot \mathbf{u} - (\mathbf{I}_{m \cdot c^s} \otimes \mathbf{G}) \cdot \mathbf{G}^{-1}(\tilde{\mathbf{t}}) \\
&= \tilde{\mathbf{t}} + (\mathbf{B}_{s-1}^{\intercal} \otimes \mathbf{G}) \cdot \mathbf{r}_1 - \mathbf{Y} \cdot \mathbf{u} - \tilde{\mathbf{t}} \\
&= (\mathbf{B}_{s-1}^{\intercal} \otimes \mathbf{G}) \cdot \mathbf{r}_1 - \mathbf{Y} \cdot \mathbf{u} \\
&= \mathbf{t}.
\end{aligned}$$

Substituting in Equation (23), we obtain

$$\mathbf{d} = (\mathbf{I}_m \otimes \mathbf{G}) \cdot \mathbf{w}_1 = (\mathbf{I}_m \otimes \mathbf{G}) \cdot \mathsf{SH}_1.\mathsf{Eval}(K_1, (\mathbf{Y}, \mathsf{aux})) - \mathsf{vec}(\mathbf{M}_0 \cdot \mathbf{K}_{\mathbf{M}_0,\mathbf{Y}} \cdot \mathbf{V} + \mathbf{M}_1)$$

which is precisely defined as in Hybrid 4.

Hybrid 5: In this hybrid, we change again how we compute $\mathbf{d}$. We set it to

$$\mathbf{d} = -\mathsf{vec}\left((\mathbf{M}_0 - \mathsf{Bits}(K_1)^{\intercal} \otimes \mathbf{G}) \cdot \mathbf{H}_{\mathbf{M}_0,\mathbf{Y},K_1} \cdot \mathbf{V} + \mathbf{M}_1\right),$$

where $\mathbf{H}_{\mathbf{M}_0,\mathbf{Y},K_1} := \mathsf{BGG.EvalH}(\mathbf{M}_0, K_1, \mathsf{SH}_1.\mathsf{Eval}(\cdot, (\mathbf{Y}, \mathsf{aux})))$. This hybrid is identical to Hybrid 4 by the key equation of BGG encodings [BGG+14]. Indeed,

$$\begin{aligned}
&(\mathbf{M}_0 - \mathsf{Bits}(K_1)^\intercal \otimes \mathbf{G}) \cdot \mathbf{H}_{\mathbf{M}_0,\mathbf{Y},K_1} \cdot \mathbf{V} \\
&\quad = (\mathbf{M}_0 \cdot \mathbf{K}_{\mathbf{M}_0,\mathbf{Y}} - \mathsf{SH}_1.\mathsf{Eval}(K_1, (\mathbf{Y}, \mathsf{aux}))^\intercal \otimes \mathbf{G}) \cdot \mathbf{V} \\
&\quad = \mathbf{M}_0 \cdot \mathbf{K}_{\mathbf{M}_0,\mathbf{Y}} \cdot \mathbf{V} - (\mathsf{SH}_1.\mathsf{Eval}(K_1, (\mathbf{Y}, \mathsf{aux}))^\intercal \otimes \mathbf{G}) \cdot \mathbf{V}.
\end{aligned}$$

Moreover, for any vector $\mathbf{v} \in \mathbb{Z}_q^{nm}$, we have that $\mathsf{vec}((\mathbf{v}^\intercal \otimes \mathbf{G}) \cdot \mathbf{V}) = (\mathbf{I}_m \otimes \mathbf{G}) \cdot \mathbf{v}$.

Hybrid 6: In this hybrid, we change the distribution of $\mathbf{M}_0$. Specifically, we sample a random $\mathbf{Q}$ and we set

$$\mathbf{M}_0 = \mathbf{Q} + \mathsf{Bits}(K_1)^\intercal \otimes \mathbf{G},$$

where $\mathsf{Bits}(K_1)$ is the representation of K_1 as a binary string. Moreover, we compute $\mathbf{d}$ as follows

$$\mathbf{d} = -\mathsf{vec}(\mathbf{Q} \cdot \mathbf{H}_{\mathbf{M}_0,\mathbf{Y},K_1} \cdot \mathbf{V} + \mathbf{M}_1).$$

This hybrid is perfectly indistinguishable from Hybrid 5.

Hybrid 7: In this hybrid, we change the way we compute $\mathbf{h}$. We set it to

$$\mathbf{h} = -(\mathbf{v}_0^\intercal \cdot \mathbf{H}_{\mathbf{M}_0,\mathbf{Y},K_1} \cdot \mathbf{V})^\intercal + \mathbf{v}_1 + \mathbf{e}_2$$

where

$$\mathbf{e}_0 \xleftarrow{\$} \mathcal{D}_\sigma^{n\ell_1} \qquad \mathbf{e}_1 \xleftarrow{\$} \mathcal{D}_\sigma^m \qquad \mathbf{e}_2 \xleftarrow{\$} \mathcal{D}_{\sigma'}^m \qquad \mathbf{v}_0^\intercal = \mathbf{s}^\intercal \cdot \mathbf{Q} + \mathbf{e}_0^\intercal \qquad \mathbf{v}_1^\intercal = \mathbf{s}^\intercal \cdot \mathbf{M}_1 + \mathbf{e}_1^\intercal.$$

Since $\sigma' = \sigma \cdot 2^{d_1 \cdot \omega(\log \lambda)}$, this hybrid is statistically indistinguishable from Hybrid 6 by a smudging argument (Lemma 2).

Hybrid 8: In this hybrid, we sample $\mathbf{v}_0 \xleftarrow{\$} \mathbb{Z}_q^{n\ell_1}$, $\mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^m$, $\mathbf{v}_1 \xleftarrow{\$} \mathbb{Z}_q^m$, and $\mathbf{C} \xleftarrow{\$} \mathbb{Z}_q^{t/n \times t}$. This hybrid is computationally indistinguishable from Hybrid 7 under the hardness of decomposed LWE. Indeed, by Equation (20), we have

$$\begin{aligned}
\mathbf{C}^\intercal &= \mathbf{U}^\intercal \cdot (\mathbf{I}_{t/n} \otimes \mathbf{s}) + \mathbf{E}^\intercal \\
&= (\mathbf{I}_t \otimes \mathbf{s}) \cdot \Pi_k(\mathbf{U}) + \mathbf{E}^\intercal \\
&= (\mathbf{I}_t \otimes \mathbf{s}) \cdot \Pi_k(\Pi_k(\mathbf{AB}) + \mathbf{I}_{t/n} \otimes \mathbf{G}) + \mathbf{E}^\intercal \\
&= (\mathbf{I}_t \otimes \mathbf{s}) \cdot (\mathbf{AB} + \Pi_k(\mathbf{I}_{t/n} \otimes \mathbf{G})) + \mathbf{E}^\intercal \\
&= (\mathbf{I}_t \otimes \mathbf{s}) \cdot (\mathbf{A}\tilde{\mathbf{B}} + \mathbf{G}) \cdot \mathbf{G}^{-1}(\Pi_k(\mathbf{I}_{t/n} \otimes \mathbf{G})) + \mathbf{E}^\intercal.
\end{aligned}$$

Note that $\mathbf{C}^\intercal$ can be simulated by multiplying $\mathbf{G}^{-1}(\Pi_k(\mathbf{I}_{t/n} \otimes \mathbf{G}))$ to a decomposed LWE sample and adding smudging noise, where statistical indistinguishability follows by a standard argument (Lemma 2).

Hybrid 9: In this hybrid, we sample $\mathbf{h} \xleftarrow{\$} \mathbb{Z}_q^m$. This hybrid is perfectly indistinguishable from Hybrid 8.

Hybrid 10: In this hybrid, we set $\tilde{K}_1 \xleftarrow{\$} \mathsf{SH}_1.\mathsf{Shift}(K_1, 0)$ and $\tilde{K}_2 \xleftarrow{\$} \mathsf{SH}_2.\mathsf{Shift}(K_2, 0)$. This hybrid is indistinguishable from Hybrid 9 by the shift-hiding properties of SH, SH_1 and SH_2 (to see it, reverse Hybrid 3, Hybrid 2 and Hybrid 1). Notice that at this point the game is identical to the case $b = 1$.

This ends the proof. □

6 Broadcast Encryption

In the following we show our main result, namely a construction of adaptively secure (distributed) broadcast encryption.

6.1 Definition

Broadcast encryption [FN94] allows one to broadcast a message to a set of N participants. The most basic notion [FN94] considers a central key generation algorithm that provides keys to all participants. *Distributed* broadcast [BZ14, KMW23] allows each party to sample their own public-secret key pair. More recently, *flexible* broadcast [FWW23] further extends this notion to remove parties identifiers. Here we show a formal definition for the latter.

Syntax. We recall the syntax of a flexible broadcast scheme.

- $\mathsf{Setup}(1^\lambda)$: On input the security parameter 1^λ, the setup algorithm returns a master public key mpk.
- $\mathsf{KeyGen}(\mathsf{mpk})$: On input the master public key mpk, the key generation algorithm returns a public-secret key pair $(\mathsf{pk}, \mathsf{sk})$.
- $\mathsf{Enc}(\mathsf{mpk}, \mathsf{pk}_0, \ldots, \mathsf{pk}_{N-1}, \mathsf{msg})$: On input the master public key mpk, a set of public keys $\mathsf{pk}_0, \ldots, \mathsf{pk}_{N-1}$, and a message $\mathsf{msg} \in \{0, 1\}$, the encryption algorithm returns a ciphertext ct.
- $\mathsf{Dec}(\mathsf{mpk}, k, \mathsf{sk}, \mathsf{ct}, \mathsf{pk}_0, \ldots, \mathsf{pk}_{N-1})$: On input the master public key mpk, the index k, the secret key sk, a ciphertext ct, and the public keys $\mathsf{pk}_0, \ldots, \mathsf{pk}_{N-1}$, the decryption algorithm returns the message msg.

For *correctness*, we require that for all $k \in [N]$ we have

$$\mathsf{Dec}(\mathsf{mpk}, k, \mathsf{sk}_k, \mathsf{Enc}(\mathsf{mpk}, \mathsf{pk}_0, \ldots, \mathsf{pk}_{N-1}, \mathsf{msg}), \mathsf{pk}_0, \ldots, \mathsf{pk}_{N-1}) = \mathsf{msg}$$

where $(\mathsf{sk}_k, \mathsf{pk}_k) \xleftarrow{\$} \mathsf{KeyGen}(\mathsf{mpk})$ and $\mathsf{mpk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$, except with negligible probability. For *efficiency*, we require that the ciphertext size should be bounded by a function that is (at most) poly-logarithmic in the number of users N.

Security. We define the following version of security, referred to as *semi-static* security [GW09, FWW23], as an experiment where the attacker is allowed to select an arbitrary set of honestly-sampled keys.

Definition 3 (Semi-Static Security)**.** *Consider the following experiment between a PPT attacker and a challenger.*

- *(Setup Phase) The challenger samples the master public key* $\mathsf{mpk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$ *and sends it to the attacker.*
- *(Query Phase) The attacker can issue polynomially-many queries to the challenger, who executes* $(\mathsf{sk}, \mathsf{pk}) \xleftarrow{\$} \mathsf{KeyGen}(\mathsf{mpk})$ *and returns* pk *to the attacker.*
- *(Challenge Phase) The attacker outputs a set of keys* $\mathsf{pk}_0, \ldots, \mathsf{pk}_{N-1}$ *and the challenger checks, for all* $k \in [N]$*, that* pk_k *is part of the keys sent in the query phase. The challenger aborts if this is not the case.*

  Otherwise, the challenger samples a random bit $b \xleftarrow{\$} \{0, 1\}$ *and returns*

  $$\mathsf{ct} \xleftarrow{\$} \mathsf{Enc}(\mathsf{mpk}, \mathsf{pk}_0, \ldots, \mathsf{pk}_{N-1}, b)$$

  to the attacker.
- *(Output Phase) The attacker returns a bit* b' *and wins if* $b = b'$*.*

We say that a flexible broadcast is semi-statically secure if for all PPT attackers the probability that they win in the above experiment is negligibly close to $\frac{1}{2}$*.*

A stronger notion of security, called *adaptive* security, is defined as above except that the attacker is allowed to corrupt keys sampled in the query phase, and receive the corresponding secret key. The challenger should additionally check that the challenge set does not contain any corrupted public key. In [GW09] it is shown that any flexible broadcast satisfying semi-static security can be turned into one satisfying adaptive security in the random oracle model. In [HWW25] they show a standard model transformation for the case of distributed broadcast. Since flexible broadcast implies distributed broadcast, the following allows us to concentrate on the notion of semi-static security.

Theorem 6 ([HWW25])**.** *Assuming the hardness of the LWE problem and a distributed broadcast satisfying semi-static security, then there exists a distributed broadcast satisfying adaptive security.*

6.2 Our Scheme

We are now ready to formally present our scheme. Let Comp denote the already derandomized succinct LWE sampler, i.e., we generate the randomness for the hashing algorithm using $\mathsf{SH}_1, \mathsf{SH}_2$ as in Definition 2. For this particular application, the auxiliary information aux

(see Definition 2) is going to be empty, so in a slight abuse of notation we drop this variable as input to the algorithms. Let q be a modulus such that

$$q = O\left((ctmn\sigma\sqrt{\lambda})^{2s+1} + m\sigma\sqrt{\lambda}np\right)$$

where p is the modulus used in SH_1 and SH_2.

- $\mathsf{Setup}(1^\lambda)$: Sample $K_1 \xleftarrow{\$} \mathsf{SH}_1.\mathsf{Setup}(1^\lambda, 1^\eta)$ and $K_2 \xleftarrow{\$} \mathsf{SH}_2.\mathsf{Setup}(1^\lambda, 1^\eta)$ and compute

  $$\mathsf{hk} \xleftarrow{\$} \mathsf{Comp.Setup}(1^\lambda) \quad \tilde{K}_1 \xleftarrow{\$} \mathsf{SH}_2.\mathsf{Shift}(K_1, 0) \quad \tilde{K}_2 \xleftarrow{\$} \mathsf{SH}_2.\mathsf{Shift}(K_2, 0) \quad \mathbf{D} \xleftarrow{\$} \mathbb{Z}_q^{k\times m}$$

  and output the master public key $\mathsf{mpk} := (\mathsf{hk}, \tilde{K}_1, \tilde{K}_2, \mathbf{D})$.

- $\mathsf{KeyGen}(\mathsf{mpk})$: Compute

  $$\mathbf{Q} = \mathbf{D} \cdot \mathbf{R}$$

  where $\mathbf{R} \xleftarrow{\$} \mathbb{Z}_2^{m\times n}$. Output $\mathsf{pk} = \mathbf{Q}$ and $\mathsf{sk} = \mathbf{R}$.

- $\mathsf{Enc}(\mathsf{mpk}, \mathsf{pk}_0, \ldots, \mathsf{pk}_{N-1}, \mathsf{msg})$: Define the matrix

  $$\mathbf{Y} = \begin{bmatrix} \mathbf{Q}_0 & \mathbf{D} & & \\ & & \ddots & \\ & & & \mathbf{Q}_{N-1} & \mathbf{D} \end{bmatrix}$$

  and compute

  $$\mathbf{r}_1 = \mathsf{SH}_1.\mathsf{ShiftEval}(\tilde{K}_1, \mathbf{Y}) \qquad \text{and} \qquad \mathbf{u} = \mathsf{SH}_2.\mathsf{ShiftEval}(\tilde{K}_2, \mathbf{Y}).$$

  Then set

  $$\mathbf{c}^\intercal = \mathbf{s}^\intercal\mathbf{D} + \mathbf{e}^\intercal + \mathsf{msg} \cdot \mathbf{t}^\intercal \qquad \text{and} \qquad \tilde{\mathbf{r}} = \begin{bmatrix} \mathbf{0} \\ \mathbf{r}_1 \end{bmatrix}$$

  where $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^k$, $\mathbf{t} \xleftarrow{\$} \mathbb{Z}_q^m$, and $\mathbf{e} \xleftarrow{\$} \mathcal{D}_\sigma^m$. Derive the matrix $\mathbf{X} \in \mathbb{Z}_q^{k\times N}$ such that $\mathsf{vec}(\mathbf{X}) = \mathbf{Y} \cdot \mathbf{u}$ and compute

  $$(\mathbf{d}, \mathsf{ek}) = \mathsf{Comp.Hash}(\mathsf{hk}, \mathbf{X}, \tilde{\mathbf{r}}) \qquad (\mathbf{C}, \mathbf{h}) \xleftarrow{\$} \mathsf{Comp.Gen}(\mathsf{hk}, \mathbf{d}, \mathbf{s}).$$

  Output $\mathsf{ct} = (\mathbf{C}, \mathbf{h}, \mathbf{c})$.

- $\mathsf{Dec}(\mathsf{mpk}, i, \mathsf{sk}, \mathsf{ct}, \mathsf{pk}_0, \ldots, \mathsf{pk}_{N-1})$: Recompute $(\mathbf{d}, \mathsf{ek})$ as in the Enc algorithm, which are deterministically derived from the public keys, then expand

  $$\mathbf{v} = \mathsf{Comp.Expand}(\mathsf{hk}, \mathsf{ek}, \mathbf{C}, \mathbf{h}).$$

Let $\mathbf{v}_i$ denote the i-th entry of $\mathbf{v}$. Parse

$$\mathbf{u} = \begin{bmatrix} \mathbf{u}_0 \\ \vdots \\ \mathbf{u}_{N-1} \end{bmatrix}, \qquad \mathbf{u}_i = \begin{bmatrix} \mathbf{u}_i^{(0)} \\ \mathbf{u}_i^{(1)} \end{bmatrix}$$

where $\mathbf{u}_i^{(0)} \in \mathbb{Z}_q^n$ and $\mathbf{u}_i^{(1)} \in \mathbb{Z}_q^m$. Finally, return 0 if

$$\left\| \mathbf{v}_i - \mathbf{c}^\intercal \mathbf{R}_i \mathbf{u}_i^{(0)} - \mathbf{c}^\intercal \mathbf{u}_i^{(1)} \right\|_\infty \leq 3(ctmn\sigma\sqrt{\lambda})^{2s+1} + m\sigma\sqrt{\lambda} \cdot (n+1) \cdot p$$

and 1 otherwise.

To see that the scheme satisfies correctness, observe that, by Lemma 15, we can rewrite

$$\begin{aligned} \mathbf{v}_i &= \mathbf{s}^\intercal \cdot \begin{bmatrix} \mathbf{Q}_i & \mathbf{D} \end{bmatrix} \cdot \mathbf{u}_i + \hat{\mathbf{e}}^\intercal \\ &= \mathbf{s}^\intercal \mathbf{Q}_i \mathbf{u}_i^{(0)} + \mathbf{s}^\intercal \mathbf{D} \mathbf{u}_i^{(1)} + \hat{\mathbf{e}}^\intercal \\ &= \mathbf{s}^\intercal \mathbf{D} \mathbf{R}_i \mathbf{u}_i^{(0)} + \mathbf{s}^\intercal \mathbf{D} \mathbf{u}_i^{(1)} + \hat{\mathbf{e}}^\intercal. \end{aligned}$$

Substituting the definition of $\mathbf{c}$, we can see that

$$\begin{aligned} \mathbf{v}_i - \mathbf{c}^\intercal \mathbf{R}_i \mathbf{u}_i^{(0)} - \mathbf{c}^\intercal \mathbf{u}_i^{(1)} &= \mathbf{v}_i - \mathbf{s}^\intercal \mathbf{D} \mathbf{R}_i \mathbf{u}_i^{(0)} - \mathbf{s}^\intercal \mathbf{D} \mathbf{u}_i^{(1)} - \mathbf{e}^\intercal (\mathbf{R}_i \mathbf{u}_i^{(0)} + \mathbf{u}_i^{(1)}) - \mathsf{msg} \cdot \mathbf{t}^\intercal (\mathbf{R}_i \mathbf{u}_i^{(0)} + \mathbf{u}_i^{(1)}) \\ &= \hat{\mathbf{e}}^\intercal - \mathbf{e}^\intercal (\mathbf{R}_i \mathbf{u}_i^{(0)} + \mathbf{u}_i^{(1)}) - \mathsf{msg} \cdot \mathbf{t}^\intercal (\mathbf{R}_i \mathbf{u}_i^{(0)} + \mathbf{u}_i^{(1)}). \end{aligned}$$

If $\mathsf{msg} = 0$, then it suffices to bound

$$\left\| \hat{\mathbf{e}}^\intercal - \mathbf{e}^\intercal (\mathbf{R}_i \mathbf{u}_i^{(0)} + \mathbf{u}_i^{(1)}) \right\|_\infty \leq 3(ctmn\sigma\sqrt{\lambda})^{2s+1} + m\sigma\sqrt{\lambda} \cdot (n+1) \cdot p.$$

On the other hand, if $\mathsf{msg} = 1$, the term $\mathbf{v}_i - \mathbf{c}^\intercal \mathbf{R}_i \mathbf{u}_i^{(0)} - \mathbf{c}^\intercal \mathbf{u}_i^{(1)}$ is uniformly distributed, so for large enough q, decryption outputs 1 with probability > 0.9. The probability of an incorrect decryption can be driven to negligible by encrypting the message multiple times and taking a majority vote.

For efficiency, it is easy to see that the size of the ciphertext, of the public/secret keys pairs, and of the master public key are bounded by a function poly-logarithmic in the number of users N.

Before proving security, we state a lemma that is going to be useful for our analysis.

Lemma 16 (Key Indistinguishability)**.** *The following distributions are statistically close*

$$\left\{ \mathbf{D}, \mathbf{DR} : \mathbf{D} \xleftarrow{\$} \mathbb{Z}_q^{k \times m}, \mathbf{R} \xleftarrow{\$} \mathbb{Z}_2^{m \times n} \right\} \approx_s \left\{ \mathbf{D}, \mathbf{DR} + \mathbf{G} : \mathbf{D} \xleftarrow{\$} \mathbb{Z}_q^{k \times m}, \mathbf{R} \xleftarrow{\$} \mathbb{Z}_2^{m \times n} \right\}.$$

Proof. Follows as a direct consequence of Lemma 1. □

We are now ready to state our main theorem of this section.

Theorem 7 (Semi-Static Security)**.** *If the decomposed LWE assumption holds, the scheme as described above satisfies semi-static security (Definition 3).*

Proof. By Theorem 5 it suffices to show an admissible adversary, i.e., such that we can extract a valid trapdoor $\mathbf{T}_i$ for all matrices $\begin{bmatrix}\mathbf{Q}_i & \mathbf{D}\end{bmatrix}$ with $i \in [N]$. Then, by Definition 2, the ciphertext is computationally indistinguishable from uniform.

First, let us consider a modified experiment where, for all key queries, we sample keys as $\mathbf{Q}_i = \mathbf{D}\mathbf{R}_i + \mathbf{G}$. By Lemma 16, the view of this modified experiment is statistically close to the original one. Note that in this modified experiment it holds that

$$\begin{bmatrix}\mathbf{Q}_i & \mathbf{D}\end{bmatrix} \cdot \underbrace{\begin{bmatrix}\mathbf{I}_n \\ -\mathbf{R}_i\end{bmatrix}}_{\mathbf{T}_i} = \mathbf{G}$$

with $\|\mathbf{T}_i\|_\infty \leq 1$. Thus, $\mathbf{T}_i$ is a valid trapdoor. Second, we modify the experiment, and we sample each $\mathbf{R}_i$ using a regular $\mathsf{PRF}(\tau, i)$, for a random τ. This change is indistinguishable by the pseudorandomness of the PRF.

Note that τ is a compact information that, together with the public keys, allows us to recompute in polynomial time all matrices $\mathbf{T}_i$ corresponding to the challenge set. This allows us to define a polynomial time $\mathsf{Extract}$ function, which outputs a valid trapdoor, thus making the adversary admissible. This ends the proof. □

7 Pseudorandom Obfuscation

An obfuscation scheme [BGI+01] consists of a PPT algorithm Obf that outputs an obfuscated version of an input function $\tilde{f} \xleftarrow{\$} \mathsf{Obf}(1^\lambda, f)$, such that for any input x it holds that

$$\tilde{f}(x) = f(x).$$

The standard notion of security for an obfuscation scheme is indistinguishability obfuscation (iO) [BGI+01].

Definition 4 (Indistinguishability Obfuscation)**.** *An obfuscation scheme* Obf *satisfies indistinguishability obfuscation if, for all functionally equivalent* $f_0 \equiv f_1$*, it holds that*

$$\left\{\tilde{f} : \tilde{f} \xleftarrow{\$} \mathsf{Obf}(1^\lambda, f_0)\right\} \approx_c \left\{\tilde{f} : \tilde{f} \xleftarrow{\$} \mathsf{Obf}(1^\lambda, f_1)\right\}.$$

More recently [BDJ+25] put forward the notion of *pseudorandom obfuscation* for keyed functions. The security of the primitive postulates that the obfuscation itself should be computationally indistinguishable from uniform.

Definition 5 (Pseudorandom Obfuscation)**.** *An obfuscation scheme* Obf *satisfies pseudorandomness if, for all functions* f_k *such that*

$$\left\{f_k(x) : k \xleftarrow{\$} \{0,1\}^\lambda\right\}_x \approx_c \left\{u_x : u_x \xleftarrow{\$} \{0,1\}^*\right\}_x$$

then it holds that

$$\left\{\tilde{f} : \tilde{f} \xleftarrow{\$} \mathsf{Obf}(1^\lambda, f_k), k \xleftarrow{\$} \{0,1\}^\lambda\right\} \approx_c \left\{U : U \xleftarrow{\$} \{0,1\}^*\right\}.$$

In [BDJ+25] it is shown that such a notion is subject to strong impossibility results for general functions f. Here we consider a special class of functions, that we refer to as *puncturable* functions [SW14]. We say that a keyed function is puncturable if there exists an algorithm $\mathsf{Puncture}$ such that for all $k \in \{0,1\}^\lambda$ and all $x \neq x^*$ it holds that

$$f_k(x) = f_{k_{x^*}}(x)$$

where $k \xleftarrow{\$} \{0,1\}^\lambda$ and $k_{x^*} = \mathsf{Puncture}(k, x^*)$. Furthermore, for all $x^* \in \{0,1\}^d$ we require that

$$\left\{f_k(x^*), \mathsf{Puncture}(k, x^*) : k \xleftarrow{\$} \{0,1\}^\lambda\right\} \approx_c \left\{u, \mathsf{Puncture}(k, x^*) : k \xleftarrow{\$} \{0,1\}^\lambda, u \xleftarrow{\$} \{0,1\}^*\right\}.$$

In this section we show how to turn an iO for puncturable PRFs into a pseudorandom one, assuming our laconic shift-hiding PRF[5] from Section 4. For notational convenience, we are going to assume that the shift algorithm inputs a function instead of a string. We describe the scheme in the following.

- $\mathsf{PRObf}(1^\lambda, f_k)$: Sample a key $K \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^d)$ and let $g_K(x)$ be the function that computes
$$g_{K,k}(x) = f_k(x) - \mathsf{Eval}(K, x).$$
Define $\tilde{g} \xleftarrow{\$} \mathsf{Obf}(1^\lambda, g_K)$ be an iO obfuscation of g_K. Return $K_{\tilde{g}} = \mathsf{Shift}(K, \tilde{g})$.

We claim that evaluating the shifted key returns the correct output. This can be easily established by observing that, by Lemma 11, we have

$$\mathsf{ShiftEval}(K_{\tilde{g}}, x) = \mathsf{Eval}(K, x) + \tilde{g}(x) = \mathsf{Eval}(K, x) + g_{K,k}(x) = f_k(x)$$

except with negligible probability. Note that in order for the construction to be well-defined, we require that the circuit depth of the shifted function $\tilde{g}$ should be bounded by a fixed polynomial in the security parameter (since the complexity of the shift-hiding PRF depends on the *depth* of the shifted function). This can be achieved by combining our construction from Section 4 with an iO where the depth of the evaluation circuit grows polynomially with the *depth* of the underlying function. In turn, this can be generically achieved using standard techniques [BDJ+25] (e.g., obfuscating a circuit that returns a randomized encoding of the output). All is left to be shown is that the construction described above satisfies pseudorandomness.

[5] Here we assume that the parameters p and q as defined in Section 4 are powers of 2 so that uniform elements of $\mathbb{Z}_p$ and $\mathbb{Z}_q$ can be canonically mapped to uniform bitstring. This is purely for notational convenience, and it is straightforward to extend the definition and construction to different moduli. We also assume a canonical invertible map from discrete Gaussians to random strings, and we refer the reader to [BDJ+25] for a precise description.

Theorem 8 (Pseudorandomness)**.** *If the sub-exponential decomposed LWE assumption holds and* Obf *is a sub-exponentially secure iO, the scheme as described above satisfies pseudorandomness (Definition 5) for all puncturable functions.*

Proof. We prove security via an sequence of hybrid experiments, where we gradually change the distribution of $\mathsf{PRO}(1^\lambda, f_k)$. For each $x^* \in \{0,1\}^d$, we define the following hybrids.

Hybrid 0: This is the original distribution where we obfuscate the function g.

Hybrid 1: We compute the punctured key $k_{x^*} = \mathsf{Puncture}(k, x^*)$ and obfuscate the function $g'_{K,k}$ defined as

$$g'_{K,k}(x) = \begin{cases} g_{K,k_{x^*}}(x) & \text{if } x \neq x^* \\ f_k(x^*) - \mathsf{Eval}(K, x^*) & \text{otherwise.} \end{cases}$$

Note that the this function is equivalent to the previous hybrid ($g'_{K,k} \equiv g_{K,k}$), so indistinguishability follows by Definition 4.

Hybrid 2: We sample a random string u and we define

$$g'_{K,k_{x^*},u}(x) = \begin{cases} g_{K,k_{x^*}}(x) & \text{if } x \neq x^* \\ u - \mathsf{Eval}(K, x^*) & \text{otherwise.} \end{cases}$$

This change is computationally indistinguishable by the puncturable pseudorandomness of f_k.

Hybrid 3: We redefine

$$g'_{K,k_{x^*},u}(x) = \begin{cases} g_{K,k_{x^*}}(x) & \text{if } x \neq x^* \\ u & \text{otherwise.} \end{cases}$$

Since u is uniform, this change is only syntactical.

Hybrid 4: We revert the modification done in the second hybrid and set

$$g'_{K,k}(x) = \begin{cases} g_{K,k_{x^*}}(x) & \text{if } x \neq x^* \\ f_k(x^*) & \text{otherwise.} \end{cases}$$

Indistinguishability follows with the same argument.

Hybrid 5: We revert the modification done in the first hybrid and set

$$g'_{K,k}(x) = \begin{cases} g_{K,k}(x) & \text{if } x \neq x^* \\ f_k(x^*) & \text{otherwise.} \end{cases}$$

Indistinguishability follows with the same argument.

Observe that, after all the hybrids (for all $x \in \{0,1\}^d$, in lexicographical order) the function $g_{K,k} \equiv f_k$ is independent of K and consequently so is its obfuscation $\tilde{g}$. Thus, by the pseudorandomness of the shift-hiding PRF (refer to Lemma 12) we have that $\mathsf{Shift}(K, \tilde{g}) = \mathsf{PRObf}(1^\lambda, f_k)$ is computationally close to uniform. □

Acknowledgments

A.D. was supported by a fellowship with the Simons Institute for the Theory of Computing. This work was supported by Input Output (iog.io) through their funding of the Edinburgh Blockchain Technology Lab. G.M. is supported by the European Research Council through an ERC Starting Grant (Grant agreement No. 101077455, ObfusQation) and by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy - EXC 2092 CASA – 390781972. L.R. was supported by the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme under grant agreement number 101124977 (DECRYPSIS). Part of this work was conducted while the authors were visiting the Simons Institute for the Theory of Computing.

References

[AJ15] Prabhanjan Ananth and Abhishek Jain. Indistinguishability obfuscation from compact functional encryption. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part I*, volume 9215 of *LNCS*, pages 308–326. Springer, Berlin, Heidelberg, August 2015.

[Ajt96] Miklós Ajtai. Generating hard instances of lattice problems (extended abstract). In *28th ACM STOC*, pages 99–108. ACM Press, May 1996.

[AMR25a] Damiano Abram, Giulio Malavolta, and Lawrence Roy. Key-homomorphic computations for RAM: Fully succinct randomised encodings and more. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 236–268. Springer, Cham, August 2025.

[AMR25b] Damiano Abram, Giulio Malavolta, and Lawrence Roy. Succinct oblivious tensor evaluation and applications: Adaptively-secure laconic function evaluation and trapdoor hashing for all circuits. In Michal Koucký and Nikhil Bansal, editors, *57th ACM STOC*, pages 1875–1886. ACM Press, June 2025.

[Ban93] Wojciech Banaszczyk. New bounds in some transference theorems in the geometry of numbers. *Mathematische Annalen*, 296:625–635, 1993.

[BDGM22] Zvika Brakerski, Nico Döttling, Sanjam Garg, and Giulio Malavolta. Factoring and pairings are not necessary for IO: Circular-secure LWE suffices. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, *ICALP 2022*, volume 229 of *LIPIcs*, pages 28:1–28:20. Schloss Dagstuhl, July 2022.

[BDJ+25] Pedro Branco, Nico Döttling, Abhishek Jain, Giulio Malavolta, Surya Mathialagan, Spencer Peters, and Vinod Vaikuntanathan. Pseudorandom obfuscation and applications. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part V*, volume 16004 of *LNCS*, pages 663–698. Springer, Cham, August 2025.

[BGG+14] Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 533–556. Springer, Berlin, Heidelberg, May 2014.

[BGI+01] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. On the (im)possibility of obfuscating programs. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, August 2001.

[BLW17] Dan Boneh, Kevin Lewi, and David J. Wu. Constraining pseudorandom functions privately. In Serge Fehr, editor, *PKC 2017, Part II*, volume 10175 of *LNCS*, pages 494–524. Springer, Berlin, Heidelberg, March 2017.

[BTVW17] Zvika Brakerski, Rotem Tsabary, Vinod Vaikuntanathan, and Hoeteck Wee. Private constrained PRFs (and more) from LWE. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 264–302. Springer, Cham, November 2017.

[BÜW24] Chris Brzuska, Akin Ünal, and Ivy K. Y. Woo. Evasive LWE assumptions: Definitions, classes, and counterexamples. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 418–449. Springer, Singapore, December 2024.

[BV11] Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) LWE. In Rafail Ostrovsky, editor, *52nd FOCS*, pages 97–106. IEEE Computer Society Press, October 2011.

[BV15a] Nir Bitansky and Vinod Vaikuntanathan. Indistinguishability obfuscation from functional encryption. In Venkatesan Guruswami, editor, *56th FOCS*, pages 171–190. IEEE Computer Society Press, October 2015.

[BV15b] Zvika Brakerski and Vinod Vaikuntanathan. Constrained key-homomorphic PRFs from standard lattice assumptions - or: How to secretly embed a circuit in your PRF. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015, Part II*, volume 9015 of *LNCS*, pages 1–30. Springer, Berlin, Heidelberg, March 2015.

[BW06] Dan Boneh and Brent Waters. A fully collusion resistant broadcast, trace, and revoke system. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006*, pages 211–220. ACM Press, October / November 2006.

[BZ14] Dan Boneh and Mark Zhandry. Multiparty key exchange, efficient traitor tracing, and more from indistinguishability obfuscation. In Juan A. Garay and Rosario

Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 480–499. Springer, Berlin, Heidelberg, August 2014.

[CGH98] Ran Canetti, Oded Goldreich, and Shai Halevi. The random oracle methodology, revisited (preliminary version). In *30th ACM STOC*, pages 209–218. ACM Press, May 1998.

[CHW25] Jeffrey Champion, Yao-Ching Hsieh, and David J. Wu. Registered ABE and adaptively-secure broadcast encryption from succinct LWE. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 3–34. Springer, Cham, August 2025.

[CJJ22] Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. SNARGs for $\mathcal{P}$ from LWE. In *62nd FOCS*, pages 68–79. IEEE Computer Society Press, February 2022.

[dCP23] Leo de Castro and Chris Peikert. Functional commitments for all functions, with transparent setup and from SIS. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part III*, volume 14006 of *LNCS*, pages 287–320. Springer, Cham, April 2023.

[DJM+25] Nico Döttling, Abhishek Jain, Giulio Malavolta, Surya Mathialagan, and Vinod Vaikuntanathan. Simple and general counterexamples for private-coin evasive LWE. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 73–92. Springer, Cham, August 2025.

[FN94] Amos Fiat and Moni Naor. Broadcast encryption. In Douglas R. Stinson, editor, *CRYPTO'93*, volume 773 of *LNCS*, pages 480–491. Springer, Berlin, Heidelberg, August 1994.

[FWW23] Cody Freitag, Brent Waters, and David J. Wu. How to use (plain) witness encryption: Registered ABE, flexible broadcast, and more. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 498–531. Springer, Cham, August 2023.

[Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st ACM STOC*, pages 169–178. ACM Press, May / June 2009.

[GKW18] Rishab Goyal, Venkata Koppula, and Brent Waters. Collusion resistant traitor tracing from learning with errors. In Ilias Diakonikolas, David Kempe, and Monika Henzinger, editors, *50th ACM STOC*, pages 660–670. ACM Press, June 2018.

[GP21] Romain Gay and Rafael Pass. Indistinguishability obfuscation from circular security. In Samir Khuller and Virginia Vassilevska Williams, editors, *53rd ACM STOC*, pages 736–749. ACM Press, June 2021.

[GSW13] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 75–92. Springer, Berlin, Heidelberg, August 2013.

[GVW13] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Attribute-based encryption for circuits. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 545–554. ACM Press, June 2013.

[GVW15a] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Predicate encryption for circuits from LWE. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 503–523. Springer, Berlin, Heidelberg, August 2015.

[GVW15b] Sergey Gorbunov, Vinod Vaikuntanathan, and Daniel Wichs. Leveled fully homomorphic signatures from standard lattices. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *47th ACM STOC*, pages 469–477. ACM Press, June 2015.

[GW09] Craig Gentry and Brent Waters. Adaptive security in broadcast encryption systems (with short ciphertexts). In Antoine Joux, editor, *EUROCRYPT 2009*, volume 5479 of *LNCS*, pages 171–188. Springer, Berlin, Heidelberg, April 2009.

[HWW25] Yao-Ching Hsieh, Brent Waters, and David J. Wu. A generic approach to adaptively-secure broadcast encryption in the plain model. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part III*, volume 15603 of *LNCS*, pages 336–365. Springer, Cham, May 2025.

[ILL89] Russell Impagliazzo, Leonid A. Levin, and Michael Luby. Pseudo-random generation from one-way functions (extended abstracts). In *21st ACM STOC*, pages 12–24. ACM Press, May 1989.

[KMW23] Dimitris Kolonelos, Giulio Malavolta, and Hoeteck Wee. Distributed broadcast encryption from bilinear groups. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part V*, volume 14442 of *LNCS*, pages 407–441. Springer, Singapore, December 2023.

[LV22] Alex Lombardi and Vinod Vaikuntanathan. Correlation-intractable hash functions via shift-hiding. In Mark Braverman, editor, *ITCS 2022*, volume 215, pages 102:1–102:16. LIPIcs, January / February 2022.

[MP12] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 700–718. Springer, Berlin, Heidelberg, April 2012.

[PS18] Chris Peikert and Sina Shiehian. Privately constraining and programming PRFs, the LWE way. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part II*, volume 10770 of *LNCS*, pages 675–701. Springer, Cham, March 2018.

[PS19] Chris Peikert and Sina Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part I*, volume 11692 of *LNCS*, pages 89–114. Springer, Cham, August 2019.

[QWW18] Willy Quach, Hoeteck Wee, and Daniel Wichs. Laconic function evaluation and applications. In Mikkel Thorup, editor, *59th FOCS*, pages 859–870. IEEE Computer Society Press, October 2018.

[Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th ACM STOC*, pages 84–93. ACM Press, May 2005.

[SW14] Amit Sahai and Brent Waters. How to use indistinguishability obfuscation: deniable encryption, and more. In David B. Shmoys, editor, *46th ACM STOC*, pages 475–484. ACM Press, May / June 2014.

[Tsa19] Rotem Tsabary. Fully secure attribute-based encryption for t-CNF from LWE. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part I*, volume 11692 of *LNCS*, pages 62–85. Springer, Cham, August 2019.

[Tsa22] Rotem Tsabary. Candidate witness encryption from lattice techniques. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 535–559. Springer, Cham, August 2022.

[VWW22] Vinod Vaikuntanathan, Hoeteck Wee, and Daniel Wichs. Witness encryption and null-IO from evasive LWE. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part I*, volume 13791 of *LNCS*, pages 195–221. Springer, Cham, December 2022.

[Wat09] Brent Waters. Dual system encryption: Realizing fully secure IBE and HIBE under simple assumptions. In Shai Halevi, editor, *CRYPTO 2009*, volume 5677 of *LNCS*, pages 619–636. Springer, Berlin, Heidelberg, August 2009.

[Wee24] Hoeteck Wee. Circuit ABE with $\mathsf{poly}(\mathrm{depth}, \lambda)$-sized ciphertexts and keys from lattices. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part III*, volume 14922 of *LNCS*, pages 178–209. Springer, Cham, August 2024.

[Wee25] Hoeteck Wee. Almost optimal KP and CP-ABE for circuits from succinct LWE. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part III*, volume 15603 of *LNCS*, pages 34–62. Springer, Cham, May 2025.

[WW21] Hoeteck Wee and Daniel Wichs. Candidate obfuscation via oblivious LWE sampling. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part III*, volume 12698 of *LNCS*, pages 127–156. Springer, Cham, October 2021.

[WW25] Hoeteck Wee and David J. Wu. Unbounded distributed broadcast encryption and registered ABE from succinct LWE. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 204–235. Springer, Cham, August 2025.